МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
им. М.В. Ломоносова

На правах рукописи

СТЕПАНОВ Евгений Павлович

ИССЛЕДОВАНИЕ МЕТОДОВ МНОГОПОТОЧНОЙ МАРШРУТИЗАЦИИ ДЛЯ ОБЕСПЕЧЕНИЯ КАЧЕСТВА СЕТЕВОГО СЕРВИСА

Специальность 05.13.11 – Математическое и программное обеспечение вычислительных машин, комплексов и компьютерных сетей

ДИССЕРТАЦИЯ

на соискание ученой степени кандидата физико-математических наук

Научный руководитель:
доктор физ.-мат. наук,
член-корр. РАН,
профессор Р. Л. Смелянский

Дубна – 2022

Оглавление

Введение **3**
Глава 1. Методы многопоточной маршрутизации **19**
1.1 Анализ работ по эффективности многопоточной маршрутизации 22
1.2 MPTCP 25
1.3 FDMP 29
1.4 Многопоточная маршрутизация в ПКС 37
1.5 Выводы 55
Глава 2. Сравнительный анализ эффективности статического и динамического методов многопоточной маршрутизации **57**
2.1 Методика проведения сравнения статического и динамического методов многопоточной маршрутизации 58
2.1.1 Описание входных и выходных параметров эксперимента 58
2.1.2 Критерии сравнения 60
2.2 Описание Стенда 61
2.3 Область допустимых значений параметров эксперимента 65
2.4 Результаты экспериментального сравнения 67
Глава 3. Применение многопоточной маршрутизации для реализации услуги «Пропускная способность по требованию» **74**
3.1 Варианты постановок задачи ПСТ 75
3.1.1 Постановка задачи ПСТ с одновременным приходом запросов. 75
3.1.2 Постановка задачи ПСТ при случайном потоке запросов с заданным распределением 79
3.1.3 Постановка задачи ПСТ при случайном потоке запросов с заданным распределением с определением недостающих сетевых ресурсов. 81
3.2 Метод решения задачи ПСТ 83
3.3 Экспериментальное исследование предложенного подхода 89
3.3.1 Методика проведения экспериментального исследования 89
3.3.2 Результаты экспериментального исследования 96
Заключение **98**
Список литературы **100**
Приложение А. Методика проверки статистической гипотезы о числовом значении вероятности события **108**
Приложение Б. Технические подробности реализации Стенда для проведения сравнительного анализа эффективности методов многопоточной маршрутизации **110**
Приложение В. Описание пакетов и их решателей для решения задачи целочисленного линейного программирования **112**

Введение

Требования приложений к скорости передачи данных растут постоянно. Так, согласно прогнозу компании Cisco Systems [1], объем видео-трафика в ближайшем будущем превысит текущий объем видео-трафика в десятки раз. Такое повышение требований связано, например, с распространением высококачественных камер наблюдения, телевизоров с разрешением 8K, а также с появлением приложений виртуальной реальности на основе потоковой передачи изображения.

Также следует отметить тенденцию к увеличению количества центров обработки данных (ЦОД). Согласно [2] в России на настоящий момент функционирует 146 ЦОД, а во всем мире их уже более 2500 [3]. Рост числа ЦОД связан с изменениями в организации приложений: происходит переход от клиент-серверной архитектуры, в которой усложняется поддержка приложения из-за монолитной серверной части, к микросервисной архитектуре [Salah T. et al.], элементы которой могут быть распределены как по ресурсам ЦОДа, так и по сети из ЦОД. Кроме того, часть вычислений может выполняться на периферии [Смелянский Р. Л.], позволяя снижать сетевые задержки на передачу информации, а также стали востребованы функции архивации и зеркалирования данных; банки и Интернет-магазины размещают в ЦОД сервера и системы хранения данных клиента [Zhang L. et al.]. Перечисленные изменения в организации приложений приводят не только к увеличению числа ЦОД и его клиентов, но и к увеличению трафика между ЦОД и клиентами. Приведенный тезис подтверждается исследованием TeleGeography [Telecommunications], которое показало, что в 2017 году доля трафика между ЦОД на самом популярном маршруте через Атлантический океан достигла 75% от общего трафика, а в 2023 году она должна превысить 93%.

Сказанное объясняет актуальность задачи повышения качества сетевого сервиса. Приведенные выше примеры демонстрируют тренд увеличения объема трафика в сети, поэтому в настоящей работе основное внимание уделено удовлетворению требований приложений к скорости транспортного соединения, в связи с чем, в диссертации термин ***качество сервиса*** трактуется как скорость

транспортного соединения. Под ***скоростью транспортного соединения*** (англ. ***throughput***) понимается объем переданных данных в единицу времени, который включает не только полезную нагрузку, но и все служебные данные: поля заголовков пакетов, переотправка потерянных пакетов и т.п. Под задачей ***обеспечения качества сервиса*** здесь будем понимать предоставление такого транспортного соединения, скорость которого не ниже требований приложения.

Можно выделить два класса методов обеспечения качества сервиса: экстенсивный и интенсивный. ***Экстенсивные методы*** обеспечения качества сервиса предполагают добавление дополнительной пропускной способности за счет добавления новых ресурсов. Например, прокладка дополнительных каналов в сети передачи данных или замена существующих каналов на более быстрые, замена или улучшение существующего каналообразующего оборудования в сети и т.п. Несмотря на то, что экстенсивные методы позволят достичь необходимых показателей качества сервиса, эти методы сильно увеличивают капитальные затраты на оборудование и операционные расходы на энергоснабжение, охлаждение, администрирование и т.д. Поэтому в настоящей работе экстенсивные методы обеспечения качества сервиса не рассматриваются.

Интенсивные методы обеспечения качества сервиса предполагают повышение эффективности использования существующих ресурсов сети. Можно выделить следующие группы интенсивных методов:

- Методы балансировки сетевого трафика;
- Методы многопоточной маршрутизации;
- Методы мультиплексирования сетевого трафика;
- Методы сжатия передаваемых данных;
- Методы управления скоростью передачи, например алгоритмы управления перегрузкой на транспортном уровне;
- Методы сегментирования транспортных соединений.

Методы балансировки сетевого трафика позволяют перераспределить нагрузку между альтернативными маршрутами в транспортной сети, в топологию

которой сознательно вносят, чтобы между любой парой точек было несколько маршрутов на случай непредвиденного отказа оборудования, тем самым повышая отказоустойчивость сети. Если представить топологию сети в виде графа, то избыточность в сети будет означать многорёберную или многовершинную связность графа.

Методы балансировки могут различаться по виду объектов, к которым они применяются (различают балансировку потоков, пакетов, всплесков [Sinha S., Kandula S., Katabi D.]), и по уровню сетевого стека TCP/IP, на котором они применяются. Например, на канальном уровне можно использовать протоколы LACP, PAgP [Irawati I. D., Hadiyoso S., Hariyani Y. S. Link aggregation control protocol on software defined network //International Journal of Electrical and Computer Engineering. – 2017. – Т. 7. – №. 5. – С. 2706.], а на сетевом уровне протоколы ECMP [Chiesa M., Kindler G., Schapira M. Traffic engineering with equal-cost-multipath: An algorithmic perspective //IEEE/ACM Transactions on Networking. – 2016. – Т. 25. – №. 2. – С. 779-792.], MPLS-TE [Awduche D. et al. Requirements for traffic engineering over MPLS. – 1999. – №. rfc2702.]. Методы балансировки позволяют, за счет использования нескольких маршрутов или физических линий связи, повысить *суммарную* скорость потоков, проходящих между одной и той же парой узлов сети, по сравнению со случаем, когда используется только один маршрут.

Однако стоит отметить, что балансировка пакетов на практике почти не используется. Это связано с тем, что характеристики маршрутов, участвующих в балансировке, могут существенно различаться. Например, суммарная задержка буферизации на одном из маршрутов может сильно превышать суммарную задержку буферизации на другом маршруте из-за наличия перегрузки. В этом случае пакеты могут поступать в точку назначения неупорядоченно. Поскольку транспортный агент TCP протокола может принять неупорядоченный приход пакетов за потерю еще недошедших пакетов, то агентом будет ошибочно детектироваться перегрузка в сети, что приведет к снижению скорости передачи данных.

Чтобы преодолеть недостатки балансировки пакетов, поток данных от уровня приложения можно разделить на несколько потоков (называемые далее в работе ***подпотоками***), которые передаются через несколько транспортных соединений, каждый из которых использует свой маршрут. Такой подход будем называть ***многопоточной маршрутизацией***, а реализующие его протоколы - ***многопоточными протоколами***. После того, как пакеты подпотоков достигнут получателя, происходит объединение подпотоков в исходный поток данных, который отправляется на уровень приложения. В настоящий момент идет активное развитие многопоточных протоколов – существует уже более 20 различных видов, версий таких протоколов [Habib S. et al. The past, present, and future of transport-layer multipath //Journal of Network and Computer Applications. – 2016. – Т. 75. – С. 236-258].

Методы мультиплексирования предполагают объединение несколько потоков данных в один с целью увеличения эффективности передачи. Примером мультиплексирования является подход, называемый сетевым кодированием [13]. В сетевом кодировании предполагается, что промежуточные устройства в сети могут заниматься не только задачей передачи данных, но и проводить операции над пакетами. Так, в линейном сетевом кодировании несколько пакетов могут быть объединены в один при помощи бинарной операции «Исключающее ИЛИ». А для дошедших до получателя измененных пакетов составляется система уравнений, по которой можно восстановить исходные пакеты. Так как вместо двух отдельных последовательных передач пакетов происходит только одна передача, то освобождаются ресурсы для передачи других пакетов, что приводит к увеличению скорости потоков.

Методы сжатия передаваемых данных позволяют увеличить скорость передачи полезной нагрузки за счет уменьшения объема фактически передаваемых данных. Применение методов сжатия передаваемых данных стало возможно благодаря возросшим вычислительным возможностям сетевых устройств, что позволяет проводить дополнительные операции над данными без ощутимых задержек. Современные методы сжатия позволяют в динамике

определять наилучшие параметры сжатия [14] и проводить необходимы операции прозрачно для уровня приложений [15].

Методы управления скоростью передачи необходимы для предотвращения перегрузки в сети – ситуации, когда скорость поступления пакетов превосходит скорость отправки пакетов. Во время перегрузки излишек пакетов может быть сохранен временно в буфере на устройстве, однако после его переполнения лишние пакеты будут сброшены и должны быть отправлены повторно. Исследование [16] показало, что постоянные повторные передачи пакетов, возникающие из-за перегрузок в сети, приводят к существенному снижению скорости соединения (почти в 1000 раз), поэтому необходимо контролировать количество пакетов, одновременно находящихся в сети для каждого соединения. Для этого были разработаны алгоритмы управления перегрузкой. Однако снижение количества отправляемых пакетов может привести к недоиспользованию доступной пропускной способности в сети, поэтому необходимо разрабатывать алгоритмы управления перегрузкой, подходящие для определенных условий в сети (пропускная способность и задержка в сети, возможности сетевого оборудования, распределение нагрузки в сети и т.д.). В настоящее время продолжается активное развитие алгоритмов управления перегрузкой, например, для алгоритма BBR [17] от Google вносятся изменения в ядро операционной системы Linux.

Методы сегментирования позволяют разбить одно транспортное соединение на несколько последовательных соединений при помощи дополнительных серверов на маршруте, называемых *прокси*. Основное преимущество сегментирования лежит в уменьшении круговой задержке последовательных соединений, что позволит увеличить скорость транспортного соединения, которую можно оценить как $\frac{1}{RTT}\sqrt{\frac{3}{2p}}$ [18], где RTT – круговая задержка соединения, а p – вероятность потери пакета на маршруте. Так как при условии небольших накладных расходов круговая задержка на промежуточном соединении с использованием прокси будет меньше из-за уменьшившейся

задержки распространения, чем на всем маршруте между отправителем и получателем, то и скорость соединения на отдельных сегментах будет выше, поэтому возрастет и скорость всего соединения. Из последних работ стоит отметить Dynamic Split TCP [19], в которой предлагается в динамике определять, через какие прокси стоит проложить маршрут транспортного соединения, исходя из приведенной выше оценки скорости и текущей загрузки прокси. Между прокси возможно применять и другие интенсивные методы обеспечения качества сервиса: можно настроить свой алгоритм управления перегрузкой, использовать многопоточную маршрутизацию или сетевое кодирование.

Настоящая работа посвящена исследованию методов многопоточной маршрутизации для задачи обеспечения качества сетевого сервиса. Современные сети передачи данных открывают много возможностей для применения многопоточной маршрутизации. Устройства пользователей имеют несколько сетевых интерфейсов для подключения к Интернет, например, мобильные телефоны могут подключиться к Интернет как через WiFi, так и через широкополосную сеть мобильной связи 3G/4G/5G. Сервера в ЦОД с целью повышения надежности тоже используют несколько сетевых интерфейсов [20]. Таким образом, между конечными узлами в сети часто существует несколько маршрутов, возможно, даже непересекающихся между собой.

Несколько маршрутов можно использовать для повышения качества сервиса транспортного соединения [21]: даже если ни один маршрут не удовлетворяют требованиям качества приложения, совокупность ресурсов маршрутов зачастую может обеспечить надлежащее качество. Многопоточная маршрутизация в отличие от классического протокола TCP позволяет использовать ресурсы нескольких маршрутов одновременно, тем самым, повышая качество сервиса.

При разработке многопоточных протоколов возникают следующие подзадачи:

1. Распределение транспортных сегментов данных между подпотоками – требуется определить для разделенных на сегменты данных от приложения, по какому из подпотоков их стоит отправить. Выбор

подпотока будет непосредственно влиять на сквозную задержку сегмента. Если задержка окажется слишком большой и последующие сегменты будут доставлены раньше, то может возникнуть ситуация, называемая *блокировкой начала очереди* [22] – из-за конечного размера буфера для хранения пришедших сегментов транспортному агенту многопоточного протокола придется простаивать в ожидании недошедшего сегмента, что в свою очередь скажется на предоставляемом качестве сервиса.

2. Синхронизация и контроль скорости отправки пакетов на разные подпотоки – необходимо контролировать скорость отправки данных не для одного транспортного потока, а сразу для нескольких подпотоков.
3. Выбор количества подпотоков – скорость транспортного соединения, создаваемого многопоточным протоколом, которое будем называть ***многопоточным соединением***, зависит от числа подпотоков. Излишнее количество подпотоков будет означать нерациональное использование ресурсов сети, которые могли быть выделены для других потоков. Заметим так же, что каждый «лишний» подпоток требует накладных расходов на поддержание. А недостаточное количество подпотоков не позволит удовлетворить требования приложения к скорости транспортного соединения.
4. Выбор маршрутов в сети для подпотоков многопоточного соединения – маршруты подпотоков одного и того же многопоточного соединения, которые будем называть ***родственными подпотоками***, должны иметь минимальное число пересечений - общих линий связи. Это ограничение связано с тем, что точки пересечения маршрутов – потенциально *узкие места*, в которых за одну и ту же линию связи будут конкурировать между собой родственные подпотоки. Эта конкуренция снизит эффективность применения многопоточной маршрутизации. Важно отметить, что задача управления маршрутами родственных подпотоков с помощью многопоточной маршрутизации рассматривается для

программно-конфигурируемых сетей (ПКС), в которых контроллер ПКС в каждый момент времени знает состояние всей сети и может проложить непересекающиеся маршруты для родственных подпотоков [21].

В настоящем исследовании основное внимание уделено задачам выбора количества подпотоков и выбора маршрутов для них при следующих предположениях:

1. Будем считать, что все родственные подпотоки проходят через одну и ту же автономную систему, которая считается однородной с точки зрения используемой технологии передачи и характеристик линий (пропускная способность, задержка, вероятность потери пакета). Однако это предположение не означает однородность маршрутов по метрике, например числу хопов. За счет варьирования этой характеристики мы можем нивелировать предположение об однородности линий связи. Главное для нам то, чтобы разница в RTT на разных маршрутах не приводила к длительной блокировке начала очереди, поэтому можно использовать алгоритм планирования, который выбирает подпоток с минимальным RTT [23]. Предположение о проходе всех подпотоков через одну автономную сеть не окажет существенного влияния на ценность полученных в работе результатов, так как для решения проблемы блокировки начала очереди можно использовать алгоритм Fine-grained Forward Prediction based Dynamic Packet Scheduling (F^2PDPS) [24]. Алгоритм F^2PDPS распределяет сегменты на основании оценки времени прихода сегмента по каждому из родственных подпотоков. Эта оценка учитывает не только RTT, но и вероятность потери пакетов на маршруте. Однако для алгоритма F^2PDPS нет реализации с открытым кодом. Поэтому его использование в настоящей работе невозможно, так как все компоненты стенда для проведения экспериментального исследования должны быть

прозрачными для обоснованного и детального анализа результатов, а самостоятельная реализация и интеграция этого алгоритма в код ядра операционной системы потребует значительное время.

2. Будем считать, что в каждом из подпотоков используется в качестве алгоритма управления перегрузкой TCP Cubic [25], который применяется по умолчанию в современных операционных системах. Для многопоточных протоколов существуют специализированные алгоритмы управления перегрузкой (такие как OLIA [26], BALIA[27], mpCubic[28] и другие). Особенностью этих алгоритмов является согласованное управление окном перегрузки у разных подпотоков. Такое управление в отличие от независимого управления окнами перегрузки не будет приводить к ущемлению в скорости не многопоточных соединений в традиционной сети, когда невозможно гарантировать отсутствие пересечений у маршрутов родственных подпотоков. Иными словами, согласованное управление окнами перегрузки в разных подпотоках позволит сделать многопоточное соединение менее агрессивным по скорости отправки данных в сеть, а, следовательно, более дружелюбным к не многопоточным транспортным соединениям. Таким образом, согласованное управление окнами перегрузки ограничивает совместный рост окон перегрузки у родственных подпотоков так, как будто в сети действует только одно транспортное соединение с одним окном перегрузки. Описанное ограничение отрицательно влияет на решение задачи обеспечения качества сервиса, так как в случае непересекающихся маршрутов многопоточное соединение будет менее эффективно использовать предоставленные ресурсы сети.
3. В настоящей работе рассматривается применение многопоточных соединений для перспективного класса сетей - программно-конфигурируемых сетей (ПКС) [21]. То, что исследование ограничивается определенным классом сетей, на самом деле не сужает

общности полученных результатов, так как для случая, когда часть маршрутов родственных подпотоков имеет пересечения, т.е. одно и то же узкое место, можно использовать алгоритм управления перегрузкой ADW-BALIA [29]. Этот алгоритм находит подпотоки с одним узким местом на основе корреляции вариации RTT и применяет согласованное управление окном перегрузки для родственных подпотоков с одним узким местом и независимое - для управления перегрузкой у подпотоков с непересекающимися маршрутами. Однако для алгоритма управления перегрузкой ADW-BALIA тоже не существует открытой реализации, поэтому этот алгоритм невозможно использовать в экспериментальном исследовании по тем же причинам, что и алгоритм распределения транспортных сегментов F^2PDPS.

Одним из важных и актуальных применений многопоточных протоколов может стать организация каналов для передачи данных между ЦОД. В настоящее время для организации передачи данных между ЦОД необходимо настраивать выделенный канал, который приходится поддерживать даже тогда, когда он не используется. Альтернативным подходом может стать стратегия “Pay as you go” [30], согласно которой можно оплатить предоставление услуги и начать ей пользоваться только в необходимые моменты времени. В случае ЦОД такой услугой может стать предоставление канала с заданной ***пропускной способностью по требованию*** (Bandwidth on Demand или сокращенн**о** BoD) [31].

Для оказания услуги BoD заключается ***контракт*** – договоренность о передаче фиксированного объема данных между указанными точками отправления и назначения за время, не больше заданного. Тогда запрос на услугу BoD подразумевает резервирование Интернет-провайдером пропускной способности сети, необходимой для реализации контракта. С точки зрения Интернет-провайдера это означает, что ему необходимо знать, на каких линиях связи своей сети и какой объем пропускной способности нужно зарезервировать, чтобы реализовать контракт в случае его наступления. От объема оставшихся

ресурсов сети будет зависеть общее количество контрактов, которые Интернет-провайдер может удовлетворить.

Услугу BoD можно реализовать при помощи однопоточных протоколов маршрутизации и протокола резервирования сетевых ресурсов RSVP [32]. Однако использование протокола RSVP может приводить к отказам в реализации контракта, так как используются традиционные алгоритмы маршрутизации для поиска маршрута с необходимой пропускной способностью. Кроме того, можно повысить количество удовлетворенных запросов по контрактам при помощи использования многопоточной маршрутизации, так как в ситуациях, когда ресурсов одного маршрута не будет хватать, чтобы передать необходимый объем данных, можно будет задействовать несколько маршрутов. В настоящей работе будет предложен подход, позволяющий найти маршруты и объем необходимого резерва для реализации потока запросов на услугу BoD.

Цель диссертационной работы

Целью диссертационного исследования является разработка методов многопоточной маршрутизации для обеспечения требуемого качества сетевого сервиса в программно-конфигурируемых сетях

Для достижения поставленной цели в рамках настоящей работы было необходимо решить следующие задачи:

1. Провести анализ существующих методов многопоточной маршрутизации.
2. Разработать многопоточный метод маршрутизации для программно-конфигурируемых сетей.
3. Разработать и обосновать методику сравнительного анализа методов многопоточной маршрутизации.
4. Провести сравнительный анализ и определить границы применимости методов многопоточной маршрутизации.
5. Разработать метод оценки резерва для услуги «Пропускная способность по требованию».

Научная новизна

В диссертации исследован новый метод динамической многопоточной маршрутизации и предложена оригинальная методика сравнения эффективности методов многопоточной маршрутизации. Её новизна заключается в том, что она позволяет оценить эффективность не только распространенного в настоящее время статического метода, но и динамического метода Flow-Demultiplexing Protocol (FDMP), экспериментально реализованного в ходе диссертационного исследования и эффективность которого не была исследована. Также в диссертации предложен метод оценки резерва пропускной способности для реализации услуги «Пропускная способность по требованию», актуальной для обмена данными между центрами обработки данных. Оценка такого резерва необходима для определения максимального числа пользователей услуги без потери качества сервиса.

Теоретическая и практическая ценность

Теоретическая ценность работы состоит в разработке алгоритма выбора маршрутов с минимальным числом пересечений по ребрам графа сети, исследовании и реализации динамического многопоточного протокола и предложенном методе оценки эффективности методов многопоточной маршрутизации в сетях Интернет-провайдеров.

Практическая ценность работы заключается в создании:

- прототипа программного обеспечения, реализующего динамический много поточный протокол FDMP,
- методики и инструментальных средств с открытым исходным кодом [33] для оценки эффективности методов многопоточной маршрутизации для сетей Интернет-провайдеров.

Кроме того, результаты диссертационной работы использовались для организации связи виртуальных машин, расположенных в ЦОД компаний Облакотека (Москва) и Cortel (Новосибирск) на канале 10GE, предоставленном

компанией ТНТК, чтобы предоставить новую услугу «Пропускная способность по требованию».

Методы исследования

При получении основных результатов диссертации использовался аппарат теории оценки статистических гипотез. Методы этой теории позволяют математически строго оценивать количество экспериментов, необходимых для обоснования выдвигаемой гипотезы об эффективности использования многопоточной маршрутизации.

Также для решения задачи оценки резерва пропускной способности была построена математическая модель, в рамках которой исходная задача была сведена к задаче линейного программирования, для решения которой применялся симплекс метод.

Степень достоверности

Достоверность полученных результатов подтверждается использованием проверенных математических методов, обширным экспериментальным исследованием, публикациями всех результатов в рецензируемых изданиях. Элементы предложенных решений были использованы облачными и интернет провайдерами для услуги «Пропускная способность по требованию», что также подтверждает их корректность. Основные результаты диссертации прошли успешную апробацию на трех международных конференциях.

Соответствие паспорту специальности

Диссертация соответствует требованиям специальности 05.13.11 – «Математическое и программное обеспечение вычислительных машин, комплексов и компьютерных сетей». Её цель, задачи и результаты отвечают двум пунктам паспорта специальности:

п. 3. «Модели, методы, алгоритмы, языки и программные инструменты для организации взаимодействия программ и программных систем».

п. 9. Модели, методы, алгоритмы и программная инфраструктура для организации глобально распределенной обработки данных.

Основные результаты

Основные положения, выносимые на защиту:

- Модель работы динамического многопоточного протокола для транспортных соединений, который позволяет повысить эффективность использования ресурсов сети не менее, чем в 95% случаев;
- Алгоритм построения маршрутов с минимальным числом пересечений по ребрам графа сети, на основе которого предложен подход к организации многопоточной маршрутизации в ПКС-сетях, который позволяет на порядок сократить количество OpenFlow правил в ПКС коммутаторах.
- Условия применимости и оценки эффективности статического и динамического методов многопоточной маршрутизации транспортных соединений в ПКС сетях.
- Метод оценки резерва пропускной способности для услуги «Пропускная способность по требованию» по заданным топологии и потокам запросов между центрами обработки данных.

Апробация работы

Результаты, представленные в работе, докладывались на научных семинарах кафедры Автоматизации систем вычислительных комплексов факультета ВМК МГУ под руководством чл.-корр. РАН, профессора Р.Л. Смелянского, консорциуме «Сетевые и облачные технологии», на международных конференциях «27-й Международный симпозиум по ядерной электронике и компьютингу NEC`2019» и «Modern Network Technologies, MoNeTec-2018», а также на конференциях «Ломоносовские чтения 2019» и «Ломоносовские чтения 2020».

Публикации

По теме диссертации подготовлено 11 работ [19, 34-43], в том числе 6 работ [19, 34-38] в рецензируемых изданиях, индексируемых международными

наукометрическими базами RSCI, Scopus и(или) Web of Science, а также Свидетельство регистрации программы для ЭВМ [42].

Личный вклад

Личный вклад автора заключается в выполнении основного объема теоретических и экспериментальных исследований, изложенных в диссертационной работе, включая разработку теоретических моделей и алгоритмов, построение их программных реализаций, создание оригинальных методик экспериментальных исследований, проведение исследований, анализ и оформление результатов в виде публикаций и научных докладов.

Структура работы

Диссертация состоит из введения, 3 глав, заключения, 3 приложений и списка литературы.

Во введении обоснована актуальность диссертационной работы, сформулирована цель и аргументирована научная новизна исследований, показана практическая значимость полученных результатов, представлены выносимые на защиту научные положения и дано краткое описание задачи.

В первой главе приведено описание статического и динамического методов многопоточной маршрутизации, а также рассматриваются вопросы, связанные с построением маршрутов и организацией многопоточной маршрутизации в ПКС.

Во второй главе приводится сравнение эффективности статического и динамического методов многопоточной маршрутизации.

В третьей главе предлагается подход к решению задачи «Пропускная способность по требованию».

В заключении сформулированы основные результаты работы; приведены возможные направления дальнейших исследований рассмотренных задач и развития предложенных подходов.

В приложении А приведено подробное описание методики проверки статистической гипотезы о числовом значении вероятности события, необходимой для экспериментальных исследований во второй и в третьей главах. **В**

приложении Б описаны технические подробности Стенда для проведения сравнительного анализа статического и динамических методов многопоточной маршрутизации. **Приложение В** содержит подробное описание пакетов для решения задачи целочисленного линейного программирования.

Полный объём диссертации составляет 114 страниц (107 страниц не считая приложений) с 24 рисунками и 5 таблицами. Список литературы содержит 94 наименования.

Глава 1. Методы многопоточной маршрутизации

Как было отмечено во введении, при разработке многопоточного протокола встают задачи распределения транспортных сегментов данных между родственными подпотоками, разработки алгоритма управления перегрузкой в родственных подпотоках, регулирования количества подпотоков и построения маршрутов для каждого из родственных подпотоков. Рассмотрим более подробно задачу регулирования количества родственных подпотоков.

Можно выделить два класса методов для решения обозначенной проблемы: статический и динамический. В статических методах количество родственных подпотоков для транспортного соединения устанавливается в каждом случае заранее и не изменяется с течением времени. Количество родственных подпотоков может зависеть от настроек пользователя или числа сетевых интерфейсов на устройстве отправителя и получателя (в этом случае подпоток открывается для каждой пары интерфейсов).

В динамических методах количество родственных подпотоков на транспортном соединении зависит от требований приложения к качеству сервиса и текущего состояния сетевого окружения. Когда ресурсов маршрутов текущего множества родственных подпотоков не хватает для обеспечения требуемого качества сервиса, то происходит открытие дополнительного подпотока. Например, на рисунке 1 показан случай, когда между отправителем и получателем необходимо обеспечить передачу данных со скоростью 10 Мб/с. Первоначально был создан только один подпоток, маршрут которого выделен красным цветом в верхней части топологии сети. Причем можем считать, что при старте соединения пропускная способность красного маршрута была больше 10 Мб/с, т.е. одного подпотока хватало для обеспечения требуемого качества сервиса. Однако наступает момент, когда пропускная способность красного маршрута падает до 8 Мб/с. Например, из-за появления новых транспортных потоков, маршрут которых частично или полностью совпадает с красным маршрутом. Так как пропускной способности маршрута одного подпотока не хватает для обеспечения требуемого качества сервиса, то согласно динамическому принципу многопоточной

маршрутизации произойдет открытие дополнительного нового подпотока. Маршрут нового подпотока изображен фиолетовой пунктирной линией, его пропускная способность равна 8 Мбит/с, и суммарная пропускная способность маршрутов двух подпотоков позволит удовлетворить требования к качеству сервиса.

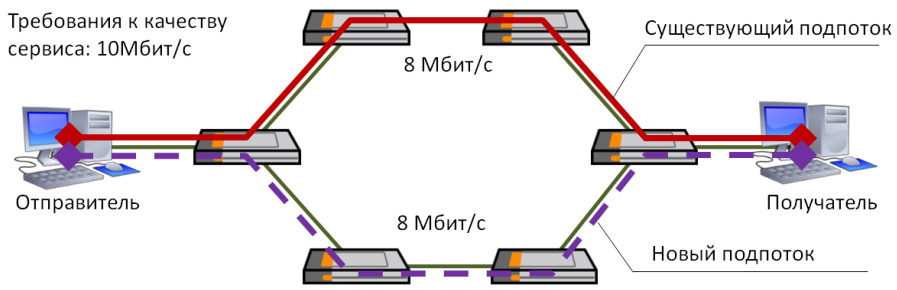

Рисунок 1. Пример работы динамического метода многопоточной маршрутизации

В случае, когда требования пользователя могут быть удовлетворены меньшим числом родственных подпотоков, лишние подпотоки будут завершены, тем самым освобождая ресурсы сети для обслуживания других потоков. Например, на рисунке 1 второй подпоток с фиолетовым маршрутом может быть закрыт, если пропускная способность красного маршрута вновь превысит 10 Мб/с.

Возникает вопрос, какой из методов к многопоточной маршрутизации – статический или динамический, будет лучше в той или иной ситуации. Несмотря на простоту реализации статического метода, можно привести пример, когда его использование приводит к неэффективному использованию ресурсов сети. Рассмотрим пример на рисунке 2. Предположим, что за каждым маршрутизатором находится пользователь сети и идет передача данных между каждой парой пользователей. Также будем предполагать, что пропускная способность всех линий связи в представленной топологии равна 1. Тогда суммарная скорость, которую можно получить, используя классические однопоточные протоколы, равна 6 (по 1 в каждую сторону на каждой линии связи).

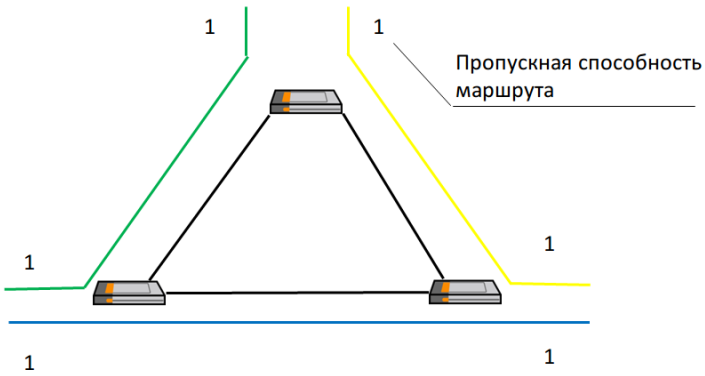

Рисунок 2. Пример топологии «треугольник», когда методы многопоточной маршрутизации не используются

Можно заметить, что между каждой парой пользователей существует два непересекающихся маршрута, поэтому в статическом методе многопоточной маршрутизации логично использовать два подпотока для каждого многопоточного соединения. Пример организации такой передачи данных изображен на рисунке 3, где маршрут второго подпотока обозначен пунктирной линией. Чтобы оценить скорость многопоточных соединений, сделаем дополнительное предположение, что пропускная способность каждой линии связи будет делиться поровну между подпотоками, маршруты которых проходят через эту линию связи (такое предположение возможно сделать, так как маршруты родственных подпотоков не имеют общих узких мест). Так как пропускная способность каждой линии связи была равна 1, а для рассматриваемой ситуации через одну и ту же линию связи проходят 3 подпотока, то скорость каждого подпотока будет равна $\frac{1}{3}$. Тогда скорость многопоточного соединения будет равна $\frac{2}{3}$, а суммарная скорость потоков - $\frac{2}{3} * 6 = 4$. Получаем, что скорость в случае использования статического метода многопоточной маршрутизации будем меньше, чем скорость в случае без использования многопоточных протоколов. Если мы будем использовать динамический метод многопоточной маршрутизации, то подобной ситуации можно будет избежать, так как второй подпоток не будет создаваться за

ненадобностью или закроется, так как не будет увеличения в скорости многопоточного соединения.

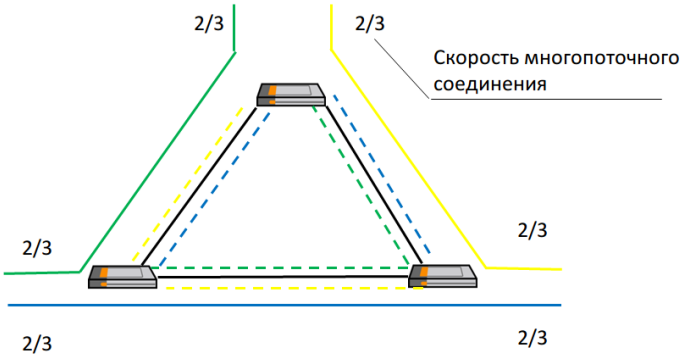

Рисунок 3. Пример топологии «треугольник», когда используется статический метод к многопоточной маршрутизации

Таким образом, становится актуальной задача сравнения эффективности статического и динамического методов многопоточной маршрутизации и определения условий их эффективного применения.

1.1 Анализ работ по эффективности многопоточной маршрутизации

В работах, направленных на анализ эффективности многопоточной маршрутизации, в большинстве случаев в качестве основного критерия исследуют прирост скорости транспортного соединения, при статическом многопоточном методе в определенной среде передаче данных или при определенной нагрузке. В работе [44] рассматривается увеличение производительности статического метода при использовании разных алгоритмов планирования сегментов между подпотоками. Авторы статьи [45] исследуют эффективность статического метода для автомобилей, подключенных к сети. Эти автомобили могут координировать свою работу между собой, с городскими службами дорожного движения, а также выполнять вспомогательные вычисления на других сетевых устройствах. Работа [46] посвящена исследованию производительности статического метода многопоточной маршрутизации в беспроводных децентрализованных самоорганизующихся сетях. В статье [47] рассматривается влияние разных

алгоритмов управления перегрузкой в случае статического многопоточного метода. Автором настоящей работы было проведено исследование массового использования статического метода к многопоточной маршрутизации в рамках одной автономной системы. Исследование показало, что применение статического многопоточного транспортного соединения имеет преимущество перед однопоточным соединением протокола TCP только при условии загрузки сети, меньшей 37% [35]. Под загрузкой сети понимается среднее значение загрузки линий связи, где загрузка линии связи рассчитывалась как отношение суммарной скорости TCP потоков, проходящих через эту линию связи, к её пропускной способности.

Из работ по анализу эффективности динамического метода многопоточной маршрутизации можно отметить только работу [48], в которой рассматривается эффективность динамического метода для беспроводных сетей.

Для проведения сравнительного анализа статического и динамического методов многопоточной маршрутизации необходимо выбрать протоколы, на которых будет исследована эффективность работы методов. Выбор проводился по следующим критериям:

1. *Класс метода.* Многопоточный транспортный протокол может соответствовать статическому или динамическому методу.
2. *Поддержка протокола*. Для получения актуальных достоверных результатов необходимо, чтобы реализация многопоточного протокола была открытой и доступной, а также имела действующую поддержку, чтобы соответствовать текущим требованиям сетей.
3. *Опорный транспортный протокол*. Многие многопоточные протоколы используют такие распространенные транспортные протоколы, как TCP, UDP, для того чтобы упростить внедрение этих протоколов в существующую инфраструктуру сети.
4. *Возможность использования произвольного алгоритма управления перегрузкой*. Так как развитие других методов обеспечения качества сервиса не стоит на месте, то важно, чтобы многопоточный протокол

мог использовать современные алгоритмы управления перегрузкой в своей работе.

Анализ схожих работ показал, что наиболее популярных многопоточным протоколом, поддерживающим статический метод, является Multipath TCP (MPTCP) протокол [23]. Несмотря на большое количество различных многопоточных протоколов [Habib S. et al. The past, present, and future of transport-layer multipath //Journal of Network and Computer Applications. – 2016. – Т. 75. – С. 236-258], многие из них являются разными вариантами (улучшениями) протокола MPTCP. Другие многопоточные протоколы, согласно обзору [Habib S. et al. The past, present, and future of transport-layer multipath //Journal of Network and Computer Applications. – 2016. – Т. 75. – С. 236-258], либо основаны на SCTP-CMT, либо представляют новый транспортный протокол, который не имеет стандарта. В обоих случаях многопоточный протокол будет сложнее интегрировать в существующую инфраструктуру сети. Из последних работ можно также отметить многопоточный протокол MP-QUIC [49], который использует UDP в качестве опорного транспортного протокола. Однако протокол MP-QUIC основан на тех же принципах работы, что и протокол MPTCP, поэтому протокол MPTCP был выбран как основной представитель статического многопоточного протокола для транспортных соединений для проведения сравнительного анализа эффективности статического и динамического методов многопоточной маршрутизации. Кроме этого, протокол MPTCP имеет поддерживаемую доступную открытую реализацию в отличие от протокола MP-QUIC.

Динамический алгоритм из работы [48] будем называть BNDEEI-MPTCP (Bandwidth-Need Driven Energy Efficiency Improvement of MPTCP). Построенная модель работы метода многопоточной маршрутизации для BNDEEI-MPTCP в качестве одного из критериев открытия подпотока использует энергоэффективность канала, что мало применимо для проводных сетей Интеренет-провайдеров. Кроме того, BNDEEI-MPTCP опирается на предположении, что используется алгоритм управления перегрузкой, который

определяет перегрузку по потере пакета (loss-based алгоритмы). Это не позволит использовать такие современные алгоритмы управления перегрузкой как BBR [17] в многопоточном транспортном протоколе. Поэтому в качестве представителя динамического метода в сравнительном анализе в работе использован протокол Flow De-Multiplexing Protocol (FDMP), предложенный Смелянским Р. Л. и разработанный и исследованный автором настоящей работы. Результаты проведенного обзора по наиболее значим многопоточным протоколам подытожены в таблице 1.

Протокол	Класс	Поддержка	Опорный транспортный протокол	Алгоритм управления перегрузкой
MPTCP	**Статический**	**Да**	**TCP**	**Любой**
SCTP-CMT	Статический	Нет	SCTP	Любой
MPQUIC	Статический	Нет	UDP	Любой
BNDEEI-MPTCP	Динамический	Нет	TCP	Loss-based

Таблица 1. Сравнение многопоточных транспортных протоколов.

Главный вывод из сказанного выше, состоит в том, что отсутствуют работы, посвященные сравнению эффективности статического и динамического методов многопоточной маршрутизации, а для проведения такого сравнения были выбраны протоколы MPTCP и FDMP соответственно. В следующих разделах приведено более подробное описание протоколов MPTCP и FDMP, которое будет необходимо для понимания устройства экспериментального стенда для сравнительного анализа.

1.2 MPTCP

Multipath TCP (MPTCP) – это модификация протокола TCP, поддерживающая статический метод многопоточной маршрутизации.

Разработчики протокола MPTCP приводят принципы, которыми руководствуются для упрощения интеграции своего протокола в современные сети [23]:

1) Передаваемые протоколом сегменты данных должны соответствовать спецификации TCP протокола: структура заголовка сегмента данных, его максимальный размер, а также процедура установки и разрыва транспортного соединения, чтобы снизить риски несовместимости с коммутационным оборудованием.

2) Протокол не должен зависеть от предположений отсутствия фрагментации сегмента данных или состава полей расширения заголовка сегмента (опциональных полей).

3) Протокол должен использовать существующий интерфейс протокола TCP для работы с приложениями.

4) Транспортный агент должен иметь возможность автоматически выбирать, по какому протоколу - TCP или MPTCP, необходимо работать. Такая возможность необходима в случае невозможной дальнейшей работы по протоколу MPTCP.

Протокол MPTCP достигает поставленных целей за счет использования протокола TCP с механизмом синхронизации управления окном перегрузки для каждого из своих подпотоков и передаче служебной информации в опциональной части транспортного заголовка пакетов. Опишем основную схему функционирования протокола MPTCP.

1. *Установка соединения*

Установка соединения происходит процедурой трехкратного рукопожатия: обменом сегментами с флагами SYN, SYN/ACK, ACK, которые находятся в транспортном заголовке сегмента. Каждый указанный сегмент содержит опцию MP_CAPABLE транспортного заголовка TCP. Эта опция указывает на поддержку работы по протоколу Multipath TCP и стремление использовать этот протокол на данном соединении.

В случае отсутствия опционального поля в заголовке сегмента в любом из пакетов трехкратного рукопожатия установка соединения и дальнейшая его работа будет идти согласно TCP протоколу. Опция MP_CAPABLE содержит 64-битный ключ, который впоследствии будет использоваться для определения родственных

подпотоков. В сегменте с флагом SYN находится ключ отправителя для этого соединения, в сегментах с флагами SYN/ACK ключ получателя для этого соединения, а в сегменте с флагом ACK – ключ отправителя, за которым следует ключ получателя.

2. *Запуск нового подпотока*

Новый родственный подпоток запускается обменом сегментов с флагами SYN, SYN/ACK, ACK, как и в процедуре трехкратного рукопожатия для установления соединения в протоколе TCP. В каждый указанный сегмент включается опция MP_JOIN. Опция MP_JOIN содержит криптографические хешы от ключей, которыми обменялись отправитель и получатель в опции MP_CAPABLE, которые идентифицируют, к какому многопоточному соединению подключается подпоток.

В случаях, когда в сегментах с флагами SYN, SYN/ACK, ACK в сети была удалена опция MP_JOIN или неверно завершилась аутентификация, посылается сегмент с флагом RST для отмены установления соединения для нового подпотока.

3. *Функции агента MPTCP в ходе жизни транспортного соединения*

- Поддержка биективного отображения между порядковыми номерами сегментов, используемых на подпотоках, и изначальными номерами, показывающие порядок, в котором формировались сегменты на отправителе при поступлении данных от приложения. Это отображение нужно для восстановления правильного порядка данных, пришедших к получателю.
- Управление потоком с использованием единого буфера для родственных подпотоков.
- Подтверждение сегментов на уровне подпотока и на уровне многопоточного соединения.

Вся служебная информация, необходимая для реализации описанной выше функциональности, передается между транспортными агентами в опциональной части транспортного заголовка протокола TCP.

• *Завершение соединения*

Сегмент с флагом FIN в протоколе MPTCP имеет значение завершения для подпотока. Чтобы завершить передачу на уровне соединения, в опциональной части транспортного заголовка передается флаг DATA_FIN. После того, как будет получено подтверждение на этот сегмент (сначала должны быть подтверждены все данные), транспортные агенты должны отправить сегмент с флагом FIN на все подпотоки.

• *Варианты настройки MPTCP агента, которые может задать пользователь*:

1) Default – агент на отправителе работает как обычный агент протокола TCP и не инициирует открытия новых подпотоков.
2) Fullmesh – у каждого отправителя и у каждого получателя есть свой пул IP адресов, агент открывает подпотоки по принципу «каждый с каждым»: с каждого IP адреса отправителя открывается подпоток на каждый IP адрес получателя. Например, если пул IP адресов каждого содержит 3 адреса, то всего у нас будет 9 подпотоков. Все подпотоки создаются при открытии соединения.
3) Ndiffports – агент открывает N потоков между IP адресом получателем и IP адресом отправителем, запрашивая новые TCP порты у операционной системы. Параметр N настраивается средствами операционной системы, все подпотоки создаются при открытии многопоточного соединения.

Протокол MPTCP предоставляет возможность использовать заранее заданное количество маршрутов между получателем и отправителем, что может повысить качество сервиса. Однако протокол MPTCP не предоставляет возможности динамически ни открывать новый подпоток, ни закрывать подпотоки при необходимости. Также протокол MPTCP не отвечает за построение маршрутов, из-

за чего построенные маршруты в сети (согласно действующей в сети политике маршрутизации) могут пересекаться и совпадать.

1.3 FDMP

В данном разделе представлен протокол FDMP - один из вариантов динамического метода многопоточной маршрутизации, согласно которому транспортный агент отправителя открывает или закрывает родственные подпотоки в зависимости от текущей загруженности сети и требований качества сервиса. Протокол FDMP является многопоточным протоколом, реализующего схожую функциональность с протоколом MPTCP, но в котором алгоритм, отвечающий за регулирование количества родственных подпотоков, работает иначе. Далее подробнее рассмотрим FDMP протокол, указывая на его отличия от MPTCP.

1. *Условие открытия подпотока*

На первый взгляд, транспортный агент FDMP протокола должен открыть новый подпоток, если скорость многопоточного соединения (определяемая как сумма скоростей родственных подпотоков) меньше заданной нижней границы качества сервиса для скорости передачи. Однако это не всегда так. Рассмотрим пример:

Пусть приложение отправляет по сети данные, введенные пользователем с клавиатуры. Из-за объема данных, передаваемых по сети, скорость потока будет не велика и, возможно, меньше заданной границы качества сервиса. Но открывать новый подпоток для увеличения скорости не имеет смысла, так как само приложение отправляет данные на TCP уровень сетевого стека операционной системы с меньшей скоростью. Из этого примера следует, что ответ на вопрос - открыть новый подпоток или нет, зависит от скорости передачи данных между приложением и транспортным уровнем сетевого стека протоколов операционной системы.

Рассмотрим еще один пример: пусть идет стабильная передача данных по сети со скоростью большей, чем требует приложение. В какой-то момент времени в сети происходит перегрузка и скорость соединения падает. На короткий момент времени скорость становится ниже границы качества сервиса и FDMP-агент открывает новый подпоток. Но затем перегрузка окончилась, и скорость снова удовлетворяет качеству сервиса, поэтому новый FDMP-подпоток был напрасно открыт. Значит, при решении вопроса об открытии нового потока необходимо использовать данные мониторинга соединения, а не просто результат однократного измерения.

Таким образом, для определения условия открытия подпотока необходимо:

- найти способ детектировать загруженность предоставленных каналов.
- предложить способ измерения текущей пропускной способности многопоточного соединения;

2. *Расчет скорости многопоточного соединения*

Определим среднюю скорость FDMP-соединения следующим образом:

$$V_{FDMP} = \frac{\widetilde{V(t_2)} - \widetilde{V(t_1)}}{t_2 - t_1} \quad (1)$$

где $\widetilde{V(t)}$- объем данных, которые были переданы и подтверждены с момента начала соединения до времени t. Получаемая величина является средней скоростью передачи данных в сети в период времени с t_1 по t_2 .

Для расчета средней скорости по определенной выше формуле транспортный агент FDMP хранит информацию по отправленным сегментам: объем данных, отправленных с момента установления соединения, и время подтверждения о доставке сегмента получателю. Информация хранится в кольцевом буфере qos_buff, каждый из элементов которого содержит следующие поля:

- t_{rcv} - время, когда пришло подтверждение на отправленный сегмент;
- $\tilde{\tilde{V}}$ - объем данных, которые были переданы приложением и подтверждены с начала установления соединения до времени t_{rcv}.

При этом в транспортном агенте FDMP информация записывается не по каждому сегменту, а сразу по группе сегментов, размер которой является параметром транспортного агента. При таком подходе собранные данные отражают среднюю скорость за продолжительный промежуток времени, не занимая чрезмерное количество памяти в операционной системе. Альтернативой было считать не сегменты, а количество отправленных байтов или делать записи через равные временные интервалы, однако при отправке сегментов небольшого размера в первом случае мы быстрее сможем узнать об изменении в скорости передачи, чем в двух других случаях.

Также у транспортного агента есть несколько указателей на элементы этого буфера:

- app_head – указывает на последний добавленный в буфер элемент (информация по сегменту, который был отправлен в сетевой стек приложением последним, и, возможно, не был еще подтвержден). Будем обозначать соответствующий элемент буфера верхним индексом ah, например, запись $\widetilde{V^{ah}}$ означает поле $\tilde{V}$ элемента буфера qos_buff, на который ссылается указатель app_head;
- rcv_head – указывает на элемент из нашего буфера, содержащий последний подтвержденный сегмент. Будем обозначать соответствующий элемент буфера верхним индексом rh;
- $tail$ – указывает на элемент, который был добавлен раньше всех в буфер (так как буфер кольцевой, то, начиная с некоторого времени, элементы буфера будут перезаписываться новыми). Будем обозначать соответствующий элемент буфера верхним индексом $tail$.

Таким образом, уточним расчет средней скорости FDMP соединения при наличии буфера:

$$V_{FDMP} = \frac{\widetilde{V^{rh}} - \widetilde{V^{tail}}}{t_{rcv}^{rh} - t_{rcv}^{tail}} \quad (2)$$

Применение формулы (2) считается невозможным, если количество элементов буфера qos_buff меньше заданного порога, являющегося параметром транспортного агента. Данное ограничение было введено, чтобы рассчитывать среднюю скорость FDMP соединения, а не мгновенную.

Также может возникнуть ситуация, когда в буфере будут слишком старые данные, которые могут повлиять на скорость в среднем. Например, соединение простаивало пару часов из-за отсутствия данных, а затем передача данных возобновилась. Средняя скорость, рассчитываемая по формуле (2), будет очень маленькой, потому что в кольцевом буфере qos_buff содержатся данные двухчасовой давности. Поэтому в транспортном агенте FDMP при расчете скорости согласно формуле (2) используется отсечение по времени: указатель tail сдвигается на элемент, время получения которого t_{rcv} было не более, чем T^{max} секунд назад, где T^{max} – параметр транспортного агента.

3. *Определение узкого места*

Ранее было отмечено, что могут возникать ситуации, когда приложение отправляет на транспортный уровень данные медленнее, чем скорость транспортного уровня. В этом случае открывать новый родственный подпоток не нужно, т.к. узким местом является приложение, а не линия связи в сети. Для того чтобы отличить такую ситуацию, рассмотрим подробнее работу сетевого стека операционной системы Linux. Приложение отправляет данные в сеть, используя системные вызовы send, sendto, sendmsg, write при работе с сокетом. Все перечисленные системные вызовы в конце концов заканчиваются вызовом функции сокета sendmsg, которая отправляет данные на следующий уровень сетевого стека (транспортный). В случае протокола TCP (и в его расширении FDMP) эта функция приводит к вызову функции tcp_sndmsg.

Функция tcp_sndmsg полученные данные добавляет в двусвязный список sk_buff (рисунок 4), в каждый элемент которого входит отдельный TCP сегмент [50].

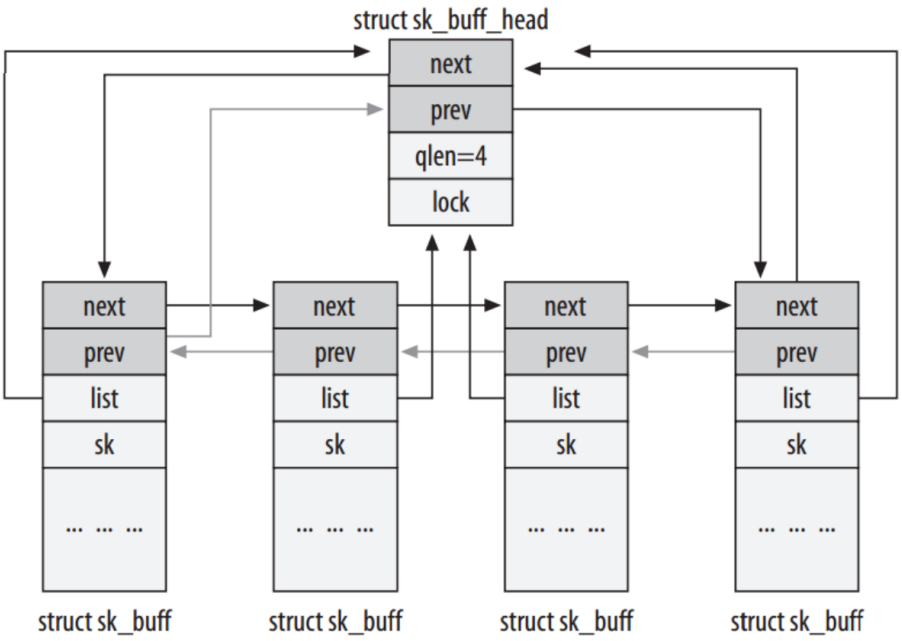

Рисунок 4. Двухсвязный список sk_buff.

После подтверждения сегмента элемент, отвечающий за этот сегмент, удаляется из списка.

Необходимо различать следующие два случая:

1) Поскольку приложение запрашивает необходимую ему пропускную способность один раз при открытии TCP соединения, то может возникнуть случай, когда приложение отправляет данные со скоростью меньшей, чем было затребовано. В этом случае транспортный агент FDMP отправляет поступающие данные и ждет поступления следующих сегментов. В итоге $V_{FDMP} < V_{QoS}$ (V_{QoS} - предъявляемые требования к качеству сервиса соединения), и новый родственный подпоток открывать не нужно, так как скорость поступления данных с уровня приложения не позволит отправлять их в сеть с большей скоростью;

2) Приложение полностью заполняет список sk_buff, размер которого ограничен размером буфера памяти, выделяемого для этого списка при инициализации сокета, и блокируется, пока не освободится место в списке. Сегменты из списка удаляются, когда приходит подтверждение об их доставке получателю. В итоге получатель будет добавлять данные в список со скоростью, с которой транспортный агент FDMP отправляет данные, т.е. $V_{FDMP} < V_{QoS}$, но

новый родственный подпоток нужно открыть для увеличения скорости FDMP соединения.

Указанные случаи неразличимы в формулах (1)-(2), поэтому для окончательного определения ситуации, когда необходимо открыть новый родственный подпоток, нужно учитывать данные мониторинга буфера sk_buff.

Как вариант, можно было бы при полном заполнении списка sk_buff сигнализировать о том, что буфер полон и при условии, что скорость FDMP соединения меньше требования качества сервиса, открывать новый подпоток. Однако передача данных с уровня приложения на транспортный уровень может быть неравномерной. Рассмотрим для примера передачу потокового видео, в которой кодек MPEG4 [51] для сжатия и распаковки некоторых видео-фреймов использует как уже сформированные видео-фреймы, так и последующие видео-фреймы, формирование которых необходимо ждать. В результате передача имеет характерные всплески активности в определенные моменты времени, вне которых передача данных не ведется (рисунок 5). Во время передачи данных список sk_buff может заполниться полностью, однако во время затишья буфер освобождается, и канал в это время пустует. В данном случае открытие нового подпотока не потребуется, так как новый подпоток будет большей частью бездействовать, а с отправкой накопленных данных в буфере справятся и другие подпотоки во время бездействия.

Поэтому транспортный агент FDMP оценивает среднюю заполненность буфера, и в случае, если в среднем список sk_buff заполнен на K% (назовем это условие IS_BUFF_FULL) и скорость FDMP соединения меньше требования качества сервиса, нужно открыть новый родственный подпоток.

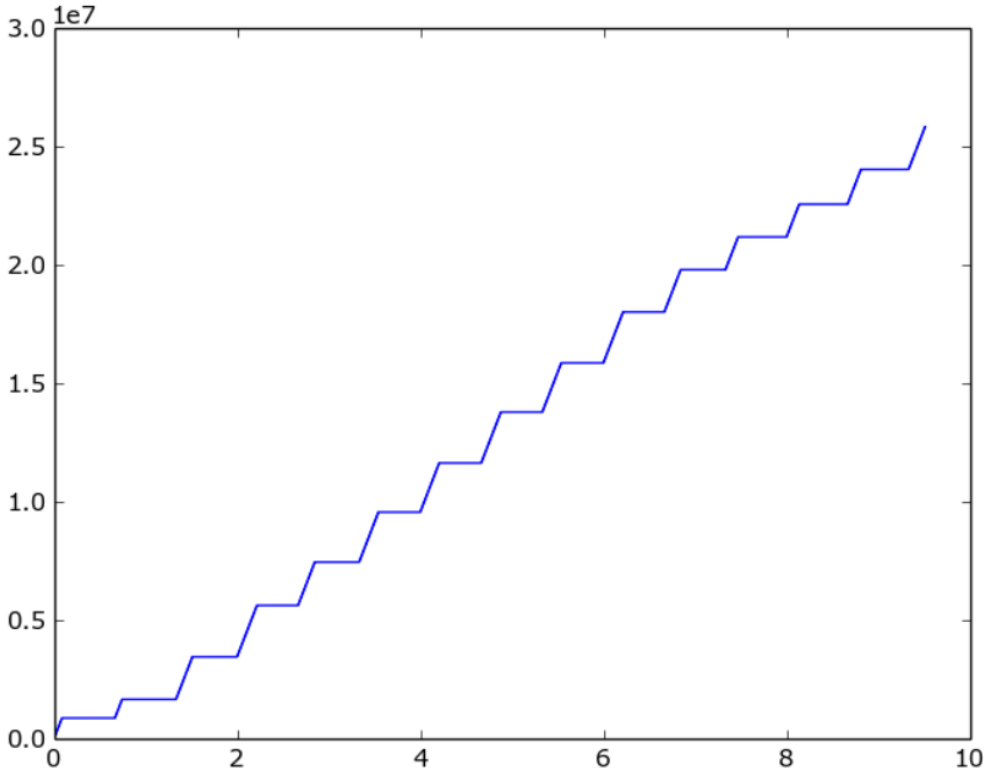

Рисунок 5. График зависимости числа переданных битов потокового видео от времени (сек)

Итак, условие открытия подпотока можно записать в следующем виде:

$$IS_BUFF_FULL \ \&\& \ (V_{FDMP} < V_{QoS}) \ (3)$$

4. *Модель работы транспортного агента FDMP*

Теперь рассмотрим предлагаемую в настоящей работе модель работы транспортного агента FDMP, согласно которой происходит изменение числа родственных подпотоков. Её идея схожа с идеей алгоритмов управления перегрузками в сети. В этих алгоритмах размер окна перегрузки cwnd меняется в зависимости от ситуации в сети: при неполной загруженности канала размер окна увеличивается, а при полной загрузке — уменьшается. Аналогично приведенному примеру, FDMP протокол динамически изменяет количество родственных подпотоков: если скорость соединения не удовлетворяет требованиям качества сервиса, то открывается новый подпоток, если же скорость соединения превышает выданные ограничения, то мы пытаемся закрыть лишние подпотоки. Закрытие подпотоков необходимо для освобождения избыточно используемых ресурсов операционной системы (например, память, выделяемая для подпотока), а также

ресурсов сети (например, правила в таблице коммутации). Данную модель можно представить в виде диаграммы состояний конечного автомата (рисунок 6).

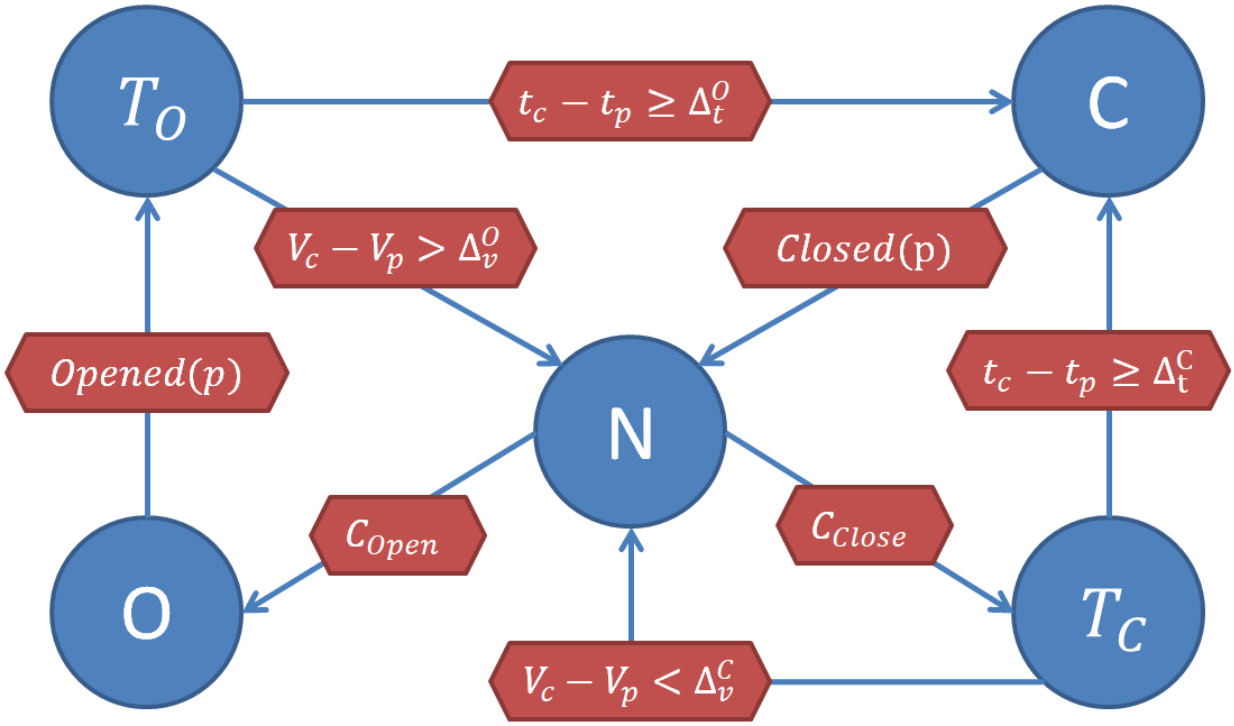

Рисунок 6. Диаграмма состояний модели работы транспортного агента.

Рассмотрим представленные на рисунке 6 состояния:

• N — нейтральное состояние, в нем мы не открываем и не закрываем подпотоки, а лишь проверяем требования качества сервиса с периодом t_N. Если условие открытия подпотока (3) невозможно проверить из-за маленького количества элемента в буфере qos_buff, то мы остаемся в нейтральном состоянии. Также может получиться, что условие открытия подпотока (3) выполнено, но мы не можем открыть новый подпоток в силу ограничения на количество подпотоков, тогда мы остаемся в нейтральном состоянии.

• O — состояние открытия нового подпотока. В это состояние можно перейти из нейтрального при выполнении условии (3) открытия нового подпотока.

• T_O - тест на открытие нового подпотока. В данном состоянии каждые t_O секунд проверяется изменение скорости соединения с новым дополнительным подпотоком по сравнению с прежней скоростью без этого подпотока. В случае

увеличения скорости мы переходим в нейтральное состояние, иначе данный подпоток не приносит никакой пользы, лишь растрачивая ресурсы операционной системы и ресурсы сети, поэтому мы закрываем открывшийся подпоток в состоянии C.

• C — состояние закрытия подпотока. Прерываем соединение на данном подпотоке, посылая сегменты с флагом RST.

• T_{C} - тест на закрытие подпотока. В это состояние мы попадаем из нейтрального в случае, если пропускная способность многопоточного соединения больше требования качества сервиса и количество подпотоков больше одного. В данном состоянии мы выбираем случайным образом подпоток, через который перестаем отправлять данные, (в текущей реализации используется round-robin). Каждые t_{C} секунд мы проверяем условие (3) и в случае его выполнения переходим в состояние восстановления R, так как меньшее количество подпотоков не способно удовлетворить требованиям качества сервиса на данный момент. Если же в течение T_{C}^{max} секунд условие (3) не выполняется, то мы переходим в состояние закрытия, где закроем этот подпоток. T_{C}^{max} является параметром транспортного агента.

• R — состояние восстановления. На время R^{max} не происходит никаких проверок, так как на значение средней скорости влияла приостановка отправления данных на один из подпотоков при тесте на закрытие. R^{max} является параметром транспортного агента. Затем происходит переход в нейтральное состояние.

1.4 Многопоточная маршрутизация в ПКС

Рассмотренные в предыдущих разделах многопоточные протоколы работают внутри транспортных агентов операционных систем, которые не обладают никакой информации о структуре сети. Поэтому задача выбора и установки маршрута является задачей самой автономной системы сети.

Одним из условий для увеличения скорости многопоточного соединения является отсутствие пересечений между маршрутами родственных подпотоков. Это условие связано с тем, что на месте пересечения маршрутов родственных

подпотоков может образоваться *узкое место в сети* – канал с наименьшей остаточной пропускной способностью. Тогда родственные подпотоки не будут увеличивать скорость многопоточного соединения, а будут разделять ресурсы одной и той же линии связи, хотя этими ресурсами мог обладать один подпоток.

Однако стоит отметить, что в сетях Интернет-провайдеров, представленных в библиотеки Topology Zoo [52], существуют топологии, в которых найдется как минимум одна пара узлов, не имеющих непересекающихся маршрутов между собой. Этот факт может быть связан с невозможностью физической прокладки запасного маршрута между рассматриваемыми узлами. В этом случае Интернет-провайдер может использовать линии с большей пропускной способностью, чтобы справиться с увеличенной нагрузкой, которая может поступать на этот маршрут из нескольких мест. Для такого случая, когда в сети физически отсутствуют непересекающиеся маршруты, мы будем рассматривать задачу построения маршрутов, имеющих минимальное число пересечений.

В традиционной сети в случае наличия нескольких маршрутов проводится балансировка транспортных потоков между этими маршрутами. Определение маршрута, по которому отправится транспортный поток, зачастую происходит на основании хэш-функции, взятой от значений полей заголовков канального, сетевого и иногда транспортных уровней. Даже если для расчета хэш-функции используется информация о транспортных портах из заголовка транспортного уровня, которые будут отличаться для родственных подпотоков, никто не может гарантировать, что хэш-функция не выдаст одинаковые значения для родственных подпотоков. А если информация из транспортного заголовка не используется, то родственные подпотоки будут иметь одинаковые маршруты.

Помимо задачи распределения родственных подпотоков между маршрутами возникает задача построения этих маршрутов. В традиционной сети эту задачу решить сложнее из-за децентрализованного управления, т.е. отсутствия текущего состояния сети. Поэтому в традиционных сетях для поддержки многопоточной маршрутизации необходимо будет добавлять дополнительную функциональность

маршрутизации многопоточных соединений в сетевое оборудование, что усложнит развертывание многопоточной маршрутизации.

В программно-конфигурируемых сетях [53] используется централизованное управление, которое выносится в отдельное логическое устройство, которое далее будем называть контроллером ПКС. Контроллер ПКС знает текущее состояние сети и может управлять установкой построением маршрутов. Рассмотрим подходы к решению этой задачи.

Множество работ [54, 55], посвященных исследованию многопоточной маршрутизации в ПКС, используют для построения маршрутов жадный алгоритм, который далее в работе будем называть Greedy Shortest Path First (GSPF). На каждой итерации алгоритм GSPF строит наикратчайший маршрут для нового родственного подпотока (например, при помощи алгоритма Дейкстры [56]). В конце каждой итерации к ребрам маршрута, полученного алгоритмом GSPF, добавляется штраф. Штраф подбирают так, чтобы маршруты с пересечением были менее предпочтительны, чем маршруты без пересечений.

Преимуществами алгоритма GSPF являются его простота для поиска нового маршрута для родственного подпотока по запросу. Однако алгоритм GSPF не всегда приводит к оптимальному решению, т.е. не всегда находит непересекающиеся маршруты, когда они есть. Например, рассмотрим топологию, изображенную на рисунке 7. Пусть необходимо передать данные, с помощью динамического многопоточного протокола. Предположим, что для этого используется два родственных подпотока между узлами A и B. Тогда алгоритм GSPF для первого подпотока построит наикратчайший маршрут (будем считать, что метрикой расстояния является количество сетевых узлов на маршруте), отмеченный красным цветом. Но нельзя подобрать ни одного маршрута для второго родственного подпотока, без пересечения с маршрутом первого подпотока. Однако само множество непересекающихся маршрутов существует и изображено на рисунке 8 (красным и фиолетовым цветом соответственно).

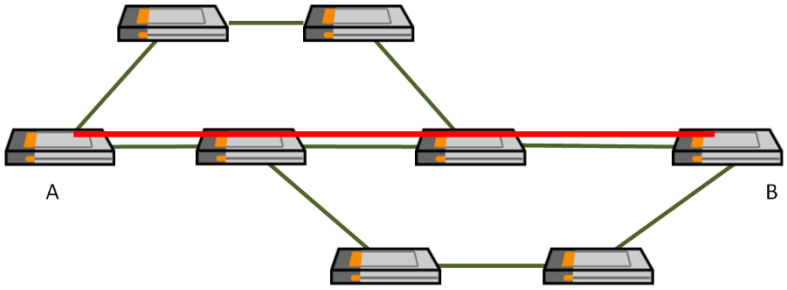

Рисунок 7. Пример неоптимальной работы алгоритма GSPF.

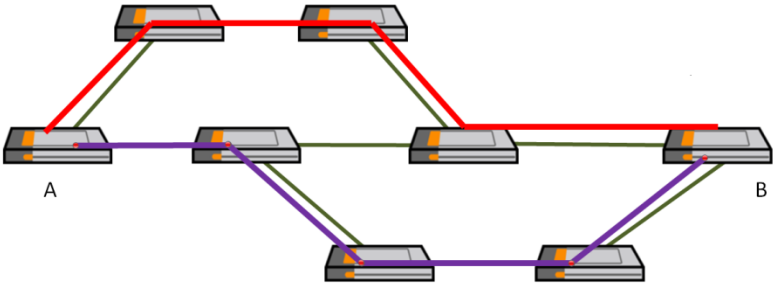

Рисунок 8. Пример непересекающихся маршрутов, не найденных алгоритмом GSPF.

Таким образом, алгоритм GSPF не подходит для поиска непересекающихся маршрутов. Для этой цели можно использовать алгоритм построения избыточных деревьев (Maximum Redundant Tree – MRT) [57]. Несколько построенных деревьев MRT позволяют отправлять пакеты не только по дереву кратчайших расстояний, но и балансировать транспортные потоки по MRT-деревьям. Если рассматривать маршруты от одной и той же точки до корня в разных MRT деревьях, то можно получить множество маршрутов с минимальным количеством пересечений. Однако использующийся в сетях алгоритм [57] позволяет строить только 2 дерева, т.е. можно получить только 2 маршрута с минимальным пересечением. Стоит отметить, что сложность алгоритма построения двух деревьев линейно зависит от количества вершин в топологии. Существуют алгоритмы построения трех MRT деревьев [58] и четырех MRT деревьев [59] с квадратичной и кубической сложностью соответственно, однако эти алгоритмы работают для специальных

видов графов (например, трех реберно-связных [58]). Работ по построению пяти и больше MRT деревьев найдено не было.

В диссертационной работе предлагается алгоритм построения маршрутов, который подходит для любых видов связных графов. В отличие от алгоритма GSPF, предложенный алгоритм находит оптимальное решение, по критерию минимизации штрафа за количество пересечений. Функция штрафа в зависимости от количества маршрутов, проходящих через одно и то же ребро графа сети, изменяется, так как ситуация, когда все маршруты имеют пересечение по одному ребру, с точки зрения отказоустойчивости, хуже ситуации, когда маршруты имеют разные попарные пересечения. Поэтому штраф для одного ребра, на котором есть пересечение, будет определяться как $|V|^z$, где z – количество маршрутов, проходящих через это ребро. Если в топологии существуют маршруты без пересечений, то минимальный штраф будет равен нулю и результатом работы алгоритма должны быть маршруты без пересечения. Далее мы также будем называть маршруты с минимальным штрафом маршрутами с минимальным пересечением, так как дополнительные пересечения будут увеличивать функцию штрафа. В предлагаемом алгоритме задача построения маршрутов сводится к задаче поиска максимального потока минимальной стоимости (Min-Cost Max-Flow), поэтому далее будем называть этот алгоритм MCMF.

В алгоритме MCMF сначала преобразуют исходный граф. Каждое ребро исходного графа заменяется на k параллельных ребер, где k является входным параметром алгоритма. Для каждого исходного ребра графа получившиеся параллельные ребра нумеруются произвольным способом. Для каждого множества параллельных ребер стоимость i-го ребра ($1 \leq i \leq k$) устанавливается равной $|V|^{i-1}$, где под $|V|$ понимается число ребер в исходном графе. Это необходимо, чтобы обеспечить как можно меньшее количество пересечений. Для графов с большим количеством ребер можно уменьшить стоимость, если известен диаметр графа. Если маршруты родственных подпотоков будут иметь пересечение в исходном графе, то в преобразованном графе маршруты этих же родственных

подпотоков будут проходить через разные параллельные ребра. Для каждого нового параллельного ребра стоимость возрастает, так как большее число родственных подпотоков, маршруты которых проходят через одно и то же ребро, может привести к большей конкуренции за ресурсы соответствующей линии связи.

Для всех ребер в преобразованном графе емкость устанавливается в 1. Наконец, к преобразованному графу добавляется дополнительная вершина - *сток*, которая соединяется с вершиной назначения в графе ребром с емкостью k. Стоимость ребра, инцидентного стоку, можно назначить произвольным образом.

Финальным шагом алгоритма MCMF является отображение маршрутов в преобразованном графе в маршруты в исходном графе. Хотя такое отображение может быть не единственным, каждый получившийся набор маршрутов будет иметь минимальное пересечение. Доказательство этого утверждения представлено ниже.

Для графа, изображенного на рисунке 7, преобразованный граф согласно работе алгоритма MCMF при $k = 2$ изображен на рисунке 9. В скобках для каждого ребра указаны его емкость и стоимость соответственно.

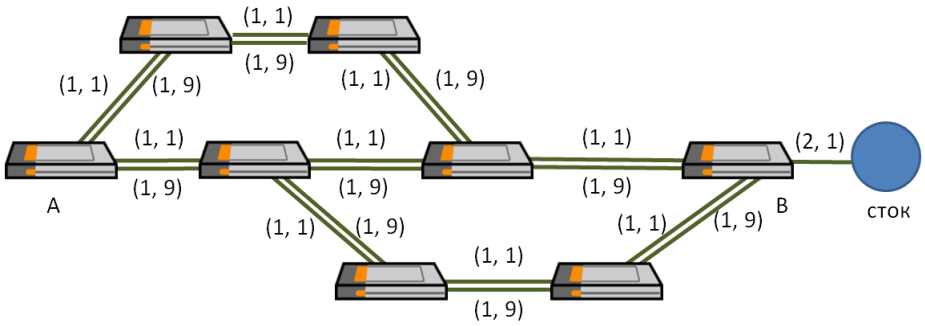

Рисунок 4. Пример преобразованного графа в алгоритме MCMF.

Теорема. Алгоритм MCMF для любого связного графа, для любого наперед заданных параметра k, вершин источника и назначения всегда находит k маршрутов с минимальным пересечением между этими вершинами

Доказательство корректности: докажем, что при заданном k в результате работы алгоритма MCMF мы получим k маршрутов, имеющих минимальное

пересечение (минимальный штраф). В преобразованном графе вычисляется максимальный поток минимальной стоимости между вершиной отправления и стоком. С одной стороны, получившийся поток получит не менее k единиц емкости, так как алгоритм MCMF заменяет каждой ребро исходного графа k ребрами с единичной емкостью. С другой стороны, получившийся поток получит не более k единиц емкости из-за узкого места перед стоком. Таким образом, максимальный поток получит ровно k единиц емкости, которые будут распределены по k различным маршрутам в преобразованном графе. Предположим, что существуют k маршрутов, штраф за которых меньше, чем у маршрутов в результате работы алгоритма. Тогда в преобразованном графе существует поток, который имеет еще меньшую стоимость. Однако алгоритм MCMF ищет максимальный поток минимальной стоимости, что противоречит сделанному предположению. Таким образом, предложенный алгоритм построит k маршрутов с минимальным пересечением. ***Теорема доказана.***

При помощи предложенного алгоритма MCMF можно найти k маршрутов в графе сети, имеющих минимальное число пересечений. Однако для применения алгоритма MCMF совместно с динамическим методом многопоточной маршрутизации необходимо дополнительно проверить, что остаточная пропускная способность на маршрутах позволит удовлетворить требования к качеству сервиса.

Отдельным вопросом встает выбор k – какое количество маршрутов необходимо строить при помощи алгоритма MCMF. Этот вопрос напрямую зависит от нескольких факторов. Во-первых, от избыточности, которую может предоставить сама топология. Под термином *степень демультиплексирования* графа сети будем понимать арифметическое среднее значение по всем возможным парам вершин графа максимального количества непересекающихся маршрутов между парой вершин. На рисунке 10 показаны результаты исследования топологий Интернет-провайдера из библиотеки Topology Zoo [52], из которого

следует что степень демультиплексирования для большинства топологий не превосходит трех. Поэтому в качестве k для текущих топологий Интернет-провайдеров достаточно строить по 3 маршрута. В проведенном исследовании степень демультиплексирования считалась при помощи алгоритма MCMF, который ищет маршруты с минимальным пересечением. Максимальное число маршрутов без пересечений искалось последовательным применением алгоритма MCMF с увеличением k до появления первого пересечения. При применении аналогичного подхода с алгоритмом GSPF в 91 случае из 259 степень демультиплексирования оказывалась меньше, максимальная разница могла составить 0,12.

Во-вторых, количество маршрутов зависит от запрашиваемого приложением качества сервиса и остаточных ресурсов сети. В работе предлагается строить количество маршрутов, определяемое степенью демультиплексирования, но выдавать многопоточному соединению такое количество маршрутов, которое позволит удовлетворить запрашиваемые требования, т.е. выдаваемое количество маршрутов может оказаться меньше построенного множества маршрутов. Если построенного множества маршрутов не хватает для обеспечения требуемого качества сервиса, то происходит отказ в обслуживании многопоточного соединения.

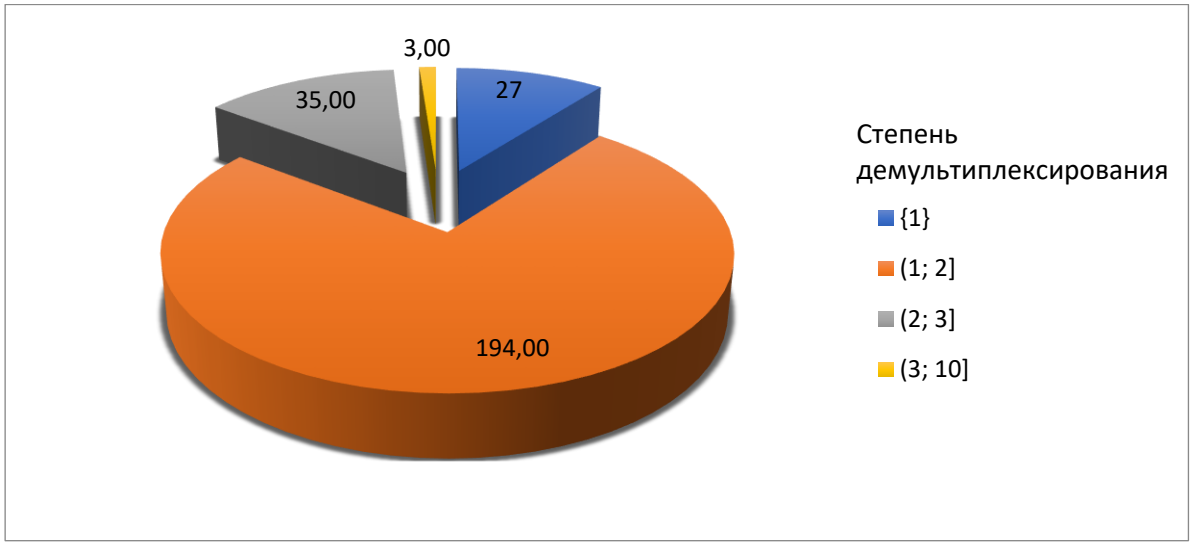

Рисунок 10. Соотношение количества топологий с разной степенью демультиплексирования

Также немаловажным вопросом является установка правил в ПКС для реализации построенных маршрутов многопоточного соединения. Существуют две стратегии работы контроллера ПКС: *реактивная* и *проактивная*. В *реактивной* стратегии контроллер будет строить маршрут и устанавливать правила по наступлению некоторого события, например, появлению первого пакета нового многопоточного соединения в сети. Для этого контроллер ПКС настраивает подконтрольные ему коммутаторы для перехвата всех пакетов TCP рукопожатий и отбирает те пакеты, которые относятся к многопоточному соединению. В терминах протокола OpenFlow [60], рассматриваемого в работе как протокола взаимодействия между контроллером ПКС и подконтрольными коммутаторами, контроллер устанавливает на коммутаторы низкоприоритетные правила для отправки PacketIn сообщений на контроллер для каждого TCP пакета. При поступлении PacketIn сообщения контроллер анализирует опциональную часть транспортного заголовка и ищет опции, относящиеся к многопоточному протоколу MPTCP или FDMP. Если соответствующие опции присутствуют, то контроллер будет воспринимать этот TCP поток как подпоток многопоточного соединения.

Чтобы определять, к какому многопоточному соединению относится рассматриваемый подпоток, контроллер сохраняет информацию о каждом подпотоке в SIB (Subflow Information Base). При поступлении первого подпотока многопоточного соединения в сеть контроллер извлекает ключ, передаваемый в опции MP_CAPABLE. Если пакет соответствует инициации соединения, т.е. пакету с флагом SYN, то создается новая запись в SIB и мы сохраняем ключ отправителя. Иначе запись уже создана в SIB и мы сохраняем ключ получателя. На основе двух ключей контроллер может вычислить хеш, который будет передаваться в последующих родственных подпотоках. Обнаружение рассчитанного хеша в пакетах других подпотоков будет означать принадлежность неизвестного до того момента подпотока к многопоточному соединению, информацию о котором мы только сохранили в SIB.

При помощи SIB контроллер может определять родственные подпотоки и прокладывать для них маршруты с наименьшим количеством пересечений. Последующие пакеты после процедуры трехкратного рукопожатия должны идти согласно правилам, установленным для соответствующих маршрутов. Каждый подпоток определяется правилом OpenFlow по паре IP адресов и паре транспортных портов. Для того чтобы поддерживать передачу пакетов в обе стороны, контроллеру необходимо установить правил в два раза больше длины маршрута.

Реактивная стратегия обладает следующими недостатками:

1. Множество сетевых приложений таких как, средства мониторинга, распределенные базы данных, инициируют тысячи новых транспортных соединений в секунду. Количество правил, сгенерированных реактивной стратегией, будет занимать практически всю память даже флагманских коммутаторов ПКС, таких как, NoviSwitch 2122, который поддерживает до 1 миллиона шаблонных и до 6 миллионов с точным соответствием записей в своей таблице [61]. Так как обычно используются более дешевые коммутаторы, то для реактивной стратегии не будет хватать записей в коммутаторах ПКС.
2. Помимо генерации слишком большого количества правил, реактивная стратегия генерирует их слишком интенсивно. Коммутаторы ПКС обычно имеют менее производительный процессор для работы с контуром управления, чем с контуром данных, который может в лучшем случае обработать тысячи обновлений таблицы в секунду. Контроллеры ПКС обладают большими вычислительными возможностями и могут обрабатывать несколько миллионов поступлений PacketIn в секунду [62]. Однако нетривиальная обработка пакетов многопоточного соединения, возможно, уменьшит это число на несколько порядков. Если скорость, с которой приложения открывают или закрывают свои транспортные соединения, превосходит возможности аппаратуры, сеть не сможет обрабатывать их.

3. Реактивная стратегия вносит значительную задержку в процедуру трехкратного рукопожатия. При получении первого пакета нового транспортного потока коммутатор ПКС генерирует PacketIn сообщение на контроллер. Контроллер анализирует поступивший PacketIn, рассчитывает маршруты и посылает FlowMod сообщения для установки маршрута. На практике суммарное время перечисленных действий превосходит 50 миллисекунд, что может соответствовать продолжительности некоторых транспортных соединений и неприемлемо для приложений реального времени.

Хотя перечисленные недостатки не делают реактивную стратегию непрактичной, её нельзя применять в сетях Интернет-провайдера непосредственно.

В противовес реактивной стратегии *проактивная* стратегия предполагает предварительную загрузку правил пересылки пакетов на этапе инициализации контроллера ПКС. Следовательно, коммутаторы ПКС должны обрабатывать пакеты полностью самостоятельно, без вмешательства контроллера ПКС. В частности, коммутаторы ПКС должны распределять родственные подпотоки по разным маршрутам, имеющим минимальное количество пересечений. Сами маршруты могут быть построены предварительно и соответствующие правила могут быть установлены заранее, однако MPTCP или FDMP пакеты не несут никаких дополнительных признаков, которые могут помочь в определении принадлежности тому или иному многопоточному соединению. Можно использовать хеш функции для балансировки по построенным маршрутам, однако, как было отмечено ранее, отсутствует гарантия того, что родственные подпотоки отправятся разными маршрутами.

Одним из возможных подходов для предотвращения совпадения маршрутов родственных подпотоков может стать отслеживание многопоточных соединений в коммутаторах ПКС так же, как это сделано в реактивной стратегии в контроллере ПКС. Однако предпочтительней будет подход, когда транспортные агенты

помогают коммутаторам определять родственные подпотоки. Например, пакеты родственных подпотоков могут содержать разное значение DSCP метки в IP заголовке. Осуществлять сравнение по метке DSCP должны уметь все коммутаторы ПКС, поддерживающими работу по протоколу OpenFlow версии не ниже 1.0, и реализовано большинством современных коммутаторов ПКС. Так как DSCP маркировка является широко распространенной процедурой, её будет проще реализовать, чем анализ опций транспортного заголовка. Наконец, несмотря на факт, что DSCP поле состоит только из 6 бит, большинство из 64 значений никогда не используется на практике – обычно поддерживается не более 14 различных классов трафика, рекомендованных стандартом RFC [63]. Таким образом, в DSCP поле достаточно места, чтобы закодировать по крайней мере 3 разных подпотока для соединения любого класса трафика.

Более радикальная, но готовая к реализации идея, заключается в обеспечении каждого хоста множеством из k сетевых адресов, в которых закодирован номер родственного подпотока на некоторых зафиксированных позициях. Хотя эта идея может быть неприменима для ограниченного пространства адресов IPv4, она становится осуществимой для IPv6 адресов, позволяющих использовать 64 бита для идентификации хоста. Сам хост может определить, использовать все адреса на одном сетевом интерфейсе или распределить их между несколькими сетевыми интерфейсами. Если транспортный агент многопоточного протокола настроить таким образом, чтобы i-ый родственный подпоток отправлялся от i-го адреса, то идентификация подпотоков внутри сети становится тривиальной. В этом случае коммутаторы ПКС должны перенаправлять пакеты i-го родственного подпотока на i-ый маршрут, проложенный заранее.

Хотя проактивная стратегия устраняет, сглаживает, смягчает недостатки реактивной стратегии, она тоже обладает несколькими недостатками:

1. Для реализации проактивной стратегии поддержки многопоточной маршрутизации необходимы изменения либо в коммутаторах ПКС, либо на хостах, что затрудняет распространение многопоточных протоколов.

2. Проактивная стратегия намного хуже сможет поддерживать динамический метод многопоточной маршрутизации, так как на контроллере ПКС заранее неизвестна нагрузка в сети, а коммутатор ПКС не обладает видением состояния всей сети, поэтому определить, какие маршруты надо использовать для обеспечения требуемого качества сервиса.
3. Проактивная стратегия не решает проблему с количеством правил для осуществления многопоточной маршрутизации, которая была описана для реактивной стратегии.

В работе предлагается использовать смешанную стратегию для реализации многопоточной маршрутизации в ПКС. Контроллер ПКС заранее строит маршруты с минимальным количеством пересечений между каждой парой точек в сети. Количество маршрутов определяется из степени демультиплексирования рассматриваемой сети. Построив маршруты заранее, мы можем уменьшить задержку на установление соединения, так как не требуется пересчитывать их каждый раз при поступлении нового подпотока. Кроме того, мы можем заранее установить правила для каждого из маршрутов за исключением точки входа и точки выхода, тем самым сократив время, затрачиваемое на установку маршрута, оставив установку только двух правил. В динамике контроллер ПКС должен поддерживать состояние этих маршрутов, где под состоянием понимается остаточная пропускная способность маршрута. Контроллер ПКС может получить состояние маршрута, собирая статистику об объеме передаваемого трафика в единицу времени (будем называть эту величину загрузкой линии связи) на каждой линии связи на маршруте и беря минимум из разностей пропускной способности и текущей загрузки. Тогда при появлении нового многопоточного соединения в сети можно оценить, хватает ли ресурсов построенных маршрутов для обеспечения Стребуемого качества сервиса для этого класса потоков или нет. Если ресурсов маршрутов хватает для удовлетворения требований рассматриваемого

многопоточного транспортного соединения, то за этим транспортным соединением резервируются необходимые ресурсы.

Для того чтобы контроллер ПКС узнал о требованиях многопоточного транспортного соединения можно использовать несколько подходов. Например, можно зафиксировать классы приложений, где для каждого класса будут запрашиваться одинаковые требования к качеству сервиса. Тогда класс приложения можно закодировать в DSCP метке, которую может распознать контроллер ПКС. Вторым подходом может быть передача требований приложения в опциональной части транспортного заголовка, тогда контроллер ПКС сможет узнать о требовании, так как все равно анализирует опциональную часть транспортного заголовка.

Ресурсы, занимаемые многопоточным транспортным соединением, должны освобождаться при завершении соединения. Более того, ресурсы, занимаемые одним подпотоком, должны освобождаться при завершении этого подпотока, так как динамический метод многопоточной маршрутизации позволяет закрывать лишние подпотоки. Контроллер ПКС может узнать о завершении подпотока при помощи idle-timeout – таймера, который привязан к определенному правилу в OpenFlow таблице и ведет свой отсчет при отсутствии пакетов, которые соответствуют этому правилу. Если появляется новый пакет в сети, который соответствует этому правилу, то отсчет начинается сначала. По истечении отведенного времени в idle-timeout коммутатор ПКС пошлет служебное сообщение на контроллер ПКС с указанием, по какому правилу сработал idle-timeout. Используя эту информацию, контроллер ПКС сможет понять, какой из подпотоков завершил свою работу.

Так как для каждого многопоточного транспортного соединения происходит резервация ресурсов и при условии отсутствия других потоков в ПКС (или под эти потоки выделен отдельный резерв пропускной способности, который известен заранее), то можно использовать упрощенную реализацию со следующей моделью. Будем хранить текущее состояние каждой линии связи. В случае появления нового соединения MPTCP определяем резервируемую пропускную

способность для каждого маршрута родственных подпотоков этого соединения при помощи алгоритма постепенного заполнения [64], так как все подпотоки MPTCP открываются сразу, на маршрутах может быть неравномерная остаточная пропускная способность, и могут быть пересечения между маршрутами. В случае соединения FDMP определяем резерв пропускной способности, последовательно рассматривая каждый маршрут и забирая всю остаточную пропускную способность маршрута, пока не наберем необходимую для выполнения требования рассматриваемого потока. Предложенная модель позволяет не тратить ресурсы контура управления на мониторинг текущей загрузки линий связи.

Теперь рассмотрим решение проблемы, связанной с нехваткой памяти коммутаторов для правил, осуществляющих многопоточную маршрутизацию. Для уменьшения количества правил нельзя использовать маршрутизацию по адресу точки назначения, так как маршруты с наименьшим количеством пересечений не являются кратчайшими маршрутами, поэтому последняя часть маршрутов, построенных из разных точек отправления до одной точки назначения, может не совпадать. Поэтому многопоточная маршрутизация даже для одного подпотока требует большого количества правил, чем для однопоточного соединения TCP.

Для того чтобы уменьшить количество правил, необходимых для многопоточной маршрутизации, предлагается поделить множество всех маршрутов на группы. Будем считать, что внутри одной группы не происходит *коллизий* – ситуации, когда два маршрута в группе проходят через одну и ту же вершину v и оставшаяся часть маршрутов от v до точки назначения не совпадает. Тогда в рамках одной группы можно проводить маршрутизацию по адресу точки назначения. Даже если группа состоит всего лишь из одного маршрута, то можно получить преимущество, так как разные подпотоки, проходящие по одному и тому же маршруту, будут обрабатываться по одному и тому же правилу, а не по отдельному набору. Ценой использования групп будет дополнительное поле в заголовке пакета, чтобы отличать пакеты, которые должны идти по маршрутам из разных групп. Хотя для этой цели можно использовать различные дополнительные метки (VLAN, MPLS и т.д.), предлагается различать группы,

используя MAC-адресы. Поскольку Интернет-провайдер обычно обеспечивает только L3-связность, то контроллеры ПКС перезаписывают MAC-адреса на граничных коммутаторах ПКС, чтобы имитировать поведение традиционных маршрутизаторов и действовать в рамках их ожиданий. В случае использования MAC-адресов мы не будем вносить дополнительную служебную информацию в заголовок пакета, поэтому этот метод лишен дополнительных накладных расходов. Коммутатор ПКС на входе пакета в сеть должен перезаписать MAC-адреса меткой, которая будет обозначать номер группы. Коммутатор ПКС на выходе пакета из сети должен заменить MAC-адрес источника на свой собственный и MAC-адрес назначения на адрес следующего маршрутизатора.

Можно оценить, что максимальное число групп маршрутов будет не превышать $k * (n - 1)$, где k – количество родственных подпотоков, а n – число коммутаторов в сети. Данная оценка связана с тем, что для одной точки назначения могут существовать по k маршрутов для каждого из $(n - 1)$ оставшихся коммутаторов. Маршруты до разных точек назначения не влияют друг на друга, поэтому их можно объединять в одной группе. Однако суммарное количество групп на практике зависит от способа объединения маршрутов в группы.

Для разбиения маршрутов на группы, при которой их число будет минимально, в работе предлагается свести эту задачу разбиения к задаче раскраски графа. Рассмотрим граф G с $k * (n - 1)$ вершиной, где каждая вершина соответствует одному из маршрутов до фиксированной точки назначения. Для каждой пары маршрутов, имеющих коллизию, соединим вершины ребром. Тогда минимальное число групп будет соответствовать минимальному числу цветов, необходимых чтобы покрасить граф G таким образом, чтобы соседние вершины не имели одного цвета. Задача нахождения раскраски графа – хорошо известная NP-полная задача, и существует множество подходов, чтобы найти оптимальное или приближенное решения. Стоит отметить, что сложность решения представленной задачи не должна навредить производительности сети, так как эта задача должна решаться на этапе инициализации сети.

В настоящей работе была проведена оценка количества групп маршрутов для топологий из библиотеки Topology Zoo. Результат оценки приведен на рис. 11, где указано зависимость между числом групп и долей покрываемых ими топологий. Толстая линия соответствует группам маршрутов, построенных согласно MCMF алгоритму, в то время как тонкая линия соответствует группам, построенным с GSPF алгоритмом. На рисунке приведены результаты как для двух подпотоков, так и для трех подпотоков. Ни для одной из исследованных топологий не понадобилось более чем 41 группа, в то время как 80% случаев требуют менее чем 10 групп.

Хотя теоретическая оценка количества необходимых групп достаточно велика, требования реальных сетей намного ниже полученной теоретической оценки. Поскольку количество групп не превосходит битовую длину MAC-адреса, мы можем выделить отдельный бит-индикатор для каждой группы, т.е. для *i*-ой группы на всех кроме *i*-ой позиции биты устанавливаются в ноль. Такой подход позволяет нам совмещать одинаковые правила отправки вне зависимости от номеров групп, которые там задействованы, при помощи битовых масок.

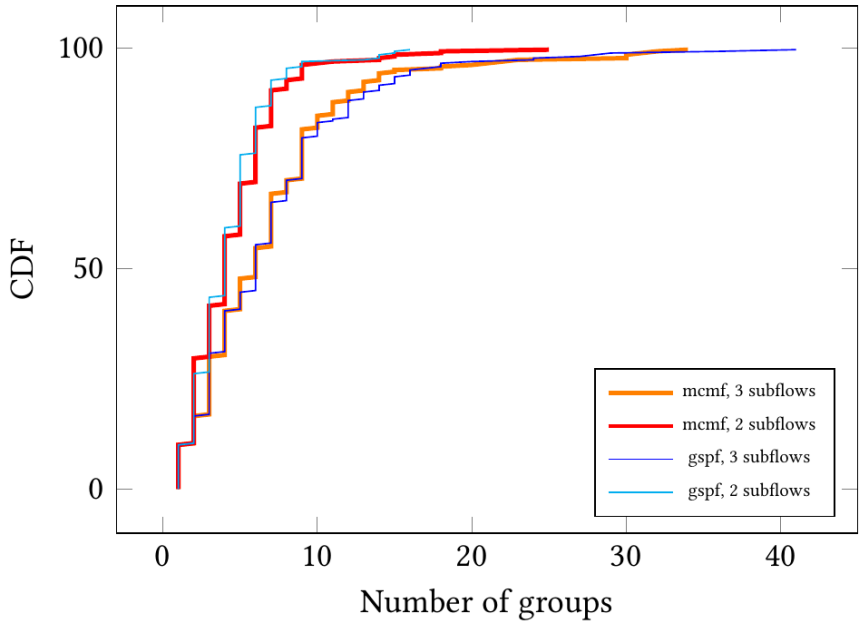

Рисунок 5. Число групп, достаточное для обработки многопоточных соединений между каждой пар узлов для разных топологий.

Также в работе была проведена оценка количества правил OpenFlow, которые получаются в случае реактивного подхода и в случае использования групп. В

каждом эксперименте предполагалось, что за каждым узлом топологии находится ровно одна IP подсеть и все IP подсети могут общаться между собой. Хотя некоторые префиксы подсетей могут быть объединены, подход CIDR в исследовании не использовался, так как результат зависит от схемы адресации. Таким образом, предложенный подход показывает минимальное количество правил, необходимых для реализации многопоточной маршрутизации с произвольной схемой адресации.

Рисунок 12 показывает количество OpenFlow правил, необходимых для реализации многопоточной маршрутизации реактивным подходом (наивный подход) и предложенным подходом с группами, составленными из маршрутов, построенных алгоритмом MCMF. На рисунке 13 представлена оценка минимального размера таблицы потоков протокола OpenFlow, необходимая для установки правил с рисунка 12. Так как результаты для GSPF алгоритма практически идентичны с результатами для алгоритма MCMF, то они были опущены.

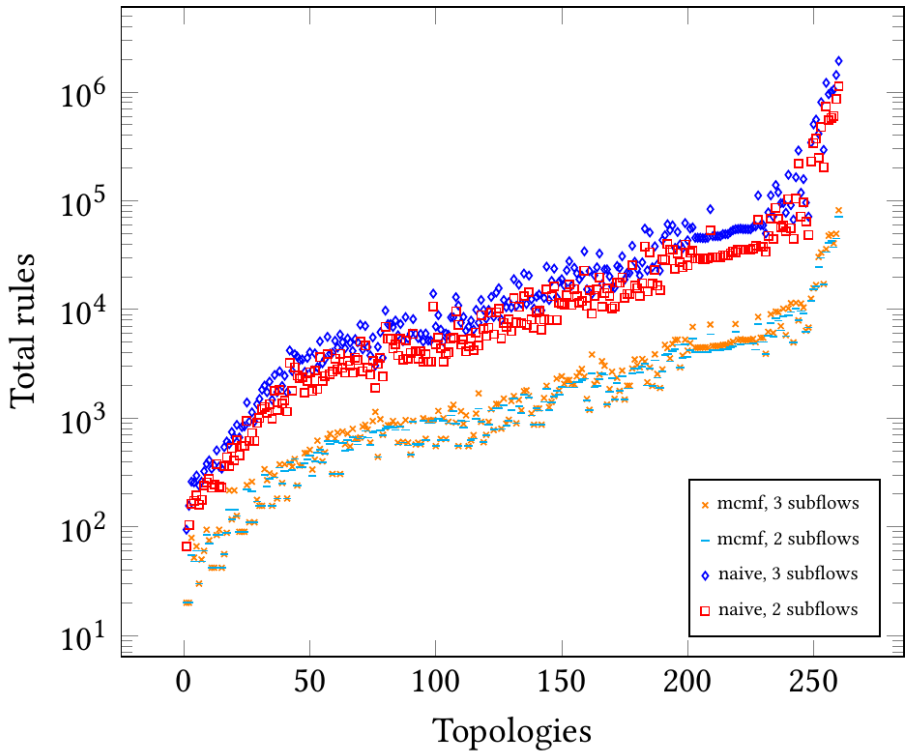

Рисунок 6. Количество OpenFlow правил, достаточное для обработки многопоточных соединений между каждой пар узлов для разных топологий.

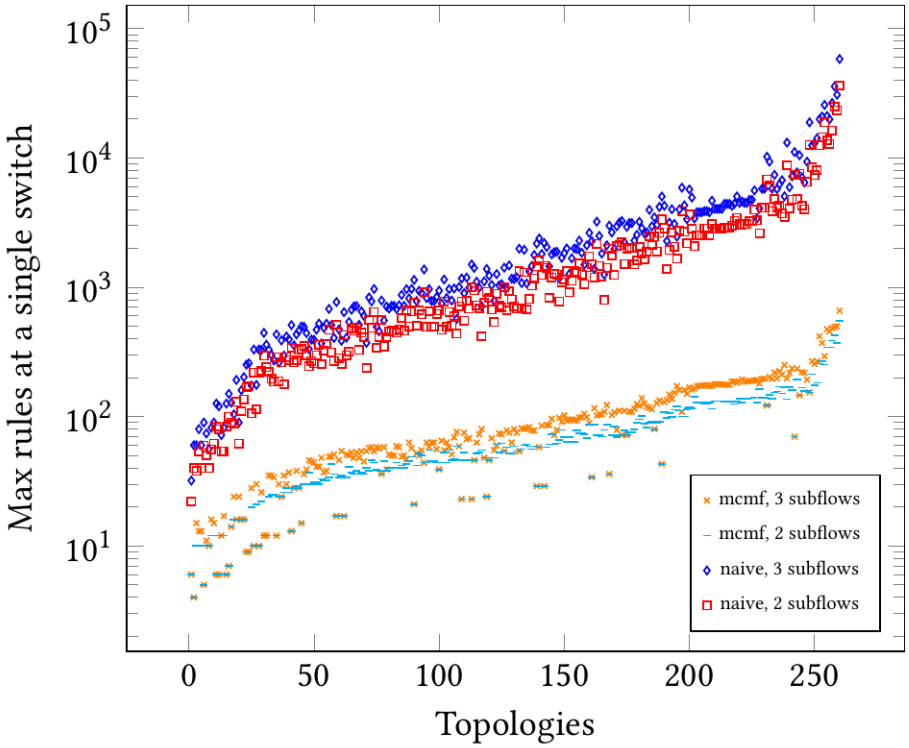

Рисунок 7. Размер таблицы потоков OpenFlow, достаточный для обработки многопоточных соединений между каждой пар узлов для разных топологий.

Согласно полученным результатам, наивный подход к построению правил требует на порядок больше правил, чем предложенный подход, основанный на группах маршрутов. Более того, преимущество предложенного подхода будет значительно возрастать, если будет увеличиваться количество многопоточных транспортных соединений между узлами сети. Как следствие, наивный подход требует большого размера таблиц потоков OpenFlow, а предложенный подход позволит обойтись более дешевыми коммутаторами ПКС.

1.5 Выводы

Существующие работы по анализу эффективности многопоточной маршрутизации в большей части анализируют только эффективность статического метода, а анализ динамического метода не проводился для сетей с фиксированной связью Интернет-провайдеров. Для проведения сравнительного анализа выбраны протоколы MPTCP и FDMP как представители статического и динамического методов. Хотя выбранные протоколы предоставляют TCP интерфейс для уровня

приложения, предлагаемая методика и результаты сравнительного анализа статического и динамического методов могут пригодиться и для новых протоколов, основанных на UDP протоколе, например, QUIC [65]. Так, для QUIC уже разработана модификация MP-QUIC [49], поддерживающая многопоточную маршрутизацию аналогично MPTCP, т.е. со статическим методом.

Для реализации многопоточной маршрутизации в ПКС предлагается использовать комбинацию реактивной и проактивной стратегий, а маршруты родственных подпотоков строить согласно предложенному алгоритму MCMF. Перечисленные подходы будут использованы в экспериментальном стенде для сравнительного анализа.

Глава 2. Сравнительный анализ эффективности статического и динамического методов многопоточной маршрутизации

В предыдущей главе были описаны статический и динамический методы многопоточной маршрутизации. Естественный вопрос для научного исследования, какой из этих методов лучше применять, и в каких случаях. Будем предполагать, что требования к скорости передачи данных для всех транспортных потоков в сети Интернет-провайдера известны. Тогда неформально задачу выбора метода многопоточной маршрутизации можно сформулировать следующим образом: выбрать такой метод многопоточной маршрутизации, который позволит удовлетворить требованиям как можно большего количества транспортных потоков в сети Интернет-провайдера за один и тот же интервал времени.

Для решения это задачи нам потребуется построить математическую модель функционирования сети передачи данных провайдера из рассматриваемого класса. Построить такую модель, в рамках которой рассматриваемая проблема имела бы аналитическое решение, не представляется возможным. Например, хотя бы потому, что очень многие параметры такой модели носят случайный характер: количество и интенсивность потоков данных. Подробное обсуждение этого утверждение увело бы нас в сторону от основной проблемы. Поэтому здесь мы не станем отвлекаться. Основным методом исследования было выбрано сочетание метода имитационного моделирования и статистических испытаний. В качестве основной гипотезы выдвигались предположения о вероятности выигрыша динамического метода многопоточной маршрутизации над статическим. Гипотезы проверялись на основе результатов проведения эмуляции работы методов многопоточной маршрутизации согласно методике, описанной в разделе 2.1, с ограниченным набором входных данных. В разделе 2.2 описаны требования к Стенду для проведения экспериментального исследования и его структура. Раздел 2.3 содержит описание области допустимых значений для входных параметров эксперимента. В разделе 2.4 приводятся результаты проведения экспериментального исследования, обосновывается корректность полученных

результатов и делается вывод об области применимости динамического и статического методов многопоточной маршрутизации.

2.1 Методика проведения сравнения статического и динамического методов многопоточной маршрутизации

Как уже было сказано выше, для проведения сравнения был использован метод проверки статистических гипотез, основанный на оценке попадания случайной величины – число потоков, требования которых по скорости были удовлетворены, в заданный интервал. Теория статистической проверки гипотез позволяет для заданного доверительного интервала гипотезы оценить количество экспериментов, которые необходимо выполнить, чтобы обеспечить требуемый уровень достоверности гипотезы.

2.1.1 Описание входных и выходных параметров эксперимента

Основными понятиями, используемыми для определения критериев сравнения и описания экспериментов, описаны ниже. Сравнительный анализ состоял из проведения серии экспериментов, каждый из которых характеризовался пятеркой $< N, F, T, M, sub_{max} >$, где:

- $N =< G, \rho >$ - транспортная сеть с топологией, представленной ориентированным графом $G = (V, E)$, на котором задана функция разметки $\rho: e \rightarrow (c(e), d(e), loss(e))$, где $e \in E, c(e) \in \mathbb{R}^+, d(e) \in \mathbb{R}^+$. Вершины графа - коммутаторы/маршрутизаторы сети, а дуги – линии. ρ характеризует качество сервиса связи линий: пропускную способность $c(e)$ линии, соответствующей дуге e, задержку передачи $d(e)$ на этой линии и вероятность потери пакета $loss(e)$.
- $F = \{f_i\}$ – множество потоков данных, передаваемых по сети N, каждый из которых представлен четверкой $< ip_i^{src}, ip_i^{dst}, tp_i^{src}, tp_i^{dst} >$, где $ip_i^{src}, ip_i^{dst}, tp_i^{src}, tp_i^{dst}$ - IP адреса и транспортные порты отправителя и получателя. Ясно, что такое представление уникально для каждого потока в сети – это фактически адрес на транспортном уровне отправителя и получателя.

- $T = [0, t_{max}]$ – дискретное время наблюдения за сетью с шагом δ.
- $M =< \phi, \psi, \tau >$ - представление свойств потоков, где:
    - $\phi: f_i \rightarrow (v_i^s, v_i^t)$ – распределение полюсов для каждого потока, где под полюсом понимается сток или исток потока. Каждый поток обладает своим истоком и стоком, соединенными с сетью в точках $v_i^s \in V$ и $v_i^t \in V$ соответственно. Предполагаем, что линии связи между истоком и сетью, а также между сетью и стоком обладают достаточной пропускной способностью для любого потока и нулевой задержкой.
    - $\psi: f_i \rightarrow (y_i, u_i)$ – требование к качеству потоков – Service Level Agreement (SLA) потока. SLA потока задает объем данных $y_i \in \mathbb{R}^+$, который необходимо передать за время $u_i \in \mathbb{R}^+$. Аналогично можно задать требование к скорости потока $q_i = \frac{y_i}{u_i}$.
    - $\tau: f_i \rightarrow t_i$ – расписание старта потоков, где $t_i \in T, t_i + u_i < t_{max}$.
- sub_{max} – максимальное количество родственных подпотоков. Напомним, что каждый поток передачи данных в наших экспериментах состоит из нескольких подпотоков, называемых родственными. В случае протокола MPTCP каждый поток f_i разбит ровно на sub_{max} родственных подпотоков, а в случае протокола FDMP количество задействованных в передаче данных родственных подпотоков может варьироваться, и может быть как меньше, так и равно sub_{max}.

Результатом каждого эксперимента является следующее отображение:

$$\theta: f_i \rightarrow \left\{L_{ij}, \{r_{ij}\}_t\right\}, \qquad t \in T,$$

где:

- $L_{ij} = (e_{ij}^1, e_{ij}^2, \dots, e_{ij}^n)$ – маршрут подпотока j потока f_i, представленный множеством упорядоченных дуг, $e_{ij}^k \in E$, где k - порядковый номер дуги в маршруте.

- $\{r_{ij}\}_t$ – временной ряд, элементы $r_{ij}(t)$ которого представляют среднюю скорость родственного подпотока j потока f_i на интервале $[t-\delta, t], t \in T, t \leq t_{max}$.

2.1.2 Критерии сравнения

Эффективность статического и динамического методов будем оценивать по следующему критерию - количество потоков, требования SLA которых было удовлетворено. Введем следующие обозначения.

Средней скоростью потока f_i *на интервале* $[t-\delta, t]$ назовем величину $r_i(t) = \sum_j r_{ij}(t)$. *Средней скоростью потока* f_i *в эксперименте* или просто *средней скоростью потока* f_i будем называть величину $r_i = \frac{\Sigma_T r_i(t)}{|\{t_k \in T: t_i \leq t_k \leq t_i + u_i\}|}$. Таким образом, средняя скорость потока – это среднее арифметическое средних скоростей потока на интервалах времени, начиная с момента старта потока и до его окончания, т. е. $t_i \leq t_k \leq t_i + u_i$. Вертикальными линиями || здесь и далее обозначаем количество элементов для множества, заключенного между линиями. Например, в формуле средней скорости потока $|\{t_k \in T: t_i \leq t_k \leq t_i + u_i\}|$ - обозначает количество дискретных моментов времени, заключенных между временем старта и временем окончания потока.

Стоимостью подпотока j *потока* f_i *на интервале* $[t-\delta, t]$ назовем величину $c_{ij}(t) = r_{ij}(t) * |L_{ij}|$. Величина $c_{ij}(t)$ показывает, какую часть пропускной способности линий связи занимает подпоток j потока f_i на рассматриваемом интервале. Чем больше длина маршрута, тем больше ресурсов сети затрачивается на передачу этого подпотока. *Стоимостью потока* f_i *на интервале* $[t-\delta, t]$ будет $c_i(t) = \sum_j c_{ij}(t)$. *Средней стоимостью потока* f_i *в эксперименте* или просто *средней стоимостью потока* f_i будем называть величину $c_i = \sum_T \frac{c_i(t)}{|\{t_k \in T: c_i(t_k) > 0\}|}$.

В терминах введенных обозначений количество потоков, для которых требование качества сервиса выполнено, можно описать так:

$$sf = |\{f_i: r_i \geq q_i\}|$$

Также определим стоимость потоков, SLA которых было удовлетворено:

$$c = \sum_{i:r_i \geq q_i} c_i$$

Использование этого параметра, определяемого в результате эксперимента, позволяет различать случаи, когда сравниваемые методы дают одинаковые значения по первому критерию – количество потоков с выполненным SLA, но требуют разных затрат ресурсов. Например, рассмотрим топологию сети «треугольник», изображенную на рисунке 3 на странице 21. Если в качестве требования к скорости многопоточного соединения выбрать $\frac{2}{3}$, то и статический метод, и динамический метод многопоточной маршрутизации позволят удовлетворить все потоки. Однако в случае статического метода потребуется два родственных подпотока со стоимостью $\frac{1}{3} * 1 + \frac{1}{3} * 2 = 1$ (маршруты родственных подпотоков будут иметь длину 1 и 2 соответственно), а в случае динамического метода - только один подпоток, так как одного маршрута будет достаточно для удовлетворения требуемого качества сервиса, а его стоимость $\frac{2}{3} * 1 = \frac{2}{3}$. Общая стоимость использования статического и динамического методов будет равна 6 и 4 соответственно. В случае статического метода будет зарезервирована вся пропускная способность сети, а динамический метод позволит обслужить дополнительные транспортные потоки.

2.2 Описание Стенда

Для целей проведения имитационных экспериментов был разработан и построен Стенд, отвечающий следующим требованиям, составленным на основе описания входных и выходных параметров эксперимента, представленных в разделе 2.1.1:

1. строить имитационную модель сети N с заданными параметрами качества сервиса линий связи ρ.
2. генерировать потоки данных F согласно заданному расписанию τ с заданными параметрами SLA ψ.

3. направлять потоки данных в имитационную модель сети согласно заданному распределению полюсов ϕ.
4. управлять передачей потоков данных F с помощью эмуляторов многопоточных транспортных агентов протоколов MPTCP и FDMP с настраиваемым параметром sub_{max}.
5. строить маршруты $\{L_{ij}\}$ с наименьшим пересечением для родственных подпотоков.
6. собирать данные о скорости родственных подпотоках $\{r_{ij}\}_t$ для дискретных моментов времени T.

Структура разработанного Стенда показана на рисунке 14. Стенд состоит из программной и аппаратной части. Архитектура программного обеспечения представлена на рисунке 15. Ниже описаны назначение, основные функции и детали реализации каждой подсистемы Стенда, представленной на рисунке 15.

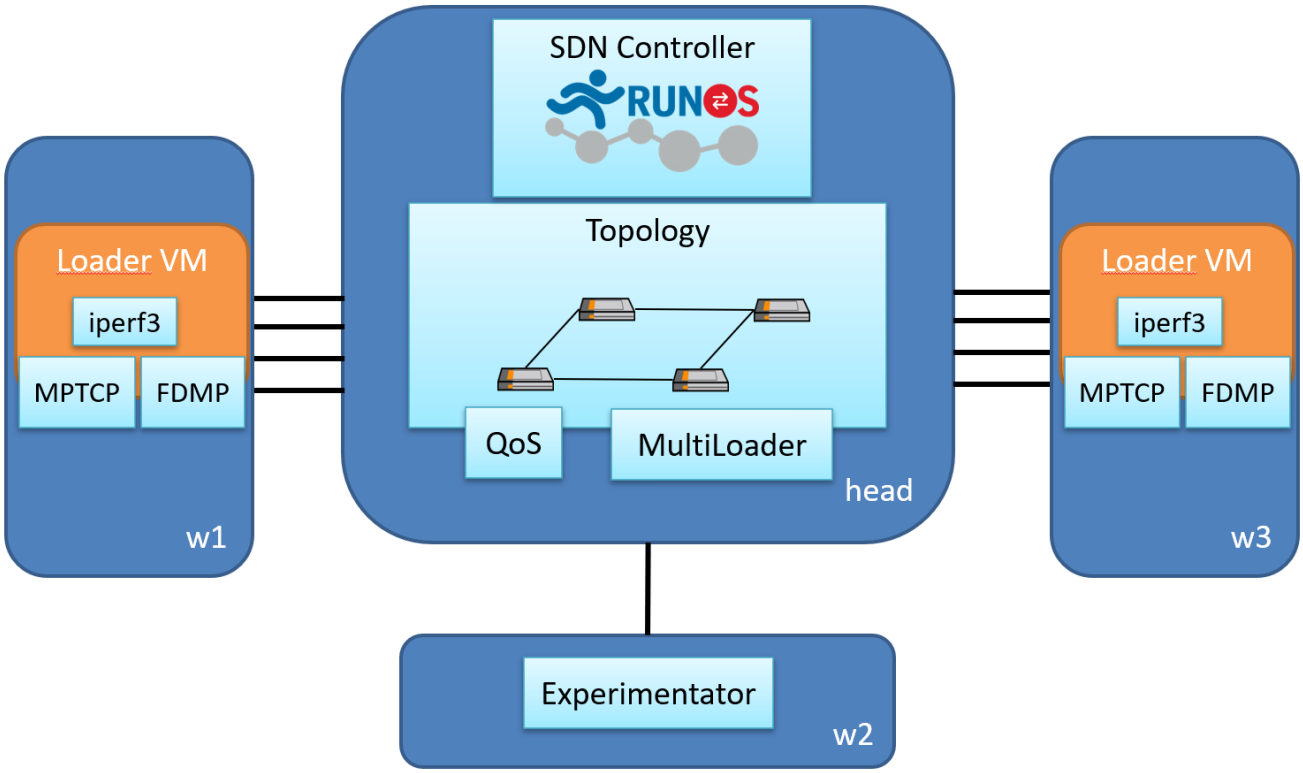

Рисунок 14 Структура Стенда

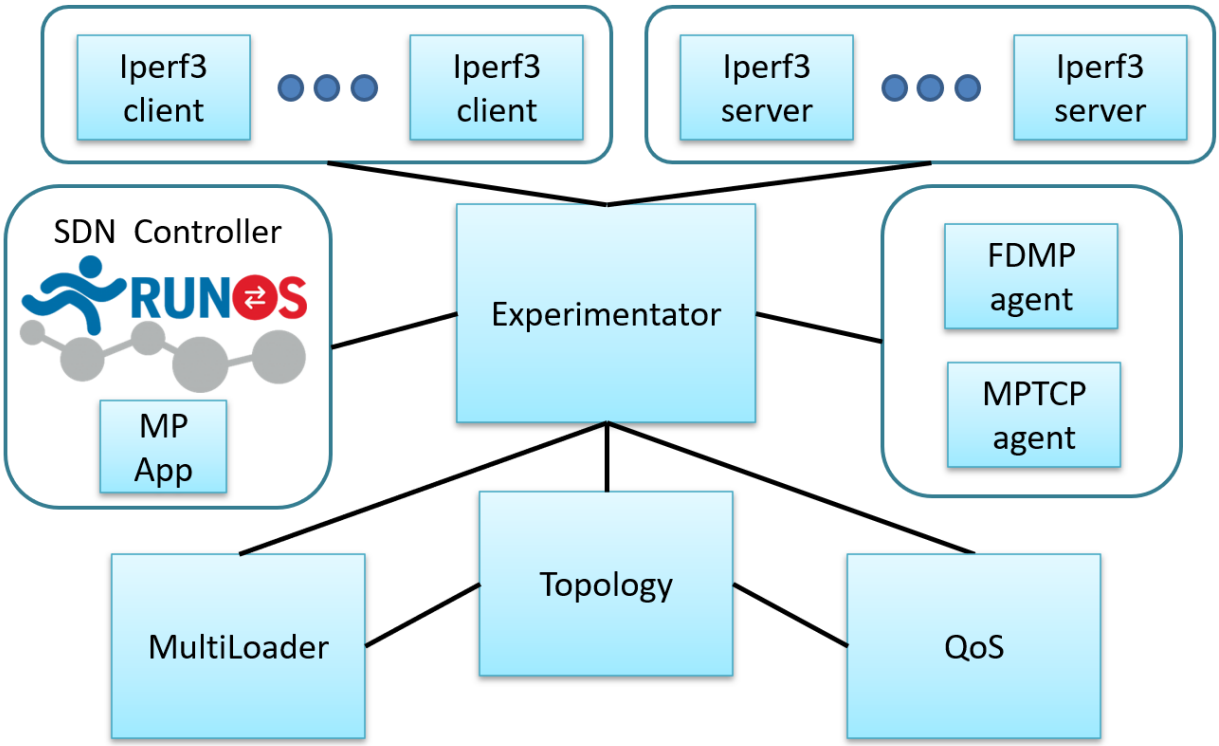

Рисунок 8. Архитектура программной части экспериментального стенда.

- Topology – подсистема, отвечающая за имитационное моделирование сети N. Подсистема Topology предоставляет интерфейс для других подсистем, который позволяет развернуть/очистить сеть, заданную параметром N, а также получить информацию о модели развернутой в настоящий момент сети, необходимую для последующей её настройки (например, для настройки качества сервиса линий связи).

- QoS – подсистема, отвечающая за настройку качества сервиса ρ линий связи имитационной модели сети N. Предоставляет интерфейс с соответствующими параметрами другим подсистемам.

- Multiloader – подсистема, отвечающая за настройку перенаправления потоков данных от генераторов трафика в имитационную модель сети и обратно согласно заданному распределению полюсов ϕ. Предоставляет

интерфейс с соответствующими параметрами другим подсистемам.

- Iperf3 – подсистема, отвечающая за генерацию нагрузки потоков данных F согласно заданному расписанию τ с заданными параметрами SLA ψ. Предоставляет интерфейс с соответствующими параметрами другим подсистемам.

- FDMP агент – подсистема, отвечающая за передачу потоков данных согласно протоколу FDMP, описанному в разделе 1.3. Подсистема предоставляет интерфейс для изменения максимального количества родственных подпотоков sub_{max}.

- MPTCP агент – подсистема, отвечающая за передачу потоков данных согласно протоколу MPTCP, описанному в разделе 1.2. Подсистема предоставляет интерфейс для изменения количества родственных подпотоков sub_{max}.

- MP App – подсистема, отвечающая за построение маршрутов $\{L_{ij}\}$ родственных подпотоков согласно смешанной стратегии, описанной в разделе 1.4, и установку соответствующих правил OpenFlow для этих маршрутов.

- Experimentator – подсистема, отвечающая за запуск экспериментов и сбор результатов. Интерфейс запуска эксперимента принимает в качестве входных параметров всю пятерку $< N, F, T, M, sub_{max} >$ и задействует все остальные подсистемы для проведения эксперимента. При этом проводится опрос статистики правил OpenFlow ПКС-

коммутаторов имитационной модели сети и опрос статистики подсистемы Iperf3 в моменты времени, определенные параметром T.

Разработанный стенд, детали реализации которого описаны в приложении Б, отвечает всем шести требованиям, перечисленным в начале раздела. Использование разных серверов для разных подсистем позволит исключить гонку за процессорное время для задач генерации потоков данных и имитационного моделирования сети Интернет-провайдера.

2.3 Область допустимых значений параметров эксперимента

Перед стартом серии экспериментов происходит генерация значений параметров $< N, F, T, M, sub_{max} >$ для каждого эксперимента. Считаем, что допустимые значения параметров имеют равномерное распределение. Область допустимых значений параметров была следующей:

a. **Топология** (G)**.** В качестве топологии используется одна из библиотеки Topology Zoo.
b. **Количество потоков** (F). В соответствии с требованиями российского Интернет-провайдера Ростелеком, количество потоков в сети может достигать 64000. Поэтому количество потоков в проводимых экспериментах изменялось от 16000 до 64000.
c. **Распределение полюсов** (ϕ). Количество полюсов, для которых проводилось исследование, изменялось от 10% до 100% от количества вершин в сети с шагом 10. Например, 100% означает, что у каждой вершины в топологии имеется свой полюс. Потоки распределяются равномерно между полюсами. Расположение полюсов в топологии генерируется случайным образом и каждый раз по-разному.
d. **Количество подпотоков** (sub_{max}). Основные значения, для которых проводилось исследование: 2, 3. Использовались данные значения, так как степень демультиплексирования для большинства топологий

Интернет-провайдеров не превосходит 3, как было показано в разделе 1.4.

e. **Расписание старта потоков** (τ). Согласно исследованию [66] время поступления пакетов в автономную систему Интернет-провайдера наилучшим образом описывается при помощи распределения Вэйбулла. Поэтому в экспериментальном исследовании время старта потока генерируется при помощи распределения Вэйбулла от 0 до некоторой константы, которая должна быть меньше времени окончания эксперимента.

f. **Классы потоков** (ψ). Для всех потоков одной пары виртуальных машин можно задать класс потока. Каждый класс определяет требуемое качество сервиса: объем данных и время, за которое эти данные должны быть переданы. Исходя из параметров класса потока, определяется требование к пропускной способности транспортного соединения. Классы потоков выбираются на основе классов видео (4K, 1440p, 1080p, 720p, 480p, 240p), представленных в источнике [67]. За основу взят видео трафик, так как согласно прогнозу Cisco [1] объем видео-трафика значительно возрастет в ближайшем будущем. Требование к времени передачи одного потока данных не превосходит одной минуты.

g. **Параметры среды передачи данных** (ρ). Пропускная способность каналов, при которой проводилось исследование - 1Гбит/с. Ограничение в 1 Гбит/с связано с ограничениями экспериментального стенда, где пропускная способность между серверами не может превышать 40 Гбит/с. Чтобы узкое место образовывалось не в линии связи между серверами, а в виртуальной сети, было выбрано ограничение в 1 Гбит/с. Дополнительно был проведена серия экспериментов, где пропускная способность каналов увеличивалась вместе с требованиями приложений (максимально в 10 раз), чтобы

проверить применимость полученных результатов в сетях с более быстрыми каналами передачи данных.

h. **Время окончания эксперимента** (T). 3 минуты. Так как время окончания потока не превосходит минуты, то за 3 минуты несколько раз смогут смениться транспортные потоки, проходящие через имитационную модель сети, что позволит лучше исследовать возможности динамического метода многопоточной маршрутизации.

При старте эксперимента подсистема Experimentator дает указание подсистеме Topology на построение имитационной модели сети, подсистеме QoS на настройку параметров качества линий связи, а подсистеме Multiloaer на создание граничного коммутатора и дополнительных линий связи в соответствии с заданным распределением полюсов. Запускается контроллер ПКС Runos вместе с приложением MP App, которое, согласно смешанной стратегии, устанавливало правила OpenFlow, реализующие передачу по построенным согласно алгоритму MCMF или GSPF маршрутам в топологии между полюсами. Количество правил сокращено при помощи группировки маршрутов, описанной в разделе 1.1.3. Подсистема Experimentator запускает генераторы трафика iperf3 согласно выбранному количеству потоков, расписанию потоков и требованием SLA потоков. Статистика собирается от iperf3 по каждому потоку и от граничного коммутатора по каждому подпотоку ежесекундно. После окончания времени эксперимента стенд подготавливается к следующему эксперименту.

2.4 Результаты экспериментального сравнения

Для объективного выбора оценки числа экспериментов, которые необходимо было провести, чтобы можно было бы сделать научно обоснованный вывод сравнительного анализа, был применен метод проверки статистических гипотез о числовом значении вероятности события, описанный в приложении А.

Вначале проверялась гипотеза о преимуществе динамического метода многопоточной маршрутизации над статическим методом. Выдвигалось предположение, что динамический метод не хуже статического с вероятностью не менее 95%. Для проверки этой гипотезы согласно методу, описанному в приложении А, и значения доверительного интервала $p_0 = 0.95$, требуется не менее 422 экспериментов. Всего было проведено 1790 экспериментов, что достаточно для обоснованного принятия решения о выполнимости гипотезы. Из 1790 проведенных экспериментов 1729 (96,59%) экспериментов показали число удовлетворенных потоков при динамическом методе FDMP больше или равно числу удовлетворенных потоков при статическом методе. Для принятия выдвинутой гипотезы с уровнем значимости $\alpha = 0.01$ необходимо, чтобы значение критерия $\varphi = \frac{(\hat{p} - p_0)}{\sqrt{\frac{p_0 * (1 - p_0)}{n}}}$ из приложения А было больше $x_{\text{лев},\alpha}^{\text{кр}} = -2.34$. Для проведенного экспериментального исследования $\varphi \approx 3.4$, поэтому гипотеза принимается.

Так как количество экспериментов достаточно велико, то подробные результаты каждого эксперимента о скорости потоков и занимаемых маршрутов заняли бы слишком много места в тексте. Поэтому далее будут приведены только агрегированные результаты по всему экспериментальному исследованию. Однако Стенд позволяет оценить эффективность методов многопоточной маршрутизации и для отдельных сетей Интернет-провайдеров. Например, для топологии NTT (рисунок 16), состоящей из 32 вершин и 63 ребер из 40 проведенных экспериментов для разного количества потоков и полюсов только 11 экспериментов показало одинаковую эффективность (т.е. оказалось равно количество потоков, для которых требования качества сервиса были удовлетворены), а для остальных 29 экспериментов динамический подход оказался лучше. Далее под термином успех будем называть поток, для которого транспортный протокол удовлетворил требования SLA потока. Зависимость количества успехов от параметров эксперимента для статического и

динамического методов демультиплексирования в случае топологии NTT показаны на рисунке 17.

Рисунок 16 Топология сети NTT (www.topolozy-zoo.org)

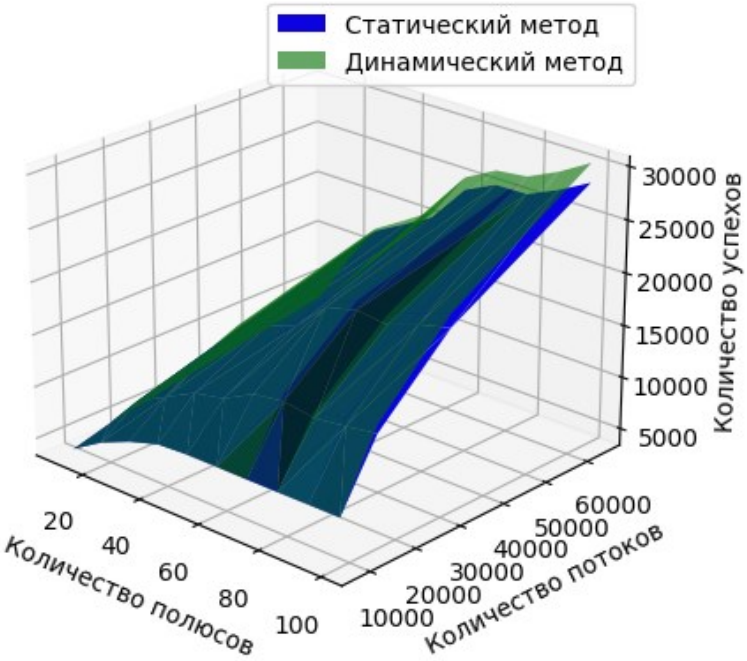

Рисунок 17 Зависимость количества успехов от параметров эксперимента для топологии NTT

Из 1729 случаев, где динамический метод был не хуже статического, в 289 случаях (16,7%) достигалось равенство по количеству успехов (рисунок 18). Причем стоимость потоков, определенная в пункте 2.1.2, в 102 случаях меньше у динамического метода, в 88 случаях меньше у статического метода, а в остальных случаях одинакова. Стоит отметить, что 80 случаев из 289 равенства количества успехов приходятся на топологии с коэффициентом демультиплексирования 1, т.е. с полным отсутствием альтернативных маршрутов. Большинство остальных случаев равенства приходится на топологии с небольшой степенью демультиплексирования и небольшой долей полюсов. Случайное распределение полюсов в топологии может привести к ситуации, когда отсутствуют альтернативные маршруты для топологий со степенью демультиплексирования меньшей 2.

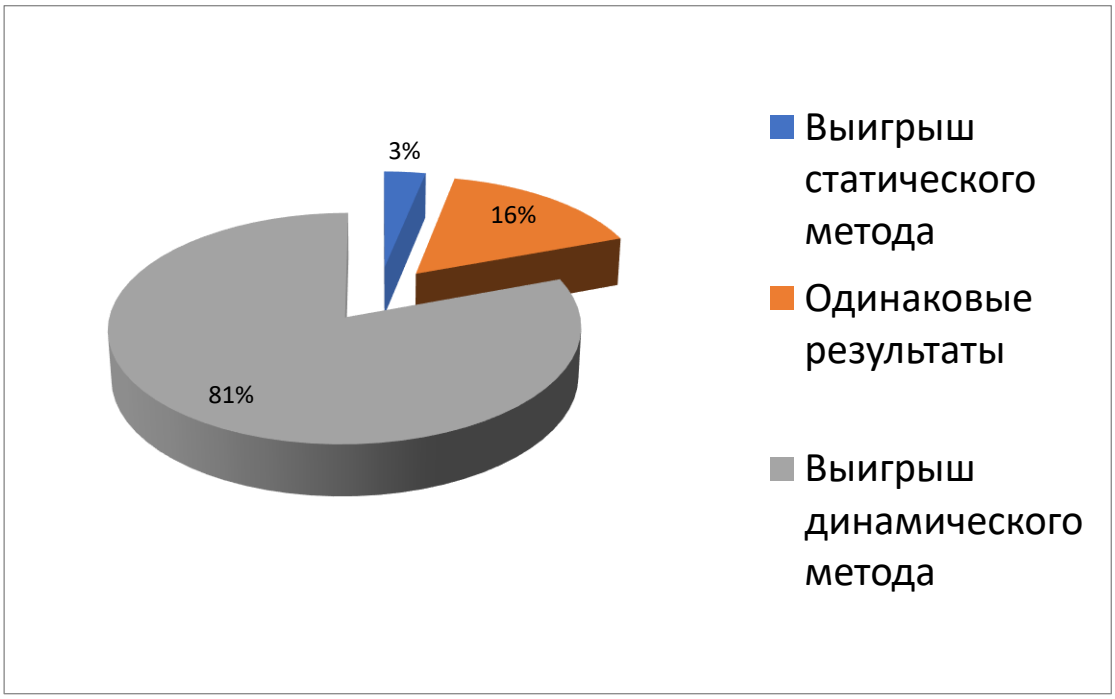

Рисунок 18 Сравнение эффективности статического и динамического методов многопоточной маршрутизации

Анализ экспериментов, в которых статический метод оказался лучше, показал, что основная область значений входных параметров, при которых статический метод имеет преимущество: топологии с количеством узлов меньше 37 и долей полюсов, меньше или равной 40% от количества узлов в топологии.

Для всех остальных случаев динамический метод будет работать не хуже, а в большинстве случаев лучше статического метода многопоточной маршрутизации.

Можно сделать предположение, что динамический метод больше обслуживает потоки из класса трафика с небольшим требованием к качеству сервиса, однако анализ результатов показал, что для каждого класса трафика в не менее чем 69% от числа экспериментов наблюдается прирост в числе удовлетворенных потоков при использовании динамического метода (рисунок 19).

Возникает вопрос, насколько приведенные результаты будут актуальны в будущем, когда возрастут пропускные способности каналов и требования приложений. Дополнительное экспериментальное исследование, в котором пропускные способности каналов и требования приложений были увеличены в одинаковое число раз, показало, что эффективность динамического метода многопоточной маршрутизации остается такой же.

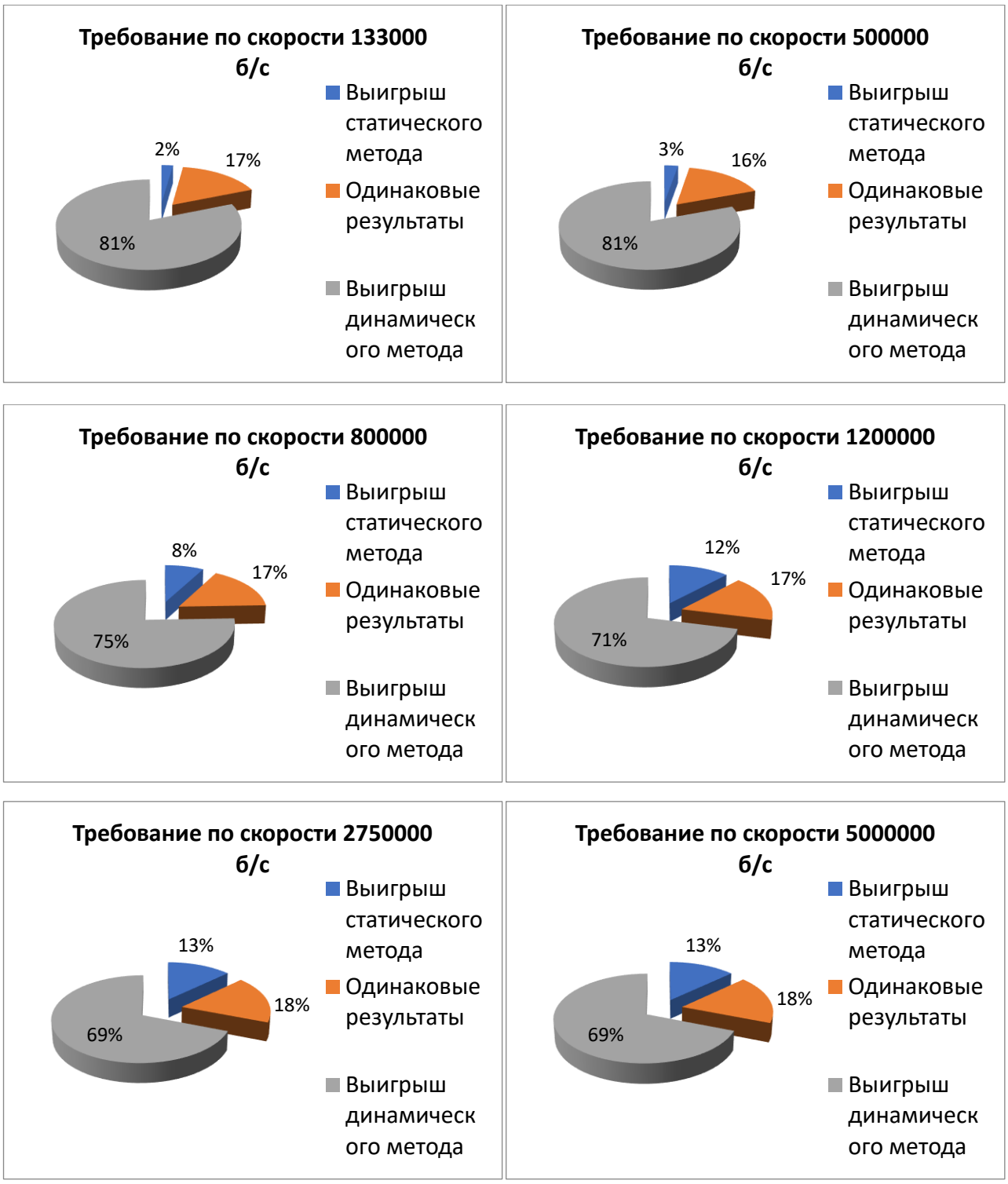

Рисунок 9 Сравнение эффективности методов многопоточной маршрутизации для разных требований по скорости

Таким образом, сравнительный анализ статического и динамического методов многопоточной маршрутизации показал преимущество динамического метода, когда количество удовлетворенных потоков не меньше, чем для статического метода, с вероятностью не менее 95%, при этом статический метод

хорошо работает только на небольших топологиях с количеством узлов меньше 36 и небольшой долей полюсов.

Глава 3. Применение многопоточной маршрутизации для реализации услуги «Пропускная способность по требованию»

В предыдущей главе приведены результаты сравнения эффективности методов многопоточной маршрутизации (МПМ), по которым определена область применимости для динамического и статического МПМ методов. В этой главе мы рассмотрим применение МПМ методов на примере реализации услуги «Пропускная способность по требованию» (ПСТ). Напомним, что услуга ПСТ состоит в предоставлении по требованию канала с заданной пропускной способностью между ЦОД. Так как ресурсов одного маршрута в сети может быть недостаточно для удовлетворения запроса услуги ПСТ, то предлагается использовать МПМ методы для передачи данных между ЦОД. Тогда Интернет-провайдер должен резервировать ресурсы сразу на нескольких маршрутах для предоставления одного канала услуги ПСТ. Естественно, возникает вопрос, а какому максимальному количеству клиентов можно оказать услугу ПСТ в заданной сети, если нам известна только вероятностная характеристика поступления запросов. Решение поставленного вопроса позволит оценить доступную емкость рынка и проанализировать, какие изменения необходимо внести в конфигурацию сети для увеличения количества потенциальных клиентов. Рассмотрим задачу определения максимального количества удовлетворенных пользователей услуги ПСТ более подробно. Пусть задана некоторая сеть – известна ее топология, пропускные способности каналов между узлами сети. Предположим, что фоновые потоки - так мы будем называть потоки данных сети, которые не относятся к услуге ПСТ - не меняются. Для оказания услуги заключается контракт – договоренность о передаче фиксированного объема данных между указанными точками отправления и назначения за время, не больше заданного. Для реализации каждого запроса по контракту резервируется необходимая пропускная способность в сети Интернет-провайдера. От объема оставшихся ресурсов сети будет зависеть общее количество контрактов, которые Интернет-провайдер может удовлетворить. Поэтому задача оценки максимального

количества удовлетворенных пользователей сводится к задаче оценки резерва пропускной способности, необходимой для реализации заданного набора контрактов. Такую задачу будем называть задачей ПСТ. Будем предполагать, что каждый контракт имеет свою функцию распределения времени поступления запросов, т.е. задана зависимость вероятности возникновения запроса по контракту от времени. При этом будем считать, что пока текущий запрос по некоторому контракту не будет выполнен, следующий запрос по этому же контракту возникнуть не может. Требуется определить, может ли сеть обслужить заданный набор контрактов при условии применения методов МПМ.

В разделе 3.1 рассмотрены разные варианты формальной постановки задачи нахождения резерва для реализации услуги ПСТ. Для постановки задачи, в которой информация о функции распределения времени поступления запросов включена во входные параметры, в разделе 3.2 приводится метод её решения с использованием методов решения задач линейного программирования. В разделе 3.3 приведены результаты экспериментальной апробации предложенного в разделе 3.2 метода.

3.1 Варианты постановок задачи ПСТ

В этом подразделе приведены разные постановки задачи ПСТ в зависимости от информации о времени поступления запросов по контрактам, информации о топологии сети и фоновых потоках. При этом решения для этих постановок задачи ПСТ позволят получить оценки резерва пропускной способности сети для разных случаев: в наихудшем случае поступления всех возможных запросов одновременно, в случае поступления запросов согласно заданному распределению и в случае, когда ресурсов сети может не хватить для обеспечения запросов по всем контрактам.

3.1.1 Постановка задачи ПСТ с одновременным приходом запросов.

В этой постановке задачи ПСТ будем считать, что время поступления запросов по всем контрактам одинаково. Это моделирует наихудшую ситуацию и

позволяет оценить максимальный резерв, который может потребоваться для удовлетворения всех контрактов.

Пусть заданы:

1. *Транспортная сеть* $N =< G, \rho >$ в метрическом пространстве, где
   - $G = (V, E)$ – связный ориентированный граф сети с множеством вершин V и множеством дуг E. С физической точки зрения все вершины представляют коммутаторы/маршрутизаторы сети, а дуги – линии связи.
   - ρ – мера на E

$$\forall \left(v_i, v_j\right) \in E: \quad \rho(v_i, v_j) \in \mathbb{R},$$

   где $\mathbb{R}$ - множество вещественных чисел. Содержательно ρ — это пропускная способность линии связи, измеряемая в бит/сек.

2. *Время наблюдения функционирования сети* N - $t \in [0, T]$ в дискретной форме с шагом ε. Тогда интервал времени $t \in [0, T]$ в дискретной форме будет задаваться как $t \in [t_0, \dots, t_M]$, где

$$t_m = \varepsilon * m, \text{ и } m \in [0, M]$$

3. Множество *фоновых потоков* $F = \{F_1(t), \dots, F_R(t)\}$ с интенсивностью $f_r(t)$ бит/сек каждый и маршрутом L_r $, r = \overline{1, R}$. Тогда ${q_{i,j}}^F(t) -$ суммарный фоновый поток через дугу (v_i, v_j)

$$ {q_{i,j}}^F(t) = \sum_{F_r \in F} \theta\left(\left(v_i, v_j\right), r\right), \text{ где}$$

$$\theta\left(\left(v_i, v_j\right), r\right) = \begin{cases} 0 & \text{если } \left(v_i, v_j\right) \notin L_r \\ f_r(t) & \text{если } \left(v_i, v_j\right) \in L_r \end{cases}$$

4. Множество *контрактов* $C = \{C_1, \dots, C_K\}$, где $k = \overline{1, K}$ и каждый C_k дает право передать по запросу Δ_k бит информации из вершины v_{a_k} в v_{b_k} за время, не превышающее τ_k.

   Тогда $t \in [0, \tau_k]$, отведенное для выполнения каждого из K контрактов по аналогии с п.2, будет иметь вид:

$$t_m = \varepsilon * m, \text{ где } m \in [0,\ M_k] \text{ и } t \in [t_0, \dots, t_{M_k}],$$

т. е. на выполнение -ого контракта будет отведено M_k интервалов. При этом, так как выражения для M_k и M имеют вид $M_k = \frac{\tau_k}{\varepsilon}$ и $M = \frac{T}{\varepsilon}$. Будем предполагать, что T и τ_k для каждого контракта из K кратны ε.

5. *Поток запросов* $\mu_k(t) \in \{0, 1\}$ на сервис контракта C_k. Причем, если $\mu_k(t') = 1$, то

$$\mu_k(t) = 0 \text{ при } t \in (t', t' + \tau_k),$$

т. е. запросы могут поступать многократно за время действия контракта. Однако новый запрос по контракту может быть инициирован только после того, как будет завершен предыдущий. В этой постановке задачи будем считать, что $\forall\, k \in [1, K]$: $\mu_k(0) = 1$, т. е. запросы по всем контрактам поступили одновременно в момент времени $t_0 = 0$.

Обозначим через $q_{i,j}{}^{C_k}(t)$ *скорость передачи данных по контракту* C_k *по дуге* $(v_i, v_j) \in E$ в момент времени t. Тогда требуется для заданных параметров 1–5 оценить возможность реализации потоков запросов на заданное множество контрактов в заданной сети в случае $\forall\, k \in [1, K]$: $\mu_k(0) = 1$, т.е. найти если это возможно такое множество $\{q_{i,j}{}^{C_k}(t)\}$, которое позволит удовлетворить запросы по каждому из контрактов и при этом не нарушить ограничение на максимальную пропускную способность линий связи. Если при этом мы будем минимизировать еще и сумму скоростей передачи данных по всем контрактам, по всем дугам и по всем моментам времени t, то мы уменьшим резерв, необходимый для реализации заданного набора контрактов, что может позволить обслужить большее количество контрактов. Решение данной задачи эквивалентно минимизации следующей целевой функции $(*_{\text{ц}_1})$ при условии выполнения ограничений $(*_{\text{огр}_1})$, указанных ниже

$$\min_{M,K,E} \sum_{m=0}^{M} \sum_{k=1}^{K} \sum_{j:(v_i,v_j)\in E} q_{i,j}^{C_k}(t_m) \qquad (*_{\text{ц}_1})$$

$$
(*_{\text{огр}_1})\begin{cases}
\sum_{k=1}^{K} q_{i,j}^{C_k}(t_m) \le \rho_{i,j} - q_{i,j}^{F}(t_m), \qquad (v_i, v_j) \in E, m \in \overline{0,M} \ (1) \\
\sum_{m=0}^{M_k} \sum_{i:(v_{a_k}, v_i) \in E} q_{a_k,i}^{C_k}(t_m) * \epsilon = \Delta_k \qquad k = \overline{1,K} \ (2) \\
\sum_{m=0}^{M_k} \sum_{i:(v_i, v_{a_k}) \in E} q_{i,a_k}^{C_k}(t_m) * \epsilon = 0 \qquad k = \overline{1,K} \ (3) \\
\sum_{m=0}^{M_k} \sum_{i:(v_i, v_{b_k}) \in E} q_{a_k,i}^{C_k}(t_m) * \epsilon = \Delta_k \qquad k = \overline{1,K} \ (4) \\
\sum_{m=0}^{M_k} \sum_{i:(v_{b_k} v_i) \in E} q_{a_k,i}^{C_k}(t_m) * \epsilon = 0 \qquad k = \overline{1,K} \ (5) \\
\sum_{i:(v_i, v_r) \in E} q_{i,r}^{C_k}(t_m) - \sum_{j:(v_r, v_j) \in E} q_{r,j}^{C_k}(t_m) = 0 \quad v_r \in V \backslash \{v_{a_k}, v_{b_k}\}, k = \overline{1,K}, m = \overline{0,M} \ (6) \\
q_{i,j}^{C_k}(t_m) - q_{i,j}^{C_k}(t_{m+1}) = 0 \quad m = \overline{0, M_k - 1}, k = \overline{1,K}, (v_i, v_j) \in E \ (7) \\
q_{i,j}^{C_k}(t_m) \ge 0 \qquad (v_i, v_j) \in E, k = \overline{1,K}, m = \overline{0,M} \ (8)
\end{cases}
$$

Опишем ограничения ($*_{\text{огр}_1}$) более подробно:

(1) Суммарный поток по всем контрактам через любую дугу не должен превосходить остаточную пропускную способность этой дуги.

(2, 4) За время выполнения контракта C_k из узла v_{a_k} должно быть передано, а узлом v_{b_k} принято объем данных Δ_k.

(3, 5) Узел отправителя v_{a_k} контракта C_k не может принимать данные, которые передаются согласно этому контракту, а узел получателя v_{b_k} не может отправлять эти данные.

(6) Баланс по каждому из контрактов $C_k, k \in [1, K]$ по объему принятой и переданной информации для всех узлов сети, исключая v_{a_k} и v_{b_k}, в момент времени t_m.

(7) Условие, согласно которому скорость передачи данных по линии связи между любыми двумя узлами v_i и v_j по контракту C_k будет одинаковой на протяжении всего интервала времени выполнения этого контракта.

(8) Условие не отрицательности каждой из переменных $q_{i,j}^{C_k}(t_m)$ обусловлено физическим смыслом, т. к. скорость передачи данных бит/сек не может быть отрицательной.

3.1.2 Постановка задачи ПСТ при случайном потоке запросов с заданным распределением

В этом разделе рассмотрим постановку задачи ПСТ, когда нам известно распределение времени поступлений запросов по каждому из контрактов.

Пусть заданы:

1. *Транспортная сеть* $\mathcal{N} = (G, \rho)$.

2. *Интервал времени* $t \in [0, T]$.

3. Множество *фоновых потоков* $F = \{F_1(t), \dots, F_R(t)\}$ с интенсивностью $f_r(t)$ бит/сек каждый и маршрутом L_r , $r = \overline{1, R}$.

4. Множество *контрактов* $C = \{C_1, \dots, C_K\}$, где $k = \overline{1, K}$.

5. *Поток запросов* $\mu_k(t) \in \{0, 1\}$ на сервис контракта C_k. Причем если $\mu_k(t') = 1$, то

$$\mu_k(t) = 0 \text{ при } t \in (t', t' + \tau_k),$$

т. е. запросы могут поступать многократно за время действия контракта. Однако новый запрос по контракту может быть инициирован только после того, как будет завершен предыдущий. Будем считать, что известны функция распределения времени поступления запроса, и следовательно, так как известно требуемое время τ_k передачи данных по запросу, функция распределения времени завершения потока передачи соответствующего запроса. Так как

работать с вероятностными характеристиками достаточно сложно, будем предполагать, что дана выборка значений времени поступления запросов и соответствующих значений времени окончания передачи данных по запросу. Как будет объяснено в следующем разделе, анализируя решение задачи ПСТ в этой постановке для множественных выборок, можно будет сделать вывод о необходимом резерве пропускной способности для заданной функции распределения поступления запросов.

6. В этой постановке задачи интервал времени $[0, T]$ разбивается не на равные интервалы времени длины ε, а согласно заданной выборке значений времени поступления запроса и выборке времени окончания передачи данных по запросу. Рассмотрим множество точек, задающих время старта и время окончания запросов по контрактам и разбивающих непрерывный отрезок времени $[0, T]$ на дискретные интервалы. Обозначим его за D. Тогда более формально можно записать, что $t_m \in D, m = \overline{0, M}$ если $\exists\, k$:

$$\left[\begin{array}{l} \mu_k(t_m) = 1 \\ t_m = t_k + \tau_k, \quad \text{где} \;\; \mu_k(t_k) = 1 \end{array}\right.$$

Будем считать, что скорость передачи данных по каждому из контрактов на интервале $[t_m, t_{m+1}], m = \overline{0, M-1}$ не меняется и равна скорости передачи в момент времени t_m. За ε_m обозначим длительность интервала $[t_m, t_{m+1}], m = \overline{0, M-1}$, за C_{t_m} - набор выполняемых на данном интервале контрактов.

Обозначим через ${q_{i,j}}^{C_k}(t)$ *скорость передачи данных по контракту* C_k *по дуге* $(v_i, v_j) \in E$ в моменты времени t_m такие, что $C_k \in C_{t_m}$.

Требуется для заданных параметров 1-6 оценить возможность реализации потоков запросов $\mu_k(t_m)$ на заданное множество контрактов в заданной сети, т.е. найти если это возможно такое множество $\{{q_{i,j}}^{C_k}(t)\}$, которое позволит удовлетворить запросы по каждому из контрактов и при этом не нарушить ограничение на максимальную пропускную способность линий связи.

$$\min_{K,E} \sum_{k=1}^{K} \sum_{i,j:(v_i,v_j)\in E} q_{i,j}^{C_k} \quad (*_{\text{ц}_2})$$

$$(*_{\text{огр}_2}) \begin{cases} \sum_{k=1}^{K} q_{i,j}^{C_k}(t_m) \le \rho_{i,j} - q_{i,j}^{F}(t_m), \qquad (v_i, v_j) \in E, m \in \overline{0,M} \ (1) \\ \sum_{m=0}^{M_k} \sum_{i:(v_{a_k},v_i)\in E} q_{a_k,i}^{C_k}(t_m) * \epsilon_m = \Delta_k \qquad C_k \in C_{t_m} \ (2) \\ \sum_{m=0}^{M_k} \sum_{i:(v_i,v_{a_k})\in E} q_{i,a_k}^{C_k}(t_m) * \epsilon_m = 0 \qquad C_k \in C_{t_m} \ (3) \\ \sum_{m=0}^{M_k} \sum_{i:(v_i,v_{b_k})\in E} q_{a_k,i}^{C_k}(t_m) * \epsilon_m = \Delta_k \qquad C_k \in C_{t_m} \ (4) \\ \sum_{m=0}^{M_k} \sum_{i:(v_{b_k}v_i)\in E} q_{a_k,i}^{C_k}(t_m) * \epsilon_m = 0 \qquad C_k \in C_{t_m} \ (5) \\ \sum_{i:(v_i,v_r)\in E} q_{i,r}^{C_k}(t_m) - \sum_{j:(v_r,v_j)\in E} q_{r,j}^{C_k}(t_m) = 0 \quad v_r \in V\backslash\{v_{a_k}, v_{b_k}\}, C_k \in C_{t_m}, m = \overline{0,M} \ (6) \\ q_{i,j}^{C_k}(t_m) \ge 0 \qquad (v_i, v_j) \in E, C_k \in C_{t_m}, m = \overline{0,M} \ (7) \end{cases}$$

Решение задачи $(*_2)$ позволит ответить на следующие вопросы:

1) Способна ли сеть $\mathcal{N}$ с фоновыми потоками F обеспечить выполнение множества запросов по контрактам C?
2) Какой объем данных, при положительном ответе на вопрос 1), должен быть передан по каждому из контрактов $C = \{C_1, \dots, C_K\}$ для каждой дуги сети $(v_i, v_j) \in E$, при фиксированном потоке запросов $\mu_k(t)$?

3.1.3 Постановка задачи ПСТ при случайном потоке запросов с заданным распределением с определением недостающих сетевых ресурсов.

При решении задач, поставленных в разделах 3.1.1 и 3.1.2, может сложиться ситуация, когда ресурсов сети не хватает, чтобы удовлетворить заданный поток запросов. В этом случае у поставщика услуги ПСТ может возникнуть вопрос: каким образом можно расширить пропускную способность линий связи или

проложить новые линии связи, чтобы все запросы были удовлетворены, а добавленные ресурсы были минимальны. Для ответа на поставленный вопрос можно внести небольшие изменения в постановку задачи $(*_2)$, который позволит оценить объем нехватающих сетевых ресурсов. Для этого воспользуемся тем же подходом, что используется в симплекс методе [68] для решения задач линейного программирования. Когда необходимо найти любое существующее решение для заданной задачи линейного программирования, в симплекс методе вводятся дополнительные переменные, которые в ходе работы метода обращаются в ноль, если решение существует. Если решения не существует, то переменные примут в сумме минимально возможное значение, так как они добавляются в целевую функцию с большим штрафным коэффициентом.

Для поиска недостающих ресурсов можно поступить таким же образом: предположим, что на каждой линии связи (при этом можем даже рассматривать дополнительные линии связи, которых нету в исходном графе сети) существует дополнительная неотрицательная пропускная способность $\rho^*_{i,j}$. Тогда в системе $(*_2)$ условие (1) перепишется как:

$$\sum_{k:C_k \in C_{t_m}} q_{i,j}^{C_k} \leq \rho_{i,j} + \rho^*_{i,j} - q_{i,j}^F$$

Необходимо добавить условие (8) о не отрицательности пропускной способности:

$$\rho^*_{i,j} \geq 0$$

Наконец, в целевую функцию необходимо добавить дополнительные переменные со штрафом P:

$$\min_{K,E}\left(\sum_{k=1}^{K} \sum_{i,j:(v_i,v_j)\in E} q_{i,j}^{C_k} + P * \rho^*_{i,j}\right)$$

3.2 Метод решения задачи ПСТ

Рассмотрим метод решения задачи ПСТ на примере постановки задачи, описанной в разделе 3.1.2. Для других постановок задач, представленных в разделе 3.1, решение будет аналогичное, но для другой системы неравенств. Описанная в разделе 3.1.2 задача является задачей целочисленного линейного программирования (ЦЛП) [69], в дальнейшем обозначаемая как $(*_2)$. Это так, поскольку предложенная ее формулировка, полностью соответствует определению общей задачи ЦЛП [69], в которой необходимо найти:

$$max \sum_{n=1}^{N} c_n x_n$$

или

$$min \sum_{n=1}^{N} c_n x_n$$

при заданных ограничениях

$$\begin{cases} a_{11}x_1 + .. + a_{1n}x_n \leq (\geq, =) b_1 \\ a_{21}x_1 + .. + a_{2n}x_n \leq (\geq, =) b_2 \\ \dots \\ a_{m1}x_1 + .. + a_{mn}x_n \leq (\geq, =) b_m \\ x_1 \geq 0, x_2 \geq 0, \dots, x_n \geq 0 \end{cases}$$

и условии, что $x_1, x_2, \dots, x_n \in \mathbb{Z}$ - множеству целых чисел.

В нашей работе в качестве неизвестных переменных x_n выступают переменные $q_{i,j}{}^{C_k}$, в качестве ограничений – система ($*_{\text{огр}_2}$), в качестве целевой функции – выражение $(*_{\text{ц}_2})$. Целочисленность неизвестных $q_{i,j}{}^{C_k}$ обусловлена их физическим смыслом – это объем данных, передаваемых за секунду, измеряемый в бит/сек, где бит – это неделимая единица измерения.

Отметим также, что рассматривать задачу $(*_2)$ как задачу ЛП [70, 71](она задается как ЦЛП, но не имеет ограничений на целостность неизвестных) и округлять полученное решение до целого значения - не корректный подход. Подробнее об этом, написано в книге Мину [72].

Далее рассмотрим, каким образом можно решать поставленную задачу как задачу ЦЛП. Для этого необходимо ответить на вопросы:

- какой язык представления задачи ЦЛП использовать для описания задачи ПСТ
- какое программное средство использовать для решения задачи ЦЛП

Задача $(*_2)$ содержит большое число неизвестных, поэтому при выборе языка важно удобство его математического представления, а именно, возможность производить запись выражения с использованием операции суммирования по индексам. Для представления задачи ЦЛП был выбран язык Python, потому что этот язык является высокоуровневым языком программирования с наличием библиотек, поддерживающих работу с различными функциями распределения, а также удобными средствами описания задачи и средствами решения задач ЦЛП, которые реализованы на этом же языке.

Перейдем к разбору программных средств, способных решить нашу задачу $\left(*_{\text{огр}_2}\right)$ целочисленного линейного программирования. Прежде чем разбирать конкретные продукты, доступные в данный момент на рынке, проясним три понятия, зачастую встречающиеся в их описании: библиотеки, решателя и пакета прикладных программ.

Библиотека – набор подпрограмм, используемых для разработки программного обеспечения, оснащенный системной частью, обеспечивающей их доступ, хранение и модификацию.

Пакет прикладных программ — набор взаимосвязанных модулей, предназначенных для решения задач определённого класса некоторой предметной области. Он отличается от библиотеки тем, что создание библиотеки не ставит целью полностью покрыть нужды предметной области, так как приложение или разрабатываемое ПО может использовать модули нескольких библиотек. Требования же к пакету программ жёстче: приложение для решения задачи должно использовать только модули пакета, а создание конкретного приложения может быть доступно непрограммистам.

Решатель (solver) – это математическое программное обеспечение, представленное, возможно, в виде отдельной компьютерной программы или библиотеки ПО, которое надстраивается над пакетом прикладных программ. Решатель принимает описание проблемы в некоторой общей форме, проводит ее анализ, на основании которого выбирает необходимые для ее решения модули из пакета. После чего макропроцессором формируется полноценная программа, предназначенная для решения нашей задачи.

Таким образом, чтобы решить некоторую задачу оптимизации, пользователю необходимо выбрать пакет прикладных программ, содержащий необходимые модули для решения данной задачи и решатель, над ним надстроенный. Так как понятия пакета и решателя тесно связаны – не все пакеты могут быть использованы одним и тем же решателем – будем рассматривать их в совокупности, т. е. выберем пару решатель-пакет, для решения задачи ($*_2$).

ППП (пакеты прикладных программ)	Решатели (солверы)
AMPL	COIN-OR CBC
AMPL	BARON
AMPL	CPLEX
AMPL	GUROBI
AMPL	FICO Xpress
AMPL	MINOS
GAMS	CPLEX
GAMS	GUROBI
GAMS	MOSEK
GAMS	FICO Xpress
MIP Python	COIN-OR CBC
MIP Python	GUROBI
Pyomo	COIN-OR CBC
Pyomo	GLPK

Pyomo	CPLEX
Pyomo	GUROBI

Таблица 2. Пакеты прикладных программ и использующие их решатели (зеленым цветом отмечены продукты свободно распространяемые, красным – имеющие коммерческую составляющую)

В таблице 2 приведено большинство известных на сегодняшний день пакетов прикладных программ и решателей с делением по политике их использования (красным цветом обозначены коммерческие продукты, зеленым – находящиеся в свободном доступе). Если некоторый пакет и решатель находят в одной строчке в таблице, значит данный решатель может быть надстроен над соответствующим пакетом.

Так как, как уже отмечалось ранее, в научном исследовании важно обеспечить воспроизводимость экспериментов, что подразумевает использование общедоступными программными средствами.

Поэтому рассмотрим далее только свободно распространяемые средства. Из таблицы 3, видно, что к ним относятся только лишь пары: MIP Python - COIN-OR CBC, Pyomo - COIN-OR CBC, Pyomo – GLPK. Краткое описание пакетов MIP Python, Pyomo и использующих их решателей COIN-OR CBC, GLPK приведено в приложении В.

Сравним эти средства по следующим параметрам:

1. Возможность записывать задачу ЦЛП при помощи языков C, C++ или Python;
2. Поддержка нахождения решения при помощи распараллеливания вычислений для увеличения скорости нахождения решения;
3. Учитывание высокой размерности задач;

Рассмотрев основные моменты по каждому из программных средств, можно прийти к выводу (таблица 3), что для решения нашей задачи подходят все три варианта: пакет Pyomo в сочетании с решателями COIN-OR CBC и GLPK и пакет

MIP Python в сочетании с COIN-OR CBC. Сравнение пар пакет – решатель, для решения задач ЦЛП

	Языки C/C++, Python	Параллельные вычисления	Учет высокой размерности задачи
MIP Python - COIN-OR CBC	Python	+	+
Pyomo - COIN-OR CBC	Python	+	+
Pyomo – GLPK	Python	+	+

Таблица 3. Сравнение пар пакет – решатель, для решения задач ЦЛП

Оценка скорости работы выбранных средств

Для того выбрать только один пакет, было проведено несколько экспериментов, оценивающих время нахождения решения каждым из них.

Первый эксперимент, представленный на рисунке 20, отражает зависимость времени нахождения решения от количества узлов в топологии. Резкое уменьшение времени выполнения для обоих пакетов в конце отрезка 180-200 узлов связано с тем, что для топологии с большим числом узлов остаточная пропускная способность была больше, по сравнению с топологией с меньшим числом узлов. В сочетании с различным числом контрактов для обоих топологий и различной загруженности фоновыми потоками и возник такой эффект. Отметим, что фиксировать некоторый набор контрактов было бы неправильно, так как естественно, что для нахождения решения для топологии из 80 узлов потребуется больше времени, чем для топологии из 200 узлов при одинаковом соотношении суммарной остаточной пропускной способности сети к ее общей пропускной способности. Это произойдет потому, как тот же объем данных можно будет передать большим количеством путей.

Что касается производительности обоих пакетов, то мы можем отметить, что при количестве узлов сети менее 50, пакет Pyomo находит решение быстрее, чем пакета MIP. Однако, когда их количество становится больше 50, быстрее находит решение пакет MIP. Отметим, что оба пакета выполняли поиск решения для одних и тех же топологий с идентичными фоновыми потоками и наборами контрактов.

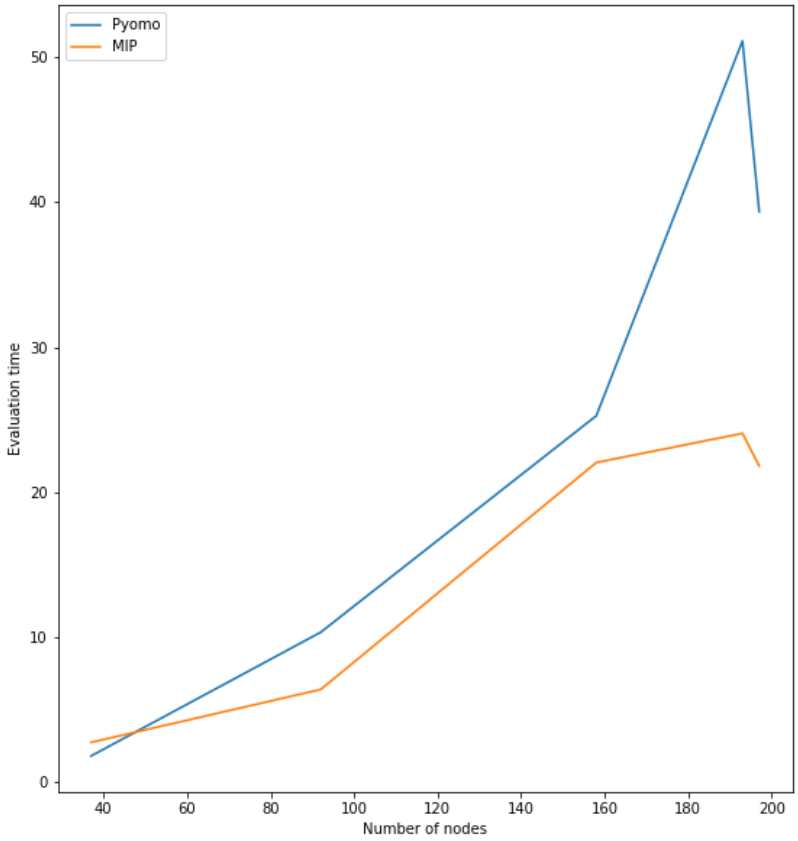

Рисунок 20. Зависимость времени нахождения решения от количества узлов сети.

Целью второго эксперимента (рисунок 21) было оценить время нахождения решения в зависимости от количества контрактов. Точно так же, как и в предыдущем эксперименте, оба пакета решали одинаковые системы уравнений - для топологии Bren фиксировались фоновые потоки и наборы контрактов. Как и в предыдущем эксперименте, пакет MIP требует больше времени для случая числа контрактов меньше 4, но для множеств, содержащих 4 и более контрактов, опережал по скорости Pyomo.

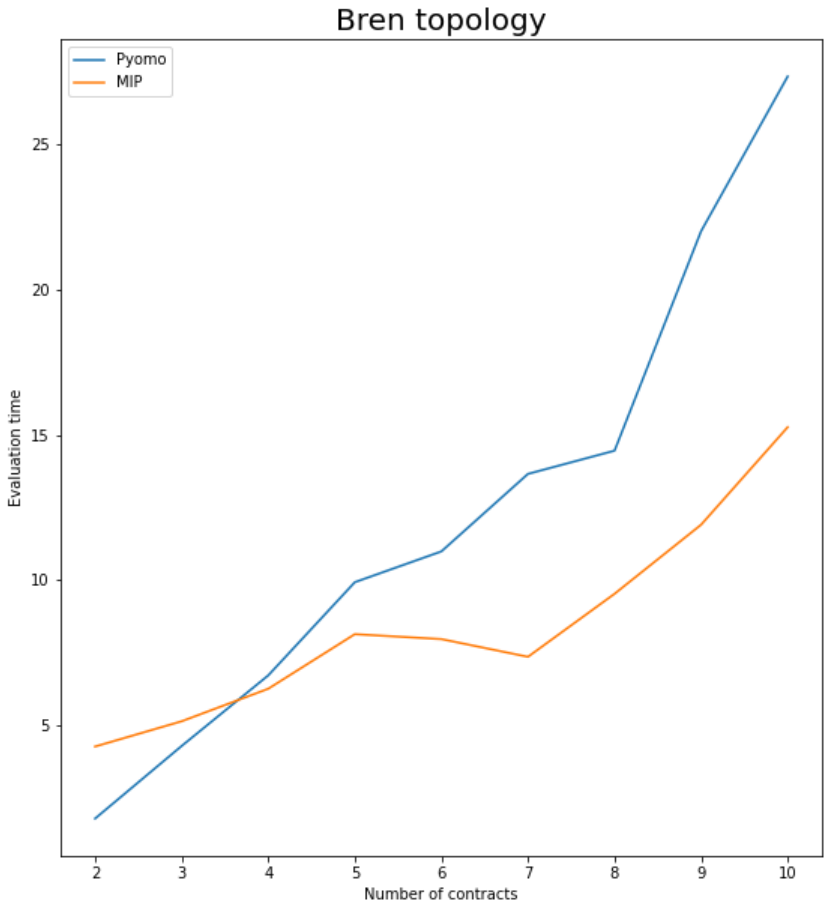

Рисунок 21. Зависимость времени нахождения решения от количества контрактов

Основываясь на полученных результатах в качестве пакета для решения системы $(*_2)$ решено использовать пакет MIP, так как он быстрее находит решение нежели пакет Pyomo для ЦЛП задач с большим набором неизвестных.

3.3 Экспериментальное исследование предложенного подхода

В данном разделе опишем характеристики, которыми обладают входные параметры системы, а именно топологию, фоновые потоки, наборы контрактов.

3.3.1 Методика проведения экспериментального исследования

В эксперименте входными данными являются следующие параметры:

1. Файл с топологией Bren и заданными для нее пропускными способностями и фоновыми потоками.

2. Фиксированный набор из 30 контрактов, где для каждого контракта указан объем передаваемых данных, узлы отправителя, получателя и время передачи.

На языке Python, выбранном в качестве языка описания задачи ЦЛП, была описана система $(*_2)$, а при помощи пакета прикладных программ Python-MIP была создана программная реализация ее решающая.

Таким образом, на начало проведения экспериментов мы имели систему, принимающую на вход фиксированную топологию с фиксированными фоновыми потоками и конкретным набором контрактов. Далее на основании полученных данных эта система формировала задачу $(*_2)$, решала ее при помощи пакета MIP и сообщала пользователю ответ: имеет ли задача $(*_2)$ решение при заданных входных параметрах и если «да», то на каких соединениях стоит оставить резерв для каждого из контрактов. В данной системе все параметры были заданы заранее кроме времен старта контрактов. Перед тем как сформировать задачу $(*_2)$ выбиралось некоторое число X – количество моментов наступления контрактов, что фактически является суммарным количеством всех контрактов, которые должны выполниться за все время наблюдения $[t_0, T]$, с учетом того, что некоторые из этих контрактов могут повторяться. Далее из равномерного распределения $Uniform(t_0, T)$ выбиралось некоторое время старта контракта t_s, после чего между всему контрактами разыгрывалось право получить запрос в этот момент времени. Розыгрыш проводился также согласно равномерному распределению, т.е. необходимо было производить выбор некоторого контракта $C_k \sim Uniform(1, K)$ до тех пор, пока не будет найден контракт с индексом k, не выполняющийся в момент времени t_s. Аналогичная процедура выбора проводилась X раз в результате чего формировались все запросы на контракты.

Предположим, что мы хотим узнать, какое количество запросов на эти контракты мы можем выполнить с вероятностью p_0 = 0.99. Пусть мы провели 10 экспериментов для параметра X = 20 и для каждого из этих 10 экспериментов система $(*_2)$ при сгенерированных t_s имела решение. Можем ли мы утверждать,

что данное число запросов всегда можно будет удовлетворить с выбранной вероятностью p_0? Для того, чтобы оценить какое количество экспериментов нужно провести чтобы это утверждение имело научное подтверждение, воспользуемся методом проверки статистических гипотез о числовом значении вероятности события, описанным в приложении А.

Оценка необходимого числа экспериментов

Обозначим нашу систему за $\Pi(t_1, \dots, t_X, k_1, \dots, k_X)$, где t_i, $i \in [1, X]$ время старта i-ого контракта, а k_i, $i \in [1, X]$ – его индекс. Результат работы программы обозначим за ξ, где

$$\xi = \begin{cases} 1 & \text{система при входных параметрах } (t_1, \dots, t_X, k_1, \dots, k_X) \text{ имеет решение} \\ 0 & \text{система при входных параметрах } (t_1, \dots, t_X, k_1, \dots, k_X) \text{ не имеет решения} \end{cases}$$

Предположим, что мы выполнили n запусков системы $\Pi(t_1, \dots, t_X, k_1, \dots, k_X)$ с различными параметрами $t_i, k_i, i \in [1, X]$. Генерацию входных параметров и запуски системы будем проводить независимо друг от друга. Тогда можно считать, что ξ есть некоторая случайная величина, принимающая значения [0,1] и имеющая свою, не известную нам, функцию распределения. Сформулируем и проверим следующую гипотезу «с вероятностью p_0 случайная величина ξ принимает значение 1». Будем обозначать выдвинутую гипотезу за H_0, а событие, при котором $\xi = 1$, за F. Тогда воспользовавшись методом, описанным в приложении А, для проверки статистической гипотезы, мы сможем найти число экспериментов n и число найденных решений m (т.е. количество ξ_j: $\xi_j = 1$ где $j \in [1, n]$) при которых выдвинутая гипотеза H_0 будет верна с заданным уровнем значимости.

Будем рассматривать гипотезу H_0 следующего вида «с вероятностью p_0 случайная величина ξ принимает значение 1». В таблице 4 приведены результаты вычислений для различных значений вероятности p_0 и уровня значимости α.

$p_0 \backslash \alpha$	0.1		0.05		0.01		0.005	
	m	n	m	n	m	n	m	n
0.99	1996	2021	1994	2021	1991	2021	1990	2021
0.98	995	1021	994	1021	991	1021	990	1021
0.97	662	688	661	688	657	688	656	688
0.96	495	521	493	521	490	521	489	521

Таблица 4: Общее количество экспериментов (n) и количество удачных исходов (m) в зависимости от вероятности p_0 и уровня значимости α

Опишем процесс получения этих значений на конкретном примере, опираясь на приложение А. Выберем вероятность $p_0 = 0.99$ уровень значимости α равным 0.05. Тогда, $\gamma = 1 - 2\alpha = 0.9$, $u_\gamma = 1.64$ и $x^{\text{кр}}_{\text{лев},\alpha} = -1.64$. Воспользовавшись оценкой, указанной в работе [85], получаем, что для $p_0 = 0.99$ необходимо брать n равным 2021. Тогда если в серии экспериментов число удачных исходов m (при которых найдено решение $(*_2)$) будет таковым, что $\varphi = \left(\frac{m}{2021} - 0.99\right) * 451.82 > -1.64$, то можно считать, что гипотеза $H_{02}: P(\xi = 1) \geq p_0$ верна и вероятность того, что система $(*_2)$ имеет решение больше или равна 0.99. Решив указанное выше неравенства для критерия получаем, что число удачных экспериментов m должно быть больше или равно 1994 из 2021 запусков (т.е. при $n = 2021, m \geq 1994$).

Топология

В данной работе использовалась топология Bren из Интернет-ресурса TopologyZoo [52]. Так как все топологии, даже одного уровня, имеют различные форматы и наборы полей, было принято решение использовать только графы этих топологий, а пропускные способности генерировать при помощи генератора случайных чисел взятых из равномерного распределения так, чтобы пропускная способность равнялась 100 Мбит/сек, 200 Мбит/сек или 300 Мбит/сек. Обработка

топологий производилась при помощи пакета NetworkX [73], используемый формат представления - “graphml”.

Фоновые потоки

Значениями фоновых потоков выбирались произвольные случайные числа из равномерного распределения в интервале от 0 до величины пропускной способности. Для каждого соединения они сохранялись в качестве отдельного поля, после чего вся топология целиком сохранялась и представляла собой уже топологию с фиксированными пропускными способностями и фоновыми потоками.

Набор контрактов

Одним из вопросов при составлении методики проведения экспериментального исследования является определение функции/функций распределения поступления потока запросов по контрактам. Выбор можно обосновывать на основании существующих моделей трафика. Моделирование трафика изучается уже более полувека, эта тема освещалась как в работах далекого 69 года [74] так и сейчас [75]. С течением времени представление людей о выборе наилучшего математического средства для моделирования трафика менялось в зависимости от конкретного вида трафика (телефонный трафик или сеть Интернет), вида сети (сеть доступа или магистральная), длительности интервала наблюдения, загруженности сети и уровня моделирования OSI (т.е. моделирование движения пакетов, установления потоков или сессий). Долгое время одним из самых широко используемых распределений для моделирования трафика сети Интернет оставалось распределение Пуассона [76], с конца 90-ых ему на смену пришло распределение Вейбулла [66].

В качестве распределения для моделирования запросов на контракты изначально предполагалось брать распределение Вейбулла, как наиболее подходящее [66] для моделирования Интернет-трафика. Однако позже, было замечено некоторое сходство запросов на контракты с телефонными звонками: аналогично телефонным звонкам, поступление запроса на контракт подразумевает

невозможность поступления нового запроса на тот же контракт, до его окончания. В таком случае, в качестве распределения для моделирования телефонных звонков обычно берут Сложное распределение Пуассона [77, 78].

Однако проблема заключается в том, что на данный момент нет никаких реальных данных о поступлении запросов на контракты, так что выбор любого из описанных распределений (Вейбулла или Сложное распределение Пуассона) нельзя подтвердить ничем кроме теоретических предположений. В случае отсутствия информации о конкретных данных или статистиках принято использовать самое простое распределение, а именно равномерное (1). Поэтому и в работе мы будем считать, что вероятности поступления запроса по любому не выполняющемуся в данный момент контракту равны для любого момента времени. Т.е. поток запросов на контракты имеет дискретное равномерное распределение (Рисунок 22), где n – номер контракта.

$$f(x) = \begin{cases} \frac{1}{n} & x \in [a, b], x \in \mathbb{Z} \\ 0 & \text{иначе} \end{cases} \qquad (9)$$

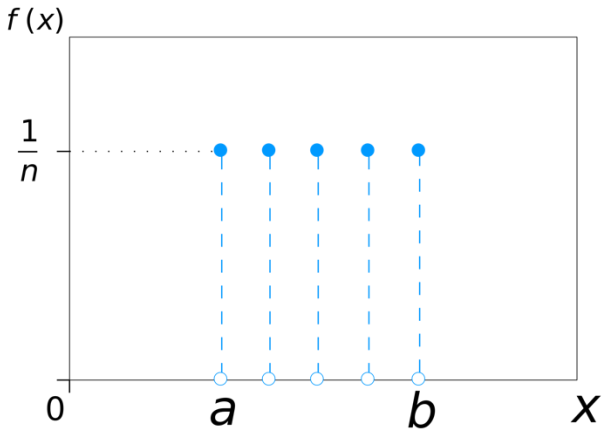

Рисунок 22. Плотность вероятности дискретного равномерного распределения

Чтобы приблизить характеристики контрактов, участвующих в эксперименте, к реальным, разделим их на три типа:

1. Первый тип контрактов соответствует большим объемам трафика, передаваемым за относительно большой промежуток времени. К такому трафику можно отнести, например, резервное копирование данных.
2. Второй тип контрактов соответствует чуть меньшим по размеру объемам трафика, передаваемых за короткое время. К этому виду можно отнести, например, миграцию виртуальных машин.
3. Третий тип контрактов соответствует небольшим объемам данных, передаваемых за небольшое время. К этому типу можно отнести обычный трафик.

На основании статей [79,80], были установлены (Таблица 5) следующие диапазоны значений для объема трафика и времени передачи каждого из трех типов.

Тип контракта	**Объем данных**	**Время передачи**
Резервное копирование	1-3 Гб	5-12 минут
Миграция виртуальных машин	300 Мбайт – 1 Гб	2-5 минут
Обычный трафик	10 Мбайт – 200 Мбайт	меньше 2 минут

Таблица 5. Численные характеристики контрактов

Для того чтобы подготовить фиксированное множество контрактов для проведения экспериментов необходимо было выбрать топологию и задать то количество контрактов каждого из типов, которое мы хотим видеть в итоговом множестве. В нашем случае была выбрана топология Bren и сгенерировано по 10 контрактов каждого типа. Итоговые 30 контрактов были записаны в файл и использовались в дальнейшем для проведения экспериментов. Между ними

разыгрывалось право быть выбранными в текущий момент в соответствии с равномерным распределением.

3.3.2 Результаты экспериментального исследования

Целью экспериментального исследования было найти максимальное количество контрактов, при котором вероятность удовлетворения всех запросов составляет 99% с уровнем значимости 5%. Для заданного количества контрактов проводился 2021 запуск с разными выборками старта запросов по контракту. Для достижения вероятности 99% удовлетворения заданного набора контрактов необходимо, чтобы 1994 запусков из 2021 будут успешными (т.е. удовлетворят все запросы на контракты). По принципу дихотомии был проведен поиск максимального количества контрактов, которое можно удовлетворить, начиная со значения в 50 контрактов.

Эксперименты для каждого нового значения количества контрактов проводились параллельно на 10 ядрах. Подсчет одного эксперимента на одном ядре для набора из 43 контрактов занимал по времени около 14 секунд и чуть больше 400Мбайт по памяти. При этом система $(*_2)$ состояла из 140 000 ограничений и имела около 320 ненулевых решений.

Результаты экспериментов приведены на рисунке 24. По вертикальной оси была отсчитана вероятность выполнения запросов на контракты (отношение удачного числа исходов к общему), по горизонтальной – количество проведенных испытаний. Наборы из разного числа контрактов отмечены разными цветами и обозначены на легенде в порядке изменения их количества. Нахождение решения заняло 6 часов 55 минут. Поскольку скорость изменения числа контрактов у провайдера не является быстро изменяемой величиной (например, динамика изменения числа клиентов у Интернет-провайдеров измеряется единицами процентов в год), то скорость нахождения решения рассматриваемой проблемы можно считать вполне приемлемой.

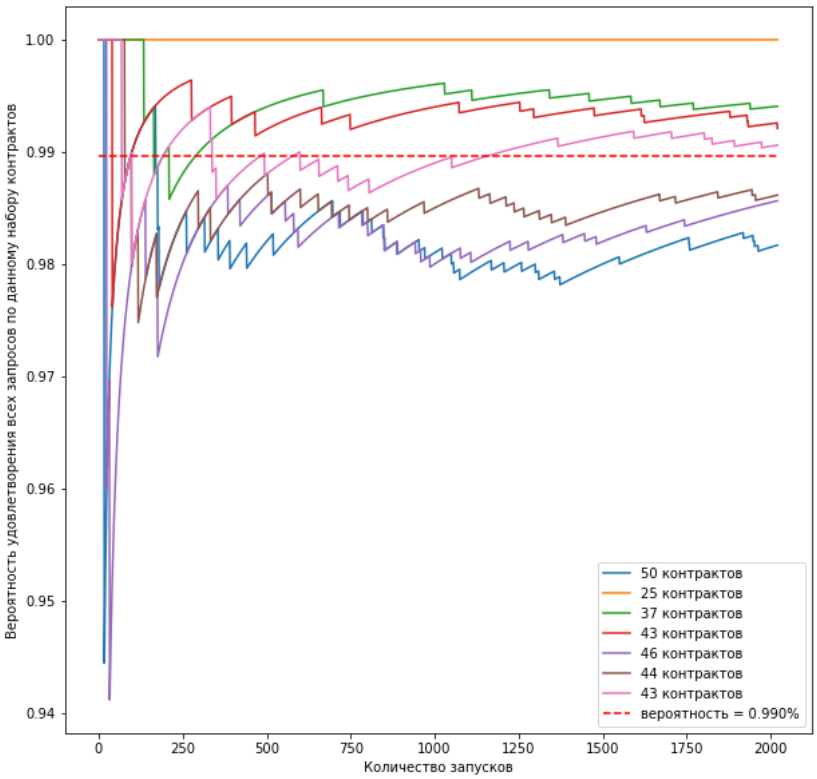

Рисунок 10. Результаты экспериментального исследования алгоритма расчета резерва, необходимого для реализации услуги ПСТ.

Заключение

Основные результаты диссертационной работы заключаются в следующем:

1. Исследован и реализован динамический многопоточный протокол для транспортных соединений, который позволяет повысить эффективность использования ресурсов сети не менее, чем в 95% случаев;

2. Для предложенного протокола разработан алгоритм построения маршрутов с минимальным числом пересечением. Для 91 топологии из рассмотренных 259 показано, что предложенный алгоритм дает больше маршрутов для родственных подпотоков, чем жадный алгоритм построения маршрутов родственных подпотоков. На основе разработанного алгоритма построения маршрутов предложен подход к организации многопоточной маршрутизации в ПКС сетях, который позволяет на порядок сократить количество OpenFlow правил в ПКС коммутаторах.

3. Проведен сравнительный анализ эффективности статического и динамического методов многопоточной маршрутизации транспортных соединений в ПКС сетях. На основе результатов исследования определены условия применения и оценки эффективности этих методов.

4. Разработан метод оценки резерва пропускной способности для реализации услуги «Пропускная способность по требованию» по заданным топологии и потокам запросов между Центрами обработки данных, который позволяет оценить максимальное количество пользователей услуги без потери качества сервиса.

Предложенный динамический многопоточный протокол может использоваться для реализации услуги «Пропускная способность по требованию».

Направления дальнейших исследований. Одним из путей развития исследований по тематике данной работы является рассмотрение следующих задач:

- Исследование влияния алгоритма управления перегрузкой и алгоритма планирования сегментов в транспортном агенте на эффективность динамического метода многопоточной маршрутизации.

- Адаптация предложенного динамического многопоточного протокола для подпотоков, работающих по протоколу UDP, что позволит использовать динамический метод многопоточной маршрутизации в современных протоколах наподобие QUIC.
- Исследование динамического метода многопоточной маршрутизации для других видов сетей.
- Реализация услуги «Пропускная способность по требованию», сбор статистических данных о частоте поступления заявок на услугу и уточнение предложенного подхода по оценке резерва.

Список литературы

[1] Cisco Annual Internet Report (2018-2023). [PDF] (https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.pdf).

[2] Data centers in Russia. [HTML] (https://cloudscene.com/market/data-centers-in-russia/all).

[3] Locations of Data Centers. [HTML] (https://www.datacenters.com/locations).

[4] Salah T. et al. The evolution of distributed systems towards microservices architecture //2016 11th International Conference for Internet Technology and Secured Transactions (ICITST). – IEEE, 2016. – С. 318-325.

[5] Смелянский Р. Л. Иерархические периферийные вычисления // Моделирование и анализ информационных систем. — 2019. — Т. 26, № 1. — С. 146–169.

[6] Zhang L. et al. Online electricity cost saving algorithms for co-location data centers //IEEE Journal on Selected Areas in Communications. – 2015. – Т. 33. – №. 12. – С. 2906-2919.

[7] Telecommunications market research that's data-driven, TeleGeography. [HTML] (https://www.telegeography.com/).

[8] Sinha S., Kandula S., Katabi D. Harnessing TCP’s burstiness with flowlet switching //Proc. 3rd ACM Workshop on Hot Topics in Networks (Hotnets-III). – 2004.

[9] Irawati I. D., Hadiyoso S., Hariyani Y. S. Link aggregation control protocol on software defined network //International Journal of Electrical and Computer Engineering. – 2017. – Т. 7. – №. 5. – С. 2706.

[10] Chiesa M., Kindler G., Schapira M. Traffic engineering with equal-cost-multipath: An algorithmic perspective //IEEE/ACM Transactions on Networking. – 2016. – Т. 25. – №. 2. – С. 779-792.

[11] Awduche D. et al. Requirements for traffic engineering over MPLS. – 1999. – №. rfc2702.

[12] Habib S. et al. The past, present, and future of transport-layer multipath //Journal of Network and Computer Applications. – 2016. – Т. 75. – С. 236-258.

[13] Fragouli C., Soljanin E. Network coding fundamentals. – Now Publishers Inc, 2007.

[14] Zou H. et al. Improving I/O performance with adaptive data compression for big data applications //2014 IEEE International Parallel & Distributed Processing Symposium Workshops. – IEEE, 2014. – С. 1228-1237.

[15] Agababov V. et al. Flywheel: Google's data compression proxy for the mobile web //12th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 15). – 2015. – С. 367-380.

[16] Jacobson V. Congestion avoidance and control //ACM SIGCOMM computer communication review. – 1988. – Т. 18. – №. 4. – С. 314-329.

[17] Grazia C. A. et al. BBR+: improving TCP BBR Performance over WLAN //ICC 2020-2020 IEEE International Conference on Communications (ICC). – IEEE, 2020. – С. 1-6.

[18] Padhye J. et al. Modeling TCP throughput: A simple model and its empirical validation //Proceedings of the ACM SIGCOMM'98 conference on Applications, technologies, architectures, and protocols for computer communication. – 1998. – С. 303-314.

[19] Stepanov E. P., Voynov N. A. Dynamic Split TCP //2020 International Scientific and Technical Conference Modern Computer Network Technologies (MoNeTeC). – IEEE, 2020. – P. 1-9.

[20] Gupta A., Shute J. High-availability at massive scale: Building google's data infrastructure for ads //Real-Time Business Intelligence and Analytics. – Springer, Cham, 2015. – С. 63-81.

[21] Chemeritskiy E., Smelansky R. On QoS management in SDN by multipath routing //2014 International Science and Technology Conference (Modern Networking Technologies)(MoNeTeC). – IEEE, 2014. – С. 1-6.

[22] Morawski M., Ignaciuk P. Influence of Congestion Control Algorithms on Head-of-Line Blocking in MPTCP-based Communication //2019 27th Telecommunications Forum (TELFOR). – IEEE, 2019. – С. 1-4.

[23] Raiciu C. et al. How hard can it be? designing and implementing a deployable multipath {TCP} //9th USENIX symposium on networked systems design and implementation (NSDI 12). – 2012. – С. 399-412.

[24] Ni D. et al. Fine-grained forward prediction based dynamic packet scheduling mechanism for multipath TCP in lossy networks //2014 23rd international conference on computer communication and networks (ICCCN). – IEEE, 2014. – С. 1-7.

[25] Ha S., Rhee I., Xu L. CUBIC: a new TCP-friendly high-speed TCP variant //ACM SIGOPS operating systems review. – 2008. – Т. 42. – №. 5. – С. 64-74.

[26] Khalili R. et al. MPTCP is not Pareto-optimal: Performance issues and a possible solution //IEEE/ACM Transactions On Networking. – 2013. – Т. 21. – №. 5. – С. 1651-1665.

[27] Peng Q. et al. Multipath TCP: Analysis, design, and implementation //IEEE/ACM Transactions on networking. – 2014. – Т. 24. – №. 1. – С. 596-609.

[28] Kato T. et al. mpCUBIC: A CUBIC-like Congestion Control Algorithm for Multipath TCP //World Conference on Information Systems and Technologies. – Springer, Cham, 2020. – С. 306-317.

[29] Kim G. H. et al. Adaptive Decrease Window for BALIA (ADW-BALIA): Congestion Control Algorithm for Throughput Improvement in Nonshared Bottlenecks //Electronics. – 2021. – Т. 10. – №. 3. – С. 294.

[30] Franklin M., Halevy A., Maier D. From databases to dataspaces: a new abstraction for information management //ACM Sigmod Record. – 2005. – Т. 34. – No. 4. – С. 27-33.

[31] Mahimkar A. et al. Bandwidth on demand for inter-data center communication //Proceedings of the 10th ACM Workshop on Hot Topics in Networks. – 2011. – С. 1-6.

[32] Zhang L. et al. RSVP: A new resource reservation protocol //IEEE network. – 1993. – Т. 7. – №. 5. – С. 8-18.

[33] Multipath Experimentator Source Code [HTML] (https://github.com/estepanov-lvk/multipath_experimentator).

[34] Степанов Е. П., Смелянский Р. Л. Сравнительный анализ многопоточных транспортных протоколов // Системы и средства информатики. — 2022. — Т. 32, № 2. — С. 155–170.

[35] Степанов Е. П. Анализ эффективности демультиплексирования транспортных потоков // Моделирование и анализ информационных систем. — 2019. — Т. 26, № 1. — С. 170–190.

[36] Stepanov E. P., Smeliansky R. L. On bandwidth on demand problem // Proceedings of the 27th International Symposium Nuclear Electronics and Computing (NEC'2019). — Vol. 2507. — CEUR-WS Budva, Montenegro, 2019. — P. 402–407.

[37] Smeliansky R. L., Antonenko V. A., Stepanov E. P. et al. On hpc & cloud environments integration. chapter 1 // Performance evaluation models for distributed service networks. — Springer Nature Switzerland AG Gewerbestrasse 11, 6330 Cham, Switzerland, 2020.

[38] Sinyakova M. A., Stepanov E. P., Kukushkin D. S. et al. Slicer: A network hypervisor with an automatic mapping of virtual network topology //2020 International Scientific and Technical Conference Modern Computer Network Technologies (MoNeTeC). – IEEE, 2020. – P. 1-10.

[39] Степанов Е. П., Смелянский Р. Л. Эффективность демультиплексирования транспортных соединений: анализ проблемы // Ломоносовские чтения 2019, секция Вычислительной математики и кибернетики, сб тезисов докладов. — Макс-Пресс Москва Москва, 2019. — С. 104–105.

[40] Степанов Е. П. Методика исследования эффективности динамического демультиплексирования // Ломоносовские чтения 2020. Секция Вычислительной математики и кибернетики. — Секция Вычислительной математики и кибернетики. — М.: М., 2020. — С. 141–143.

[41] Степанов Е. П., Смелянский Р. Л. О массовой многопоточной передаче данных // Ломоносовские чтения 2018 ф-т ВМК МГУ. — Макс-Пресс, 2018. — С. 111–112.

[42] Степанов Е. П., Чемерицкий Е. В. Средство адаптивной балансировки нагрузки на уровне транспортного соединения. ФИПС. Св. о рег. программы для ЭВМ № 2017615377 от 15.05.2017

[43] ARCCN NEWS (2021) Demonstration of the Bandwidth on Demand application based on the SDN controller RunOS [демонстрация работы приложения] // YouTube. 4 мая. (https://www.youtube.com/watch?v=TuzDDZT4NL4). Просмотрено: 21.04.2022

[44] Xing Y. et al. MPTCP Meets Big Data: Customizing Transmission Strategy for Various Data Flows //IEEE Network. – 2020. – Т. 34. – №. 4. – С. 35-41.

[45] Khan I., Chen K. EBA: Efficient Bandwidth Aggregation for Connected Vehicles with MPTCP //IEEE Internet of Things Journal. – 2021.

[46] Zhang T., Zhao S., Cheng B. Multipath Routing and MPTCP-Based Data Delivery Over Manets //IEEE Access. – 2020. – Т. 8. – С. 32652-32673.

[47] Morawski M., Ignaciuk P. A Price to Pay for Increased Throughput in MPTCP Transmission of Video Streams //2020 24th International Conference on System Theory, Control and Computing (ICSTCC). – IEEE, 2020. – С. 673-678.

[48] Palash M. R., Chen K., Khan I. Bandwidth-need driven energy efficiency improvement of MPTCP users in wireless networks //IEEE Transactions on Green Communications and Networking. – 2019. – Т. 3. – №. 2. – С. 343-355.

[49] De Coninck Q., Bonaventure O. Multipath quic: Design and evaluation //Proceedings of the 13th international conference on emerging networking experiments and technologies. – 2017. – С. 160-166.

[50] Benvenuti C. Understanding Linux Network Internals. - O'Reilly Media, 2005. - 1066 с.

[51] Understanding MPEG-4: Technologies, Advantages, and Markets. - An MPEGIF White Paper, 2005. - 44 с.

[52] The Internet Topology Zoo [HTML] (http://topology-zoo.org/).

[53] Смелянский Р. Настоящее и будущее SDN&NFV //Первая миля. – 2016. – №. 3. – С. 78-85.

[54] Kukreja N. et al. SDN based automated testbed for evaluating multipath TCP //2016 IEEE International Conference on Communications Workshops (ICC). – IEEE, 2016. – С. 718-723.

[55] Nakasan C. et al. A simple multipath OpenFlow controller using topology-based algorithm for multipath TCP //Concurrency and Computation: Practice and Experience. – 2017. – Т. 29. – №. 13. – С. 34-41.

[56] Dijkstra E. W. et al. A note on two problems in connexion with graphs //Numerische mathematik. – 1959. – Т. 1. – №. 1. – С. 269-271.

[57] Enyedi G., Rétvári G. Finding multiple maximally redundant trees in linear time //Periodica Polytechnica Electrical Engineering (Archives). – 2010. – Т. 54. – №. 1-2. – С. 29-40.

[58] Zehavi A., Itai A. Three tree-paths //Journal of Graph Theory. – 1989. – Т. 13. – №. 2. – С. 175-188.

[59] Curran S., Lee O., Yu X. Finding four independent trees //SIAM Journal on Computing. – 2006. – Т. 35. – №. 5. – С. 1023-1058.

[60] Pfaff B. et al. OpenFlow Switch Specification—1.3 Version //Open Networking Foundation: Menlo Park, CA, USA. – 2012.

[61] NoviFlow Inc. 2021. NoviSwitch 2122 Datasheet (2019). Retrieved "May 17, 2021" from https://noviflow.com/wp-content/uploads/2019/11/NoviSwitch-2122-Datasheet-400_V5.pdf

[62] Shalimov A. et al. The runos openflow controller //2015 Fourth European Workshop on Software Defined Networks. – IEEE, 2015. – С. 103-104.

[63] Heinanen J. et al. Assured forwarding PHB group. – RFC 2597, june, 1999. – Т. 470. – С. 471-472.

[64] Le Boudec J. Y. Rate adaptation, congestion control and fairness: A tutorial //Web page, November. – 2005. – Т. 4.

[65] Langley A. et al. The quic transport protocol: Design and internet-scale deployment //Proceedings of the Conference of the ACM Special Interest Group on Data Communication. – ACM, 2017. – С. 183-196.

[66] Arfeen M. A. et al. The role of the weibull distribution in internet traffic modeling //Proceedings of the 2013 25th International Teletraffic Congress (ITC). – IEEE, 2013. – С. 1-8.

[67] Choose live encoder settings, bitrates and resolution. Google. [HTML] (https://support.google.com/youtube/answer/2853702?hl=en)

[68] Ping-Qi P. A. N. Linear programming computation. – Springer Berlin Heidelberg, 2014

[69] Корбут А. А., Финкельштейн Ю. Ю. Дискретное программирование. – Наука. Гл. ред. физ.-мат. лит., 1969.

[70] Дунаев Д. Н. Виды задач линейного программирования, способы их решения //Наука и современность. – 2013. – №. 21. – С. 121-125.

[71] Банди Б. Б23 Основы линейного программирования: Пер. с англ. - М.: Радио и связь, 1989. – 176 с.

[72] Мину М. Математическое программирование: теория и алгоритмы: Пер. с фр. – Наука. Гл. ред. физ.-мат. лит., 1990.

[73] “NetworkX documentation.” [HTML] (https://networkx.org/documentation/stable/index.html).

[74] Fuchs E., Jackson P. E. Estimates of distributions of random variables for certain computer communications traffic models //Proceedings of the first ACM symposium on Problems in the optimization of data communications systems. – 1969. – С. 205-230.

[75] Agamy A., Mohamed A. M., Ali A. M. Review on Traffic Modeling of Wireless Access Networks LTE &WiFi technologies

[76] Gerlough D. L., Schuhl A. Use of Poisson distribution in highway traffic. – Saugatuck, Conn. : Eno Foundation for Highway Traffic Control, 1955. – С. 1-58.

[77] Karlis D., Xekalaki E. Mixed poisson distributions //International Statistical Review/Revue Internationale de Statistique. – 2005. – С. 35-58.

[78] Jongbloed G., Koole G. Managing uncertainty in call centres using Poisson mixtures //Applied Stochastic Models in Business and Industry. – 2001. – Т. 17. – №. 4. – С. 307-318.

[79] Dargie W. Estimation of the cost of VM migration //2014 23rd International Conference on Computer Communication and Networks (ICCCN). – IEEE, 2014. – С. 1-8.

[80] Medina V., García J. M. A survey of migration mechanisms of virtual machines //ACM Computing Surveys (CSUR). – 2014. – Т. 46. – №. 3. – С. 1-33.

[81] Балашов В.В Диссертация "Обеспечение совместимости требований к расписанию обмена по каналу с централизованным управлением-// Московский Государственный университет имени М.В. Ломоносова. Москва. 2010

[82] Гмурман В. Е. Теория вероятностей и математическая статистика. – М.: Высшая школа, 2003. – 479 с.

[83] Калинина В. Н., Панкин В. Ф. Математическая статистика. – М.: Высшая школа, 1998. – 336 с.

[84] Кремер Н. Ш. Теория вероятностей и математическая статистика. – М.: Юнити, 2004. – 573 с.

[85] Чистяков В. П., Курс теории вероятностей. Наука, 1978.

[86] OpenVSwitch [HTML] (http://www.openvswitch.org).

[87] Iperf3 – the ultimate speed test tool for TCP, UDP and SCTP [HTML] (https://iperf.fr/).

[88] Multipath TCP версии 0.92 [HTML] (https://github.com/multipath-tcp/mptcp/tree/mptcp_v0.92)

[89] SDN Controller Runos [HTML] (http://github.com/arccn/runos).

[90] Fabric high level Python library [HTML] (http://www.fabfile.org/).

[91] “COIN-OR CBC documentation.” https://github.com/coin-or/Cbc.git (accessed May 13, 2021).

[92] “GNU Linear Programming Kit Reference Manual for GLPK Version 4.55,” 2000. [HTML]. (http://www.chiark.greenend.org.uk/doc/glpk-doc/glpk.pdf).

[93] “Python MIP documentation.” [HTML] (https://docs.python-mip.com/en/latest/).

[94] “Pyomo main page.” [HTML] (http://www.pyomo.org/).

Приложение А. Методика проверки статистической гипотезы о числовом значении вероятности события

Общая схема проверки гипотезы H_0 о попадании случайной величины ξ в интервал $[\xi_{min}, \xi_{max}]$ с заданной вероятностью p_0 проводится аналогично работе [81], более подробно схема описана и обоснована в [82, 83].

1) Зададим уровень значимости или, что то же самое, вероятность ошибки первого рода α равным некоторому значению. Его не стоит выбирать очень маленьким, так как это может привести к увеличению вероятности ошибки второго рода, то есть вероятности принять нулевую гипотезу, когда на самом деле она не верна. После выбора конкретного α можно утверждать, что выдвинутая гипотеза H_0 будет принята с вероятностью $1 - \alpha$.
2) Определим величину $\hat{p} = \frac{m}{n}$, где m – число испытаний, при которых $\xi \in [\xi_{min}, \xi_{max}]$, а n – число всех испытаний. Также введем случайную величину ψ, которая равная 1 в случае $\xi \in [\xi_{min}, \xi_{max}]$ и 0 иначе.
3) Если гипотеза H_0 верна, то математическое ожидание $\hat{p}$ равно p_0, и поскольку испытания для ψ являются испытаниями Бернулли, то при достаточно больших n ,согласно интегральной теореме Муавра-Лапласа, распределение для $\hat{p}$ близко к нормальному с дисперсией $\frac{p_0*(1-p_0)}{n}$. Если далее мы введем случайную величину $U = (\hat{p} - p_0) * (\sqrt{\frac{p_0*(1-p_0)}{n}})^{-1}$, то она будет иметь нормальное распределение с нулевым математическим ожиданием и единичной дисперсией.
4) Таким образом мы получаем, что исходной гипотезе $H_0: P(\xi \in [\xi_{min}, \xi_{max}]) = p_0$ в терминах случайной величины U будет соответствовать гипотеза $\mathbb{E}(U) = 0$. Альтернативными гипотезами будут являться:
   - $H_1: P(\xi = 1) < p_0$ и соответствующая ей гипотеза $H_1': \mathbb{E}(U) < 0$
   - $H_2: P(\xi = 1) > p_0$ и соответствующая ей гипотеза $H_2': \mathbb{E}(U) > 0$
5) Будем проверять гипотезу H_0 против альтернативной гипотезы H_1 с уровнем значимости α. Для этого применим к гипотезам H_1' и H_2' аппарат проверки

гипотез о значении математического ожидания для нормального распределения с известной дисперсией. Так как в случае верности альтернативной гипотезы $\mathbb{E}(U) < 0$, то критической областью для проверки является интервал $(-\infty, x_{\text{лев},\alpha}^{\text{кр}})$, где $x_{\text{лев},\alpha}^{\text{кр}}$ находится исходя из $P\left(N(0,1) < x_{\text{лев},\alpha}^{\text{кр}}\right) = \alpha$. Для вычисления значения величины $x_{\text{лев},\alpha}^{\text{кр}}$ используется функция Лапласа $\Phi(u) = \sqrt{\frac{2}{\pi}}\int_0^u e^{\frac{-t^2}{2}}dt$. Порядок нахождения $x_{\text{лев},\alpha}^{\text{кр}}$ следующий: изначально находится значение $\gamma = 1 - 2\alpha$, далее u_γ исходя из табличных значений $\Phi\left(u_\gamma\right) = \gamma$ [83] и наконец $x_{\text{лев},\alpha}^{\text{кр}} = -u_\gamma$.

6) Далее описывается критерий $\varphi = \frac{(\hat{p} - p_0)}{\sqrt{\frac{p_0 * (1-p_0)}{n}}}$, согласно которому если $\varphi > x_{\text{лев},\alpha}^{\text{кр}}$, то гипотеза H_0 не отвергается в пользу H_1.
7) Если по результатам проверки в п.6 гипотеза H_0 не отвергается в пользу H_1, то либо верна H_0 и $P(\xi = 1) = p_0$, либо верна H_2 и $P(\xi = 1) > p_0$. Т.е. в этом случае статистически обоснована выполнимость гипотезы H_{02}: $P(\xi = 1) \geq p_0$.

В различных источниках даны различные оценки минимального объема выборки n, при котором случайная величина $\hat{p}$ может быть рассмотрена как нормально распределенная.

- $n * p_0 * (1 - p_0) > 9$ [82]
- $n * p_0 > 10,\ n > 100$ [83]
- $n * p_0 * (1 - p_0) \geq 20$ [84]
- $n * p_0 * (1 - p_0) > 20, n > 100$ [85]

В данной работе будет использовать наиболее строгую оценку, а именно, приведенную в работе [85].

Приложение Б. Технические подробности реализации Стенда для проведения сравнительного анализа эффективности методов многопоточной маршрутизации

Опишем детали реализации подсистем Стенда, описанного в разделе 2.2:

- Подсистема Topology реализована на языке Python3 с использованием программных OpenFlow коммутаторов OpenVSwitch [86] и виртуальных линий связи VETH, поддерживаемых операционной системой на базе ядра Linux.
- Подсистема QoS реализована на языке Python3 с использованием внешней утилиты tc (traffic control), позволяющей настраивать параметры виртуальных линий связи VETH.
- Подсистема Multiloader реализована на языке Python3 с использованием программного OpenFlow коммутатора и виртуальных линий связи VETH. Параметры качества сервиса не настраиваются на линиях связи, ведущих к имитационной модели сети, что позволяет передать данные до имитационной модели с максимальной для операционной системы скоростью, чтобы исследовать эффективность многопоточных протоколов внутри сети Интернет-провайдера.
- Подсистема Iperf3 реализована на языке C и основана на одноименной утилите [87], в которую был добавлена функциональность старта потоков согласно заданному расписанию.
- Подсистема FDMP agent реализован на языке C как часть ядра операционной системы на базе Linux версии 4.14.91.
- Реализация подсистемы MPTCP agent взята из открытого репозитория [88].
- Подсистема MP App реализована на языке C++ как пользовательское приложение отечественного контроллера ПКС Runos [89], поддерживающего работу по протоколу OpenFlow версии 1.3 [60].

- Подсистема экспериментатор реализована на языке Python3. Так как остальные подсистемы могут находиться на отдельных серверах/виртуальных машинах, то для обращения одной подсистемы к другой используется библиотека fabric [90] для удаленного вызова методов, которые реализуют интерфейсы вышеописанных подсистем.

Аппаратная часть Стенда представлена на рисунке Б.1. Стенд состоит из 4 серверов: w1, w2, w3, head. Сервера w1, w2, w3 имеют процессор Intel Xeon E5-2667 v4 (8 ядер, базовая частота 3,20 GHz), 32 ГБ оперативной памяти. Сервер head имеет два процессора Intel Xeon E5-2650 v4 (всего 24 ядра, базовая частота 2,20 GHz), 64 ГБ оперативной памяти. Сервер head соединен с серверами w1 и w3 четырьмя каналами с пропускной способностью 10Гбит/с, а с сервером w2 одним каналом с пропускной способностью 10Гбит/с.

На серверах w1, w3 расположены по 4 виртуальных машины с FDMP агентом/MPTCP агентом и Iperf3. Эти сервера отвечают за генерацию потоков данных, который отправляется в сеть согласно многопоточному протоколу FDMP или MPTCP. Трафик от разных виртуальных машин направляется к серверу head через разные линии связи.

На сервере head работают подсистемы Topology, QoS, Multiloader, а также контроллер ПКС Runos с приложением MP App. Сервер head отвечает за имитационное моделирование сети Интернет-провайдера.

На сервере w2 работает подсистема Experimentator, которая отвечает за запуск экспериментов и сбор результатов.

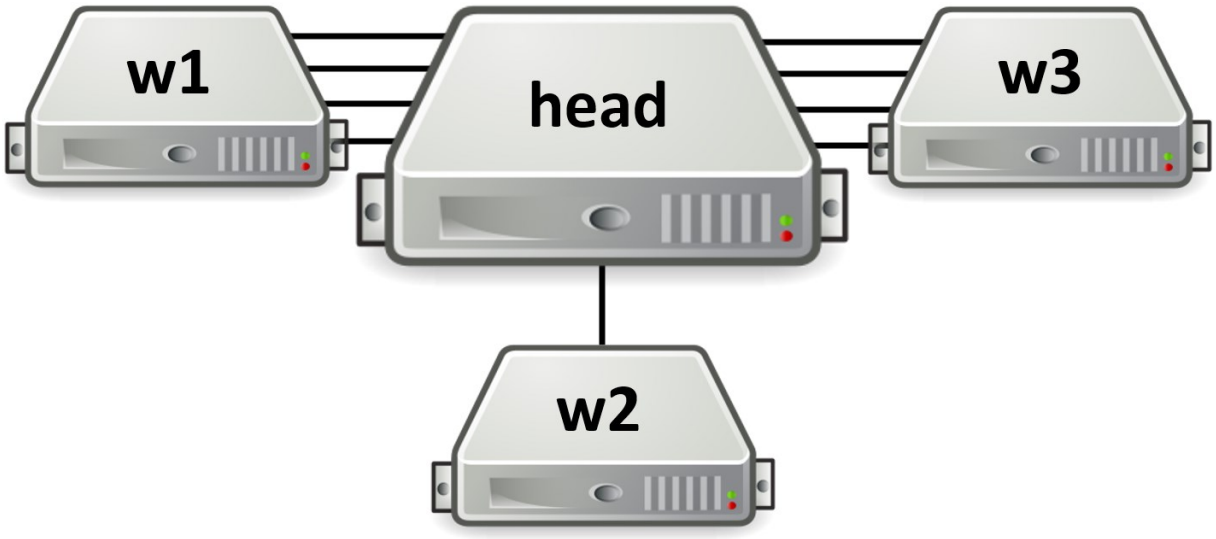

Рисунок Б.1. Схема аппаратной части экспериментального стенда.

Приложение В. Описание пакетов и их решателей для решения задачи целочисленного линейного программирования

COIN-OR CBC

COIN-OR Branch and Cut solver (CBC) – решательдля задач ЦЛП, названный так в честь метода, который он для этого использует (branch and cut комбинация метода ветвей и границ и метода секущих плоскостей [69]). Код CBC написан на языке C++ и находится в открытом доступе [91]. CBC подразумевает, что пользователь, задающий задачу, может сам настраивать параметры некоторых шагов алгоритма в процессе решения (например, выбор следующего узла при ветвлении) и даже практически полностью контролировать его выполнение. Также для работы с ним пользователю необходимы знания C++ и OSI – общего для всех проектов инфраструктуры COIN-OR интерфейса. Что касается скорости вычисления, CBC предоставляет возможность распараллеливать вычисления алгоритма (например, искать решение по нескольким ветвям одновременно).

GLPK

GLPK (сокращенно от GNU Linear Programming Kit) – это решатель, разработанный для решения высоко размерных задач линейного программирования и задач целочисленного линейного программирования. В качестве метода для решения последних, он использует сочетание метода Гомори с методом ветвей и границ (вышеупомянутый branch and cut [69]). GLPK не содержит каких либо средств, позволяющих увеличить скорость вычисления задачи. В качестве языка для описания задачи пользователь может использовать GNU MathProg, LP, MPS, либо интерфейс к нему для языка Java. Ему также свойственно главное качество решателей: пользователь сам может управлять параметрами алгоритма, решающего задачу (более подробно методы, используемые для этого, описаны здесь [92]).

MIP Python

Python MIP это пакет, используемый для решения задач ЦЛП с использованием языка python. Его можно установить так же, как обычную питоновскую библиотеку через pip, задать описание задачи на языке python, используя несколько специализированных функций этого пакета и далее решать задачу одним из предложенных решателей: коммерческим решателем GUROBI, либо бесплатным COIN-OR CBC. При этом пользователю не нужно разбираться в тонкостях алгоритма, решающего задачу, т. к. пакет самостоятельно подбирает коэффициенты, наиболее подходящие для решения задачи. Также можно отметить, что этот пакет устанавливается на вычислительное устройство уже совместно с решателем CBC. В дополнение этому, к плюсам можно отнести и то, что Python MIP напрямую вызывает встроенную динамически загружаемую библиотеку выбранного решателя с использованием C Foreign Function Interface для python, за счет чего решатель может эффективно хранить и оптимизировать модель, что ведет к уменьшению времени вычисления. Полная документация по этому пакету доступна здесь [93].

Pyomo

Pyomo [94] это еще один пакет, использующий в качестве интерфейса язык python, созданный для решения разного рода оптимизационных задач. Его используют множество решателей, среди которых задачи ЦЛП решают COIN-OR CBC, GLPK и GUROBI, CPLEX. Пакет Pyomo также содержит в себе компоненту, оптимизирующую построенную модель для каждого конкретного решателя и позволяющую быстро находить решение для задач больших размерностей. Также к плюсам, касательно скорости выполнения можно отнести поддержку параллельного программирования на стадии вычисления решения и параллельную декомпозицию структурированных задач. Сам пакет был разработан в 2008 году и является, как и решатель COIN-OR CBC одним из проектов COIN-OR.

Итоги

Стоит отметить, что создателями ни одного из решателей или пакетов не было предоставлено каких-либо подробных данных относительно точности полученного решения. Самое большое, что сообщают создатели таких программных средств это общие фразы вида «данный пакет хорошо подходит для решения задач высокой размерности». То же самое можно сказать об оценке скорости нахождения решения – она будет зависеть от параметров самой задачи, от характеристик вычисляющего устройства и от реализации самого решения. Именно поэтому рассматривается лишь возможность проводить параллельные вычисления, что увеличивает скорость вычисления любой задачи.